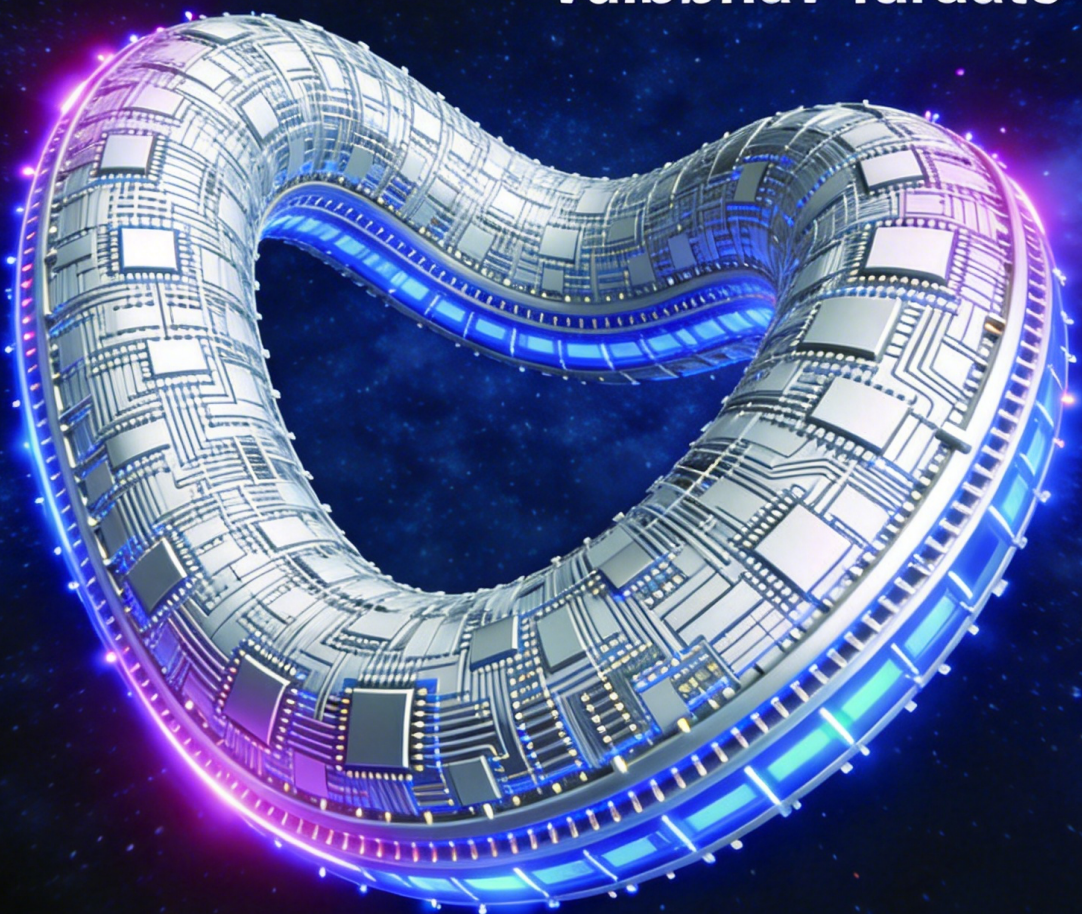
ASIC Design and Synthesis

RTL Design Using Verilog

Vaibhav Taraate



Vaibbhav Taraate

ASIC Design and Synthesis

RTL Design Using Verilog

Springer

Vaibbhav Taraate
1 Rupee S T
Pune, Maharashtra, India

ISBN 978-981-33-4641-3 ISBN 978-981-33-4642-0 (eBook)

© Springer Nature Singapore Pte Ltd. 2021
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Dedicated to my inspiration and my friend who
wish to create many entrepreneurs in VLSI design

Late Ajit Shelat

Ajit unfortunately passed away on 1st September 2010 in car accident. Due to wish of Ajit I have continued work in ASIC and VLSI design.

Preface

The complexity of the ASIC designs has grown exponentially during the past decade, and during this decade, we are experiencing the AI/ML-based designs and AI-based processor cores to improve the performance of the designs.

The book is the origin of my thought process, and I have tried to document the design concepts and practical issues with their solutions in this book.

The book mainly covers the ASIC design concepts, semi-custom ASIC design flow and the case studies which can be helpful to the postgraduates and professionals. The book uses the Synopsys DC and PT commands and their use during the synthesis and timing closure.

The physical design flow with the basic steps are covered so that readers can have better understanding about the overall ASIC design cycle.

The book is organised in 20 chapters and covers the basics of ASIC design and the concepts used during the RTL design to GDSII flow.

Chapter 1: **Introduction**: Understanding of the ASIC and programmable ASIC plays an important role for the beginners and the experience engineers. The flow can be full-custom, semi-custom, or programmable ASIC, and the major objective of an engineer is to understand the design steps to plan the milestone delivery. The objective of this chapter is to have the basic understanding of ASICs and what should be the focus of the design team.

Chapter 2: **ASIC Design Flow**: The chapter discusses about the ASIC design flow with few of the examples. The chapter is useful to understand about the logic design (frontend design) flow, physical design (backend design) flow and the design flow for the programmable ASICs.

Chapter 3: **Let Us Build Design Foundation**: Understanding of the design abstraction at various levels is always useful during the RTL design and synthesis phase. In this context, the chapter discusses about few of the elements and their use during the design. Even the chapter discusses about the area improvement techniques and role of the design elements in the ASIC design.

Chapter 4: **Sequential Design Concepts**: The objective of this chapter is to have discussion on the synchronous sequential circuits and asynchronous designs and their use during the design phase. For better understanding, the chapter discusses about the sequential elements and their use during design cycle.

Chapter 5: **Important Design Considerations**: The chapter is useful to understand about the basics of timing, skew, latency, and other design considerations such as parallelism and concurrency.

Chapter 6: **Important Considerations for ASIC Designs**: The chapter discusses about these techniques which are useful during the ASIC design architectures and micro-architectures.

Chapter 7: **Multiple Clock Domain Designs**: The chapter discusses about the multiple clock domain designs and strategies which can be useful during the architecture and micro-architecture design.

Chapter 8: **Low Power Design Considerations**: The chapter is useful to understand about the low power design techniques and important strategies which are useful during the ASIC design.

Chapter 9: **Architecture and Micro-architecture Design**: The chapter discusses about the architecture and micro-architecture design concepts and strategies which can be useful during the ASIC design phase.

Chapter 10: **Design Constraints and SDC Commands**: The chapter discusses about the design constraints and the important SDC commands. SDC stands for the Synopsys Design Constraints which is format and used to specify the design intent, including timing, power and area constraints for a design!.

Chapter 11: **Design Synthesis and Optimization Using RTL Tweaks**: The chapter discusses about the ASIC and FPGA synthesis and important concepts useful during the design optimization ad even during RTL design phase.

Chapter 12: **Synthesis and Optimization Techniques**: The chapter is useful to understand the different optimization techniques used during logic synthesis and use of Synopsys DC commands while optimizing the design.

Chapter 13: **Design Optimization and Scenarios**: The chapter discusses about the optimization for the speed and area with the practical design scenarios.

Chapter 14: **Design for Testability**: The DFT and the testability basics for the ASIC design are discussed in this chapter.

Chapter 15: **Timing Analysis**: The STA and the performance improvement techniques are discussed in this chapter.

Chapter 16: **Physical Design**: The chapter discusses about the physical design flow and important issues during the physical design and how to overcome them.

Chapter 17: **Case Study: Processor ASIC Implementation**: The chapter discusses about the overall strategies which are useful during RTL to GDSII for the moderately complex processors. Even the chapter discusses about the performance improvement and the processor architecture strategies with and without pipelining stages.

Chapter 18: **Programmable ASIC**: The FPGA and the role in prototyping is discussed in this chapter. Even the chapter is useful to understand about the FPGA flow and the FPGA synthesis.

Chapter 19: **Prototyping Design**: The design prototyping and the strategies are discussed in this chapter. The use of multi-FPGA architecture, use of the multiple FPGAs during prototyping and the prototyping flow is also discussed in this chapter!

Chapter 20: **Case Study: IP Design and Development**: The IP development and the strategies are discussed in this chapter. The H.264 architecture design and strategies to implement the design is also included in this chapter!

The book is useful to understand the ASIC design flow and the important design concepts useful during the various phases from the architecture design to layout of the chip.

Pune, India Vaibbhav Taraate

Acknowledgements

The book is originated due to my extensive work in FPGA and ASIC design from year 2000. The journey to develop the algorithms and architectures will continue in future also and will be helpful to many professionals and engineers.

This book is possible due to help of many people. I am thankful to all the participants to whom I taught the subject FPGA and ASIC design, synthesis, and timing closure in few multinational corporations. I am thankful to all those entrepreneurs, design/verification engineers, and managers with whom I worked in the past almost around 20 years.

I am thankful to all my dearest friends and well-wishers for their constant support. Especially, I am thankful to my teammates and all my family members for their support and cooperation!.Thankful to Niraj and Deepesh for their support and cooperation during the manuscript completion phase!

Finally, I am thankful to Springer Nature staff, especially Swati Meherishi, Ashok Kumar, Rini Christy, and Jayarani for their great support during the various phases of the manuscript.

Special thanks in advance to all the readers and engineers for buying, reading, and enjoying this book!

Contents

1 Introduction 1
- 1.1 ASIC Design 2
- 1.2 Types of ASIC 2
- 1.3 Abstraction Levels 6
- 1.4 Design Examples 9
- 1.5 What We Should Know? 9
- 1.6 Important Terms Used Throughout Design Cycle 11
- 1.7 Chapter Summary 12

2 ASIC Design Flow 13
- 2.1 ASIC Design Flow 13
  - 2.1.1 Logic Design 16
  - 2.1.2 Physical Design 20
- 2.2 FPGA Design Flow 22
- 2.3 Examples and Thought Process 24
- 2.4 Design Challenges 25
- 2.5 Chapter Summary 25

3 Let Us Build Design Foundation 27
- 3.1 Combinational Design Elements 27
- 3.2 Logic Understanding and Use of Construct 28
- 3.3 Arithmetic Resources and Area 29
- 3.4 Code Converter 31
  - 3.4.1 Binary to Gray Code Converter 31
  - 3.4.2 Gray to Binary Code Converter 33
- 3.5 Multiplexers 35
- 3.6 Cascading Stages of MUX Using Instantiation 38
- 3.7 Decoders 40
- 3.8 Encoders 43
- 3.9 Priority Encoders 43

3.10 Strategies During ASIC Design 46
3.11 Exercises 47
3.12 Chapter Summary 47

4 Sequential Design Concepts 49
4.1 Sequential Design Elements 49
4.2 Let Us Understand Blocking Versus Non-blocking Assignments 50
4.2.1 Blocking Assignments 51
4.2.2 Reordering of the Blocking Assignments 51
4.2.3 Non-blocking Assignments 53
4.2.4 Reordering of the Non-blocking Assignments 54
4.3 Latch-Based Designs 55
4.4 Flip-Flop-Based Designs 58
4.5 Reset Strategies 60
4.5.1 Asynchronous Reset 61
4.5.2 Synchronous Reset 64
4.6 Frequency Divider 65
4.7 Synchronous Design 68
4.8 Asynchronous Design 70
4.9 RTL Design and Verification for Complex Designs 70
4.10 Exercises 71
4.11 Chapter Summary 72

5 Important Design Considerations 73
5.1 Timing Parameters 74
5.2 Metastability 75
5.3 Clock Skew 75
5.3.1 Positive Clock Skew 77
5.3.2 Negative Clock Skew 79
5.4 Slack 80
5.4.1 Setup Slack 80
5.4.2 Hold Slack 80
5.5 Clock Latency 80
5.6 Area for the Design 81
5.7 Speed Requirements 81
5.8 Power Requirements 82
5.9 What Are Design Constraints? 83
5.10 Exercises 83
5.11 Chapter Summary 84

6 Important Considerations for ASIC Designs 85
6.1 Synchronous Design and Considerations 85
6.2 Positive Clock Skew and Impact on Speed 86
6.3 Negative Clock Skew and Impact on the Speed 88

6.4 Clock and Network Latency . . . 89
6.5 Timing Paths in the Design . . . 89
6.5.1 Input to Reg Path . . . 90
6.5.2 Reg to Output Path . . . 90
6.5.3 Reg to Reg Path . . . 91
6.5.4 Input to Output Path . . . 91
6.6 Frequency Calculations . . . 91
6.7 What Is On-Chip Variations . . . 93
6.8 Exercises . . . 94
6.9 Chapter Summary . . . 94

7 Multiple Clock Domain Designs . . . 97
7.1 General Strategies for Multiple Clock Domain Designs . . . 97
7.2 What Are Issues in the Multiple Clock Domain Design . . . 98
7.3 Architecture Design Strategies . . . 99
7.4 Control Path and Synchronization . . . 102
7.4.1 Level or Multiflop Synchronizer . . . 102
7.4.2 Pulse Synchronizers . . . 106
7.4.3 MUX Synchronizer . . . 106
7.5 Challenges in the Multiple Bit Data Transfer . . . 106
7.6 Data Path Synchronizers . . . 108
7.6.1 Handshaking Mechanism . . . 108
7.6.2 FIFO Synchronizer . . . 110
7.6.3 Gray Encoding . . . 111
7.7 Summary and Future Discussions . . . 111

8 Low Power Design Considerations . . . 113
8.1 Introduction to Low Power Design . . . 113
8.2 Sources of Power . . . 114
8.3 Power Optimization During the RTL Design . . . 116
8.4 Switching and Leakage Power Reduction Techniques . . . 121
8.4.1 Clock Gating and Clock Tree Optimizations . . . 122
8.4.2 Operand Isolations . . . 122
8.4.3 Multiple V_{th} . . . 123
8.4.4 Multiple Supply Voltages (MSV) . . . 123
8.4.5 Dynamic Voltage and Frequency Scaling (DVSF) . . . 123
8.4.6 Power Gating (Power Shut Off) . . . 123
8.4.7 Isolation Logic . . . 124
8.4.8 State Retention . . . 124
8.5 Low-Power Design Architecture and Use of UPF . . . 124
8.6 Chapter Summary . . . 127

9 Architecture and Micro-architecture Design . . . 129
9.1 Architecture Design . . . 129
9.2 Micro-architecture Design . . . 131

9.3 Use of Document During Various Design Phases 131
9.4 Design Partitioning 132
9.5 Multiple Clock Domains and Clock Grouping 132
9.6 Architecture Tweaking and Performance Improvement 133
9.7 Strategies for the Micro-architecture Design of Processor 134
9.8 Chapter Summary 138

10 Design Constraints and SDC Commands 139
10.1 Important Design Concepts 140
10.1.1 Clock Tree 140
10.1.2 Reset Tree 140
10.1.3 Clock and Reset Strategies 141
10.1.4 What Impacts on Design Performance? 141
10.2 How to Interpret the Constraints 142
10.2.1 Area Constraints 142
10.2.2 Speed Constraints 142
10.2.3 Power Constraints 143
10.3 Issues in the Design 143
10.4 Important SDC Commands Used During Synthesis 143
10.4.1 Synopsys DC Commands 144
10.4.2 Checking of the Design 145
10.4.3 Clock Definitions 145
10.4.4 Skew Definition 146
10.4.5 Defining Input and Output Delay 147
10.4.6 Specifying the Minimum (min) and Maximum (max) Delay 147
10.4.7 Design Synthesis 149
10.4.8 Save the Design 149
10.5 Constraint Validation 149
10.6 Commands for the DRC, Power, and Optimization 150
10.7 Chapter Summary 151

11 Design Synthesis and Optimization Using RTL Tweaks 153
11.1 ASIC Synthesis 153
11.2 Synthesis Guidelines 154
11.3 FSM Design and Synthesis 155
11.4 Strategies for the Complex FSM Controllers 158
11.5 How RTL Tweaks Are Useful During Synthesis? 158
11.6 Synthesis Optimization Techniques Using RTL Tweaks 165
11.6.1 Resource Allocation 165
11.6.2 Dead Zone Elimination 167
11.6.3 Use of Parentheses 171
11.6.4 Grouping the Terms 173

11.7 FPGA Synthesis . . . 175
11.8 Chapter Summary . . . 177

12 Synthesis and Optimization Techniques . . . 179
12.1 Introduction . . . 179
12.2 Synthesis Using Design Compiler . . . 180
12.3 Synthesis and Optimization Flow . . . 182
12.4 Area Optimization Techniques . . . 186
12.4.1 Avoid Use of Combinational Logic as Individual Block . . . 187
12.4.2 Avoid Use of Glue Logic Between Two Modules . . . 187
12.4.3 Use of set_max_area Attribute . . . 188
12.4.4 Area Report . . . 189
12.5 Design Partitioning and Structuring . . . 190
12.6 Compilation Strategy . . . 192
12.6.1 Top-Down Compilation . . . 192
12.6.2 Bottom-Up Compilation . . . 193
12.7 Chapter Summary . . . 193

13 Design Optimization and Scenarios . . . 195
13.1 Design Rule Constraints (DRC) . . . 195
13.1.1 max_fanout . . . 196
13.1.2 max_transition . . . 196
13.1.3 max_capacitance . . . 197
13.2 Clock Definitions and Latency . . . 198
13.2.1 Clock Network Latency . . . 198
13.2.2 Generated Clock . . . 198
13.2.3 Clock Muxing and False Paths . . . 199
13.2.4 Clock Gating . . . 199
13.3 Commands Useful During Design Synthesis and Optimization . . . 200
13.3.1 set_dont_use . . . 201
13.3.2 set_dont_touch . . . 201
13.3.3 set_prefer . . . 202
13.3.4 Command for the Design Flattening . . . 202
13.3.5 Commands Used for Structuring . . . 203
13.3.6 Group and Ungroup Commands . . . 203
13.4 Timing Optimization and Performance Improvement . . . 204
13.4.1 Design Compilation with ‘map_effort high’ . . . 204
13.4.2 Logical Flattening . . . 205
13.4.3 Use of group_path Command . . . 205
13.4.4 Submodule Characterizing . . . 208
13.4.5 Register Balancing . . . 209
13.5 FSM Optimization . . . 209

13.6 Fixing Hold Violations . . . 210
13.7 Report Command . . . 211
13.7.1 report_qor . . . 211
13.7.2 report_constraints . . . 212
13.7.3 report_contraints_all . . . 212
13.8 Multicycle Paths . . . 214
13.9 Chapter Summary . . . 214

14 Design for Testability . . . 217
14.1 What Is Need of DFT? . . . 217
14.2 Testing for Faults in the Design . . . 218
14.3 Testing . . . 218
14.4 Strategies Used During the DFT . . . 219
14.5 Scan Methods . . . 220
14.5.1 Mux-Based Scan . . . 221
14.5.2 Boundary Scan . . . 221
14.5.3 Built-In Self-Test (BIST) . . . 221
14.6 Scan Insertion . . . 223
14.7 Challenges During the DFT . . . 223
14.8 DFT Flow and Test Compiler Commands . . . 224
14.9 The Scan Design Rules to Avoid DRC Violations . . . 224
14.10 Chapter Summary . . . 227

15 Timing Analysis . . . 229
15.1 Introduction . . . 229
15.2 What Are Timing Paths for Design . . . 230
15.2.1 Input to Reg Path . . . 231
15.2.2 Reg to Output Path . . . 231
15.2.3 Reg to Reg Path . . . 231
15.2.4 Input to Output Path . . . 232
15.3 Let Us Specify the Timing Goals . . . 232
15.4 Timing Reports . . . 235
15.5 Strategies to Fix Timing Violations . . . 236
15.5.1 Fixing Setup Violations in the Design . . . 238
15.5.2 Hold Violation Fix . . . 242
15.5.3 Timing Exceptions . . . 242
15.6 Chapter Summary . . . 242

16 Physical Design . . . 245
16.1 Physical Design Flow . . . 245
16.2 Foundation and Important Terms . . . 246
16.3 Floor Planning and Power Planning . . . 248
16.4 Power Planning . . . 249
16.5 Clock Tree Synthesis . . . 251
16.6 Place and Route . . . 252

16.7 Routing . . . 253
16.8 Back Annotation . . . 255
16.9 Signoff STA and Layout . . . 255
16.10 Chapter Summary . . . 257
Reference . . . 258

17 Case Study: Processor ASIC Implementation . . . 259
17.1 Functional Understanding . . . 259
17.2 Strategies During Architecture Design . . . 260
17.3 Micro-architecture Strategies . . . 263
17.4 Strategies During RTL Design and Verification . . . 265
17.5 The Sample Script Used During Synthesis . . . 267
17.6 Synthesis Issues and Fixes . . . 267
17.7 Pre-layout STA Issues . . . 268
17.8 Physical Design Issues . . . 269
17.9 Chapter Summary . . . 270

18 Programmable ASIC . . . 271
18.1 Programmable ASIC . . . 271
18.2 Design Flow . . . 273
18.3 Modern FPGA Fabric and Elements . . . 274
18.4 RTL Design and Verification . . . 279
18.5 FPGA Synthesis . . . 283
18.5.1 Arithmetic Operators and Synthesis . . . 283
18.5.2 Relational Operator and Synthesis . . . 284
18.5.3 Equality Operator Synthesis . . . 287
18.6 Design at Fabric Level . . . 288
18.7 Chapter Summary . . . 290

19 Prototyping Design . . . 293
19.1 FPGAs for Prototyping . . . 293
19.2 Synthesis Strategies During Prototyping . . . 295
19.2.1 Fast Synthesis for Initial Resource Estimation . . . 295
19.2.2 Incremental Synthesis . . . 295
19.3 Constraints During FPGA Synthesis . . . 297
19.4 Important Considerations and Tweaks . . . 299
19.5 IO Pad Synthesis for FPGA . . . 301
19.6 Prototyping Tools . . . 301
19.7 Chapter Summary . . . 301

20 Case Study: IP Design and Development . . . 303
20.1 IP Design and Development . . . 303
20.2 What We Consider During the IP Selection . . . 304
20.3 Strategies Useful During the IP Design . . . 304
20.4 Prototyping Using Multiple FPGA . . . 307

20.5 H.264. Encoder IP Design and Development 309
20.5.1 Features and Micro-architecture Design Strategies . . . 309
20.5.2 Strategies During RTL Design and Verification 310
20.5.3 Strategies During Synthesis and DFT 311
20.5.4 Strategies During Pre-layout STA 311
20.5.5 Strategies During Physical Design 312
20.6 ULSI and ASIC Design 312
20.7 Chapter Summary 313

Appendix A 315

Appendix B 321

Bibliography 323

Index 325

About the Author

Vaibbhav Taraate is an entrepreneur and mentor at "1 Rupee S T". He holds B.E. (Electronics) degree from Shivaji University, Kolhapur (1995) and received a Gold Medal for standing first in all engineering branches. He completed his M.Tech. (Aerospace Control and Guidance) at the Indian Institute of Technology (IIT) Bombay, India, in 1999. He has over 18 years of experience in semi-custom ASIC and FPGA design, primarily using HDL languages such as Verilog, VHDL and SystemVerilog. He has worked with multinational corporations as a consultant, senior design engineer, and technical manager. His areas of expertise include RTL design using VHDL, RTL design using Verilog, complex FPGA-based design, low power design, synthesis and optimization, static timing analysis, system design using microprocessors, high-speed VLSI designs, and architecture design of complex SOCs.

Chapter 1
Introduction

If we recall the beginning of the miniaturization era, then we can imagine the basic bipolar junction transistor invention at Bell Labs (now AT&T) during year 1947. The first bipolar junction transistor was invented by William Shockley, Bardeen, and Brattain at Bell Labs and got the Nobel Prize in physics during the year 1956. The first integrated circuit was introduced by the 26-year-old engineer Jack Kilby at Texas Instruments (TI).

The popularity of CMOS devices increased during the year 1963 due to low power, high package density, and high-speed requirements.

During the year 1965-1975, Gordon Moore stated Moore's law that is 'Number of transistors in dense integrated circuit doubles in approximate 18–24 months.' The observation of Gordon Moore was valid till the year 2015. It may require modification for the technology node below 10 nm. According to my observation to double the transistors, it may require almost 36 months which may be true during next few decades.

To have basic understanding of the ASIC design and flow, the objective of the remaining sections is to get familiarity with the types of ASICs, different abstraction levels, and few examples which can be useful to think about the ASIC design and strategies.

ASIC is an application specific integrated circuit and designed for specific application.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

1.1 ASIC Design

The era of miniaturization from the year 1960 to 2020 has witnessed lot of the evolutions and design changes. What we need to understand is that, what is exactly ASIC design? Now consider the small square of few micrometers or nanometer which is empty box. Now for the specific functionality, design team will fill this empty box with functional blocks. The design team which performs this is front-end (logic) design team.

The backend or physical design team works in the area of floor planning to physical verification at the chip level for the specific technology node.

The manufacturing unit which is foundry performs the manufacturing and packaging of the chip in mass, and initially few sample pieces will be tested by the design houses to understand the intended design outcome.

Now how all above is achieved? All the design-related work is achieved by the intelligent chip designers working in the various areas of the chip design using the Electronic Design and Automation (EDA) tools. The various popular EDA tools are from Synopsys and Cadence and extensively used in the design of chip and to improve or to achieve the desired performance.

With the functionality of the chip, it requires to understand about the constraints such as area, speed, and power and the main goal of the logic design and physical design team is to understand the block-level and top-level constraints and to have the better strategy to achieve the desired performance.

For the basic understanding, consider the pipelined processor which performs the basic arithmetic and logic operations such as addition, subtraction, multiplication, division, XOR, OR, AND, and NOT. What we need to imagine at the higher level is the functional blocks, complexity of the design that is rough estimation of the area, what constraints we should apply, and what exactly we will achieve. At the beginning, we will have just the basic idea of the blocks and as the design evolves we will land up into the phase of the chip architect.

For the idea of the architecture of the above chip, the basic layout is shown in Fig. 1.1. The subsequent chapter discusses the design flow, chip architecture, micro-architecture!

1.2 Types of ASIC

ASIC Design: ASIC stands for the application-specific integrated circuit and is designed to perform the specific application. For example, the processor or controller is used to process the specific information.

The following are different types of ASICs

1. Full-custom ASIC
2. Semi-custom ASIC
3. Gate array-based designs
4. Structured ASICs.

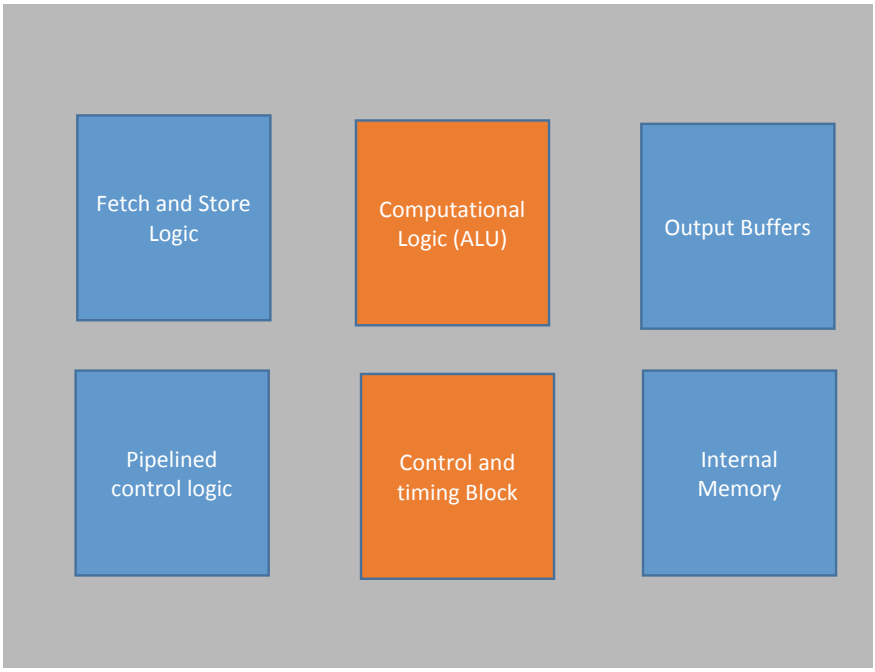


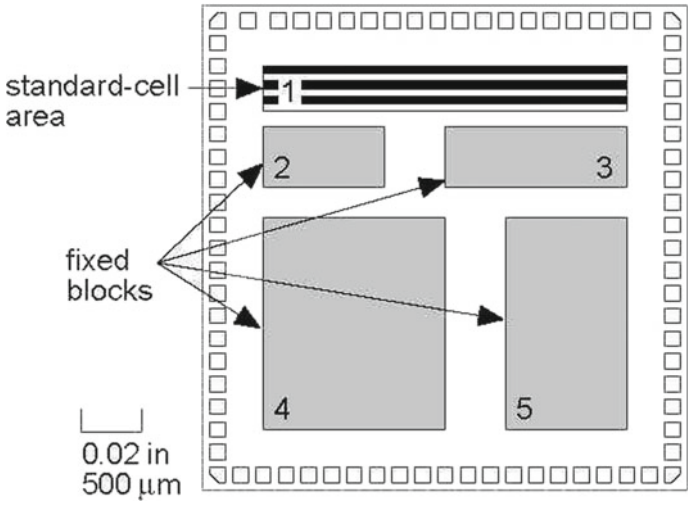
Fig. 1.1 Basic chip layout

Full-Custom ASIC: In this type of ASICs, the design starts from scratch for the specific technology node. Each cell is designed depending on the technology node requirements. This approach is useful for high volume production, and one can imagine the microprocessors and floating-point processors, which are required in the design and can be designed using the full-custom design flow.

The major advantage of the full-custom design is that for the high volume production it gives the lower power, high speed, and the least gate count. Achieving the constraints of the speed, area, and power is time consuming for this flow. But as the cells are designed from the scratch for the desired technology nodes, the desired constraints can be achieved.

The major disadvantage of this flow is the high non-recurring expenditure and the long design cycle time.

Standard Cell-Based ASICs



In this type of design flow, the standard library cells such as NAND, NOR, XOR, and flip-flops are used during the design. The beauty of this flow is that it uses the pre-defined and prefabricated cells, for example, RAM hard macro-cores, etc. The transistors and interconnects are customized; that is, all the mask layers are customized.

The advantage of this flow is that, as compared to full-custom ASIC the design cycle time is shorter and for the specific technology node the pre-validated standard cells like microprocessors and macros are available during the design.

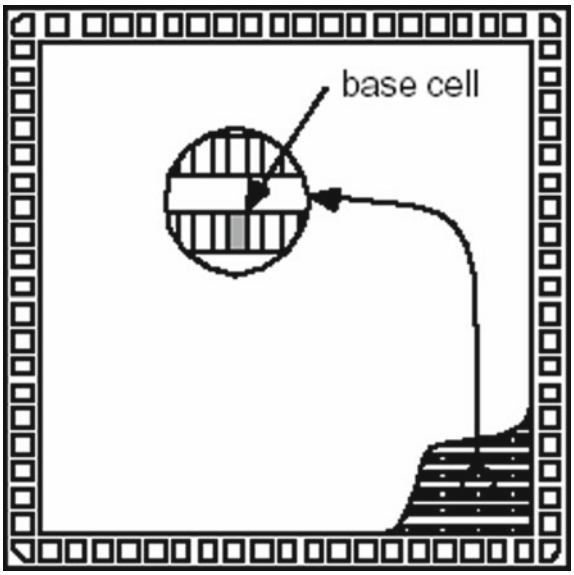
The disadvantage is that as compare to the gate array based ASICs, the design has the high NRE and it needs separate fabrication mask for each design.

Gate Array-Based ASIC

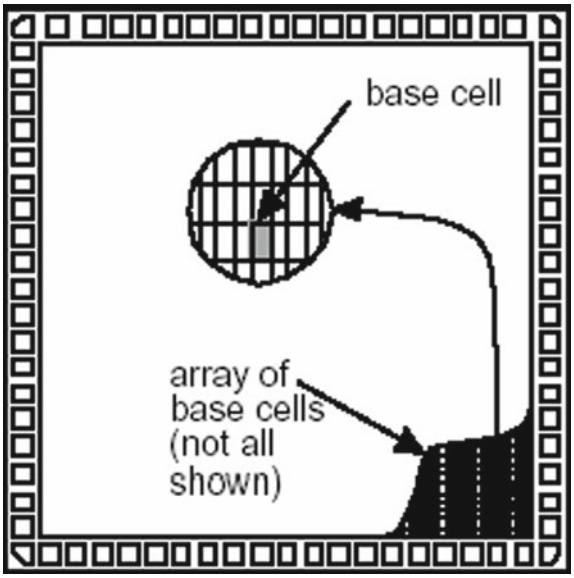
In this type of ASICs, the wafers are prefabricated with unconnected gate array. That is, wafers are common for all the designs. The types of gate array-based ASICs are mainly of following two types.

1. Channeled gate array
2. Channel-less gate array.

Channeled Gate Array: In this type of ASIC, the interconnects use the pre-defined spaces between the rows of base cells.



Channel-Less Gate Array: In this type of ASICs, the few top mask layers are customized.



The major advantage of the gate array-based ASIC is that, lower NRE cost as the same wafer is fabricated for the multiple designs. Another main advantage is the low turnaround time.

The main disadvantages are the low density, lower volume, and the less optimized design.

Structured ASICs

A structured ASIC falls between and gate array and a standard cell-based ASIC.

The main design task involves mapping the design into a library of building block cells and interconnecting them as necessary. The main important points regarding the structured ASICs are components are 'almost' connected in a variety of pre-defined configurations and only a few metal layers are needed for fabrication which in turn drastically reduces turnaround time.

The advantages of the structured ASIC are low NRE cost, less complexity, low power consumption, high performance, and the smaller marketing time.

The main disadvantage is that the team needs to have better understanding of the design constraints due to the use of prefabricated design cells.

1.3 Abstraction Levels

The design can go through the different abstraction levels such as functional design, logic design, gate-level design, and the switch-level design. This section discusses these abstraction levels in more detail.

1. **Functional Design**: Now imagine a scenario to design a product or chip, so the first thought is the product idea or/and depending on the idea the chip functionality can be extracted. The functional design is basically the outcome of the functional specification, and the group of team members can create the high-level and low-level design document and can code the functionality using the higher-level language such as C or C++. For example, consider the H.264 encoder design, and the functional design team can create the golden reference model using the high-level language by using the following

   (a) The types of frames which need to be processed
   (b) The frame support
   (c) The prediction blocks and functionality
   (d) The quantization and transform algorithms required
   (e) The entropy coding methods

   If the desired functionality is validated, then the design can be considered as golden reference model which can be used throughout the design.
2. **Logic Design**: The logic design team understands the architecture of the chip and the partitioning mechanisms to complete the RTL design, where RTL stands for the register transfer level. The team of professionals uses the HDL such as VHDL, Verilog, and SystemVerilog to have the RTL design and verification at the block and top level. The main advantages of the HDL used during the RTL design are as follows:

(a) The HDL supports the concurrent and sequential constructs.
(b) HDL supports the notion of time.
(c) HDL supports describing the interfaces and ports as input, output, bidirectional.
(d) HDL supports the edge- and level-sensitive design constructs.

For more details about the RTL design and verification, refer Chaps. 3 and 4. The logic design flow is discussed in Chap. 2.

The RTL design example using the Verilog to infer the 2-bit shift register using the non-blocking assignment is described in Example 1.

Example 1 The RTL description using Verilog

```
////////////////////////////////////////////////////////////////////////////////
module  non_blocking_assignments
 ( input data_in, clk, reset_n,
   output reg data_out);

reg  tmp;

always @ ( posedge clk or negedge reset_n)
begin

        if ( ~ reset_n)
        begin
                { data_out, tmp } <= 2'b00;

        end

        else
   begin

              data_out <= tmp;

                        tmp <= data_in;
                end
end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

3. **Gate-Level Design**: The RTL is given as one of the inputs to synthesis tool to get the gate-level netlist. The synthesis is process of getting the lower level of abstraction from the higher-level design (Fig. 1.2).

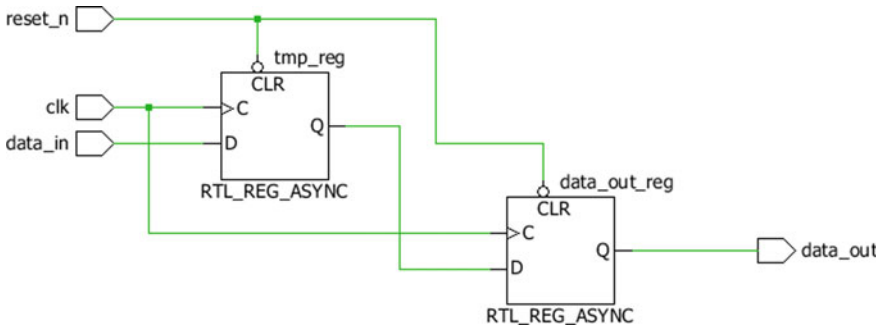
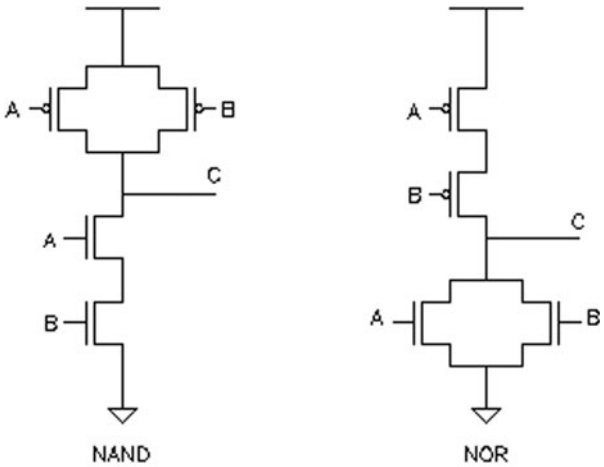


Fig. 1.2 RTL schematic of Example 1

4. **Switch-Level Design**: The design using the CMOS standard cells and switches is called as switch-level design. In the simple term, the physical design or backend design is like playing with the switches and standard cells, macros for the specific technology node. The backend/physical design flow is discussed in Chap. 2.



1.4 Design Examples

Now consider the ASIC design of H.264 encoder and decoder, and what we should do?

1. Market survey to understand the availability of various products in the market
2. The functional specification of the H.264 encoder and decoder
3. The functional design documentation such as high-level design (HLD) and low-level design (LLD) and design planning
4. Logic Design: Plan the design
   (a) Specification understanding and the architecture design
   (b) RTL design and verification
   (c) Synthesis/DFT and timing verification
5. Physical Design: Design from floor planning to physical verification
   (a) Planning of the design (floor planning and power planning)
   (b) CTS
   (c) Place and route
   (d) Physical and timing verification
   (e) GDSII
6. Manufacturing and Test: The design manufacturing and test phases
   (a) Fabrication
   (b) Packaging
   (c) Test.

Consider the initial floor plan of the H.264 encoder (Fig. 1.3).

1.5 What We Should Know?

During the ASIC design cycle with functional design and validation, we should focus on the area, speed, and power constraints.

1. **Area**: The chip area and the logic area which defines the overall density of the design in the few micrometer square. Meeting the area constraints is one of the important tasks during the logic and physical synthesis. The area optimization can be achieved at various levels such as
   (a) Architecture tweaks
   (b) RTL tweaks
   (c) Using synthesis commands
   (d) At the physical design using dedicated cells

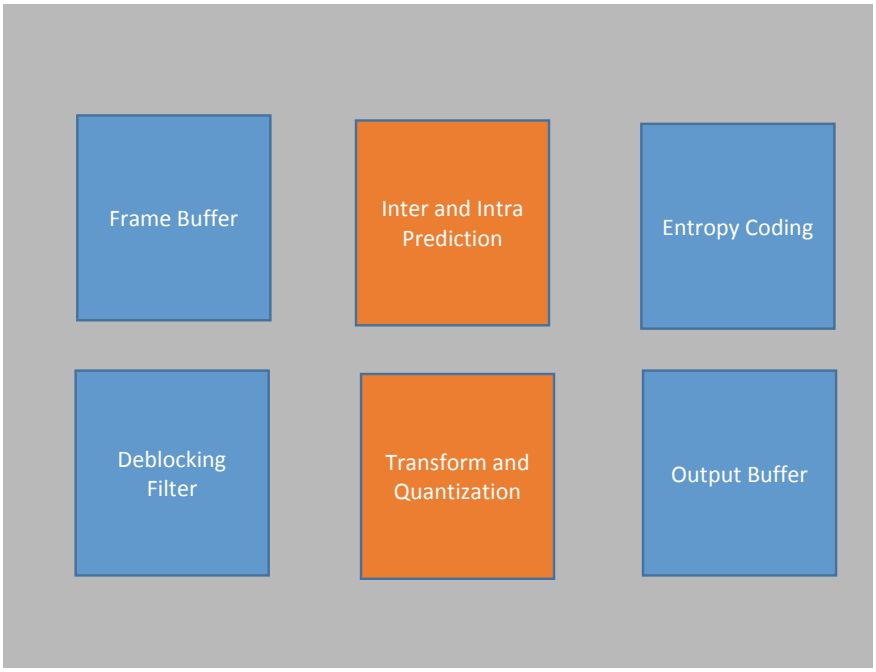


Fig. 1.3 Initial floor plan of H.264 encoder

2. **Speed**: The speed is another important constraint and can be achieved at the block and top level using the

   (a) Synopsys PT commands
   (b) RTL tweaks
   (c) Architecture tweaks
   (d) During physical design
   (e) Using the dedicated IPs

3. **Power**: The power, static and dynamic are another important constraints for ASIC and can be achieved using the following

   (a) Use of low-power architecture
   (b) Low power cells
   (c) RTL tweaks to reduce the dynamic power
   (d) Low power format

4. **Clock Skew**: The skew is the difference between the clock arrivals at two different coordinates.

   (a) **Positive Clock Skew**: The launch flip-flop is triggered first and then capture flip-flop.

   (b) **Negative Clock Skew**: The launch flip-flop is triggered last, and capture flip-flop is triggered first.

5. **Slack**: The slack is the difference between the two different time instances.
   (a) **Setup Slack**: The setup slack is the difference between the data required time and data arrival time.
   (b) **Hold Slack**: The hold slack is the difference between the data arrival time and data required time.
6. **Clock Gating**: The use of the clock gating cells to minimize the dynamic power.
7. **Synchronous Design**: All the flip-flops in the design are triggered by using the common clock source.
8. **Asynchronous Design**: The flip-flops in design are triggered by the different clock sources.

1.6 Important Terms Used Throughout Design Cycle

The following are few important terms which we should know during the ASIC design cycle.

1. **Architecture**: Block-level representation of design
2. **Micro-architecture**: Sub-block-level representation of design
3. **RTL**: Register transfer level
4. **RTL design**: Design using the HDL synthesizable constructs
5. **RTL verification**: The testbench and automation using the non-synthesizable constructs
6. **Synthesis**: The process of getting the gate-level netlist from the RTL. Or it is the process of getting the lower level of abstraction from the higher-level design
7. **DFT**: Design for test to find the manufacturing defects
8. **STA**: Static timing analysis at the pre-layout or post-layout
9. **Floor planning**: The chip floor plan
10. **Power planning**: The power mesh and ring planning for the chip
11. **CTS**: Clock tree synthesis, the clock trees for uniform distribution of skew, and the strategy
12. **P and R**: Placement and routing that is the placement of standard cells and macros, IPs, and to route them
13. **Physical verification**: The verification that is LVS and DRC
14. **LVS**: Layout versus schematic check
15. **DRC**: Design rule check
16. **Back annotation**: The RC extraction
17. **GDSII**: GDSII stream format, common acronym GDSII, is a database file format which is the de facto industry standard for data exchange of integrated circuit or IC layout artwork.

1.7 Chapter Summary

The following are few important points to conclude the chapter.

1. 'Number of transistors in dense integrated circuit doubles in approximate 18–24 months' is called as Moore's law.
2. ASIC stands for the application-specific integrated circuit.
3. FPGA stands for the field programmable gate array.
4. The major advantage of the full-custom design is that for the high volume production it gives the lower power, high speed, and the least gate count.
5. The main advantage of the semi-custom ASIC is, as compared to full-custom ASIC the design cycle time is shorter and for the specific technology node the pre-validated standard cells like microprocessors and macros are available during the design.
6. The major advantage of the gate array-based ASIC is that lower NRE cost as the same wafer is fabricated for the multiple designs.
7. The main design constraints are area, speed, and power.

Chapter 2
ASIC Design Flow

As discussed in the previous chapter, the ASICs can be of type full-custom, semi-custom, gate array-based ASICs. The major objective of the following few sections is to have the detailed discussion about the semi-custom ASIC design flow and the programmable ASIC design flow. The important design examples are also discussed in the next few sections and useful during the ASIC and FPGA design.

2.1 ASIC Design Flow

The semi-custom ASIC design in which the standard cells and macros which are pre-validated is used. As discussed in Chap. 1, we can have different types of ASICs such as full-custom, semi-custom, gate array-based and depending on the design requirements we can choose one of the flows. Figure 2.1 describes few of the important design phases during the ASIC design cycle.

1. **Market Survey and Specification Extraction**: It is one of the important phases during the design cycle. Before the logic design, the team performs the market survey to understand what are the different products of similar type available in the market. Origin of any design idea or product can be realized in the quick time, and product should be excellent in all the aspects that are the major objective of any organization. The excellence in the design and product innovation is objective of many research and development organizations. For example, consider Intel as processor design organization, what they work on the processing capability, low power architecture design, high-speed designs, signal integrity, and more reliability to their chipsets.

Understanding of the ASIC design flow plays an important role during design cycle.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

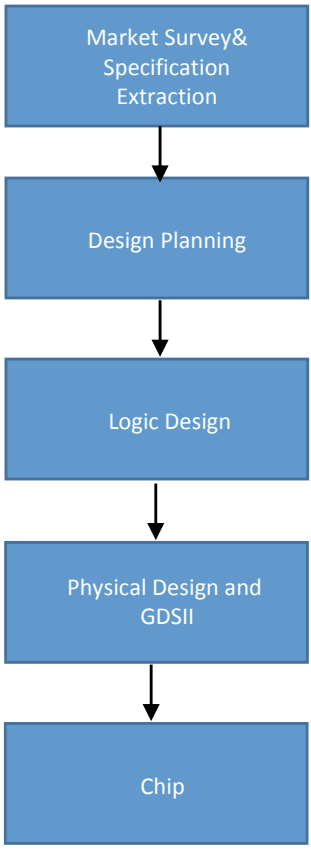


Fig. 2.1 Semi-custom ASIC design flow

For new idea, finalization of the specification and the architecture of the chip is the primary task and for that the market survey plays an important role. Following the team does during the market survey.

(a) The detailed understanding of the available products
  (i) Understanding of functionality, speed, power, and area
  (ii) Understanding about the electrical characteristics
  (iii) Understanding about the mechanical assembly and packaging
  (iv) Understanding about the user interfaces
(b) The volume and cost of product
(c) End customer base
(d) How the new idea can be better as compared to existing products?

The main outcome of all above is to extract the specifications of the product or chip at various levels. Our goal is to work on the functional design of the chip so we will consider the functional specifications.

Let us consider the 32-bit processor what we need is the following!

(a) The operations performed by the processor such as arithmetic, logical, data transfer, branching, and floating point
(b) The complexity of bus interfaces such as address bus and data bus
(c) The performance improvement mechanism such as pipelining and the configuration support
(d) The electrical characteristics of interfaces such as slew, voltage levels, and power
(e) The external interface information and compatibility
(f) The internal storage information and the data computation schemes
(g) The IP availability and their specifications
(h) The technology node for ASIC and the performance
(i) What are the constraints achieved such as area, speed, and power?

By using all above, the specifications of the product can be documented and the team understands the feasibility of the product using few risk and dependability matrix parameters. Consider the parameter as a speed, existing chipset is working on the operating frequency of 400 MHz and technology node is 10 nm then can the desired product can operate at 450 MHz or not?
If answer is no due to the technology library cell characteristics, then the choice can be work on the 400 MHz but with more parallel computational elements or can use the lower technology node such as 7 nm to achieve the 450 MHz operating frequency.

2. **Design Planning**: The design planning in technical terminology is the architecture and micro-architecture design, but practically with this we need to work on the project planning and chip delivery plan to the market. So broadly concurrent teams of technical, man management, and the delivery will work to accomplish the design task. The project planning is ruled out as per as discussion in this book is concerned. Our objective is to work on the understanding of the specification to have the top-level architecture in place so that we can plan for the logical, physical design and chip manufacturing and test phases.
The specification extraction document is used as input during this phase to get

(a) Architecture design and micro-architecture design
(b) Architecture tweaks to estimate the rough area and the possibility of achieving constraints
(c) Having information about the top-level interfaces and the timing
(d) Can be used to understand about the storage and memory requirements
(e) Useful for the project planning and planning of milestone delivery

We will consider the outcome of the design planning stage as the architecture and micro-architecture evolution to have the better architecture design.

3. **Logic Design**: The logic design phase of the ASIC is very important as the quality of the RTL design and verification decides about the quality of chip. During the logic design phase, broadly we need to perform the following and discussed in Sect. 2.1.1.

(a) RTL design
(b) RTL verification
(c) Synthesis
(d) DFT and scan insertion
(e) Equivalence checking
(f) Pre-layout STA

4. **Physical Design and GDSII**: The physical design phase of the ASIC is also called as backend design, and the physical design team uses the gate-level netlist as one of the inputs with the technology libraries to get the GDSII. During the physical design phase, broadly we need to perform the following and discussed in Sect. 2.1.1.

(a) Floor planning
(b) Power planning
(c) CTS
(d) Place and route
(e) LVS
(f) DRC
(g) Signoff STA
(h) GDSII

5. **Chip**: To get the chip from foundry, there are several manufacturing and packaging processes. The sample chips will be issued to test houses to perform the testing.

2.1.1 Logic Design

The logic design flow uses the functional design specifications and the target technology library for the ASIC to get the gate-level netlist. The logic design flow is shown in Fig. 2.2 and consists of the important steps as follows

1. Design partitioning at the architecture level
2. RTL design and verification
3. Synthesis and DFT
4. Equivalence checking
5. Pre-layout STA.

This section discusses these important milestones.

1. **Design Partitioning**: As the ASIC design architecture is complex in nature and consists of million or billion gates, the better idea is to have the partitioning of the design at the architecture level. The architecture and micro-architecture document is evolved from the functional specifications and is used as reference for the design.

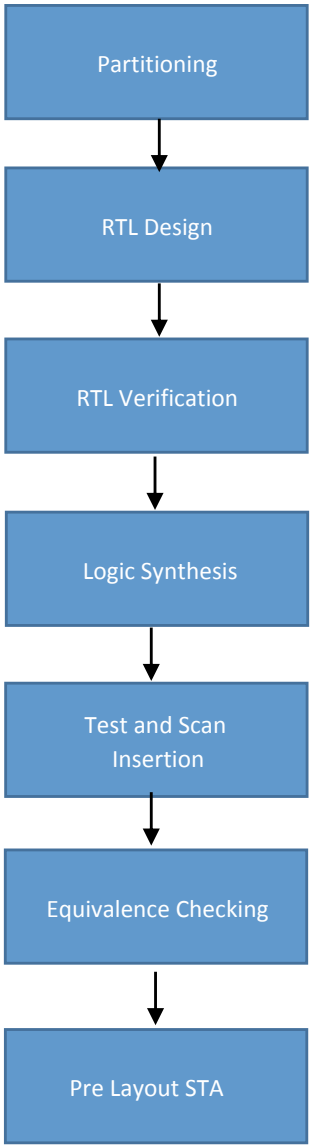


Fig. 2.2 ASIC logic design flow

The design partitioning goal is to have the modular design approach to get the better quality of the RTL. The design partitioning team considers the following points while partitioning of the design.

(a) The complexity of the blocks
(b) IPs used
(c) Single clock verses multiple clock
(d) Low-power design strategies
(e) Software and hardware partitioning
(f) Constraints for the functional blocks

By considering all above, the design is partitioned into multiple blocks and used as the basic foundation for the RTL design and verification. Better design partitioning can yield into the efficient RTL design and in turn the chip development. The design partitioning can be achieved at the different levels such as the architecture, RTL, and netlist level.

2. **RTL Design**: As discussed in Chap. 1, the RTL is register transfer level and it is the representation of the design functionality using the hardware description language. Throughout this book, our focus is to have discussion on the ASIC design using Verilog as hardware description language. During the RTL design phase, the following is accomplished by the design team.

(a) Understanding of the functionality of the design and the design partitioning
(b) The block-level RTL design. If the block functionality is complex, then the micro-architecture of the design is to implement the sub-block RTL
(c) Use of the registered inputs and outputs for the better and clean timing
(d) Use of the area and speed improvement features at the RTL level. The features such as resource sharing and pipelining
(e) The RTL design using the modular design approach
(f) Use of the synchronizers for passing of the data between the different clock domains
(g) Use of the required low power cells and the power formats during the RTL design stage
(h) Understanding of the use of required IPs and the integration to get the desired functional output
(i) Top-level integration of the functional block to yield into the desired intended functionality and the clean timing
(j) Sanity-level verification to confirm the design functionality and functional correctness of the design

3. **RTL Verification**: For any kind of chip, the functional correctness at the block, top, and chip level should be achieved and that is the primary objective of the verification team. The following are few of the important tasks which verification team should complete during the verification cycle.

(a) Verification planning for the chip
(b) Verification architecture
(c) Creating the test cases and test vectors which can be used during the verification of the block and top functionality
(d) Assertion-based verification mechanism and automation in the testbenches to achieve the desired coverage goals

(e) Efficient reporting mechanism and better communication strategies with the RTL design team
So in the simple way what the verification team does is that better verification plans and strategy to check for the functional correctness of the block and tip-level design using the robust and automated testbenches!

4. **Logic Synthesis**: The synthesis is the processes of getting the lower level of abstraction from the higher level. During the logic design, the objective is to get the gate-level netlist from the Verilog RTL. During this book, our goal was to have the focus on the FPGA synthesis using the Xilinx EDA tool Vivado, and ASIC synthesis using the Synopsys Design Compiler which is popular EDA tool and mostly referred as Synopsys DC.
The ASIC synthesis tool uses the following inputs to get the gate-level netlist.

   (a) RTL design (.v files)
   (b) Optimization constraints such as area, speed, and power
   (c) Technology library

   The synthesis tool should be efficient enough to achieve the target constraints such as area, speed, and power. If constraints are not achieved, then it is recommended to have the architecture tweaks and RTL tweaks.
   Few of the architecture tweaks can be

   (a) Improve the partitioning strategies
   (b) Use the pipelined design approach
   (c) Partition design at the sequential boundaries
   (d) Include the parallelism by replicating design into multiple blocks

   Few of the RTL design tweaks can be

   (a) Use resource sharing
   (b) Use dead zone elimination and constant folding
   (c) Use the pipelining for the clean timing
   (d) Use the sequential boundaries and modular approach during the block-level design

   The netlist can be saved in the file format and can be .v file or the Synopsys database.
5. **Test and Scan Insertions**: The design for testability is used to get the manufacturing defects from the design. The DFT is mainly to find the stuck at faults from the design may be single, double, or triple stuck at faults. The DFT is mainly to check for the controllability and observability of the different nodes in the design. The DFT is an important milestone to get the information about the defects. The DFT schemes can be of various types, and few of them are

   (a) Ad hoc DFT
   (b) Structured DFT

(i) Scan-based DFT
   1. Partial scan
   2. Full scan

(ii) Built in Self-test
   1. LBIST
   2. MBIST

(iii) JTAG

The DFT schemes and the case study are discussed in the subsequent chapters, and the major goal is to understand about the DFT schemes with the objective and the role of EDA tools.

6. **Equivalence Checking**: To preserve the design intent, the equivalence checking needs to be performed to check the logic equivalence. The equivalence checking uses the formal verification techniques.
   The objective of the equivalence checking is to verify the RTL design functionality.
7. **Pre-layout STA:** The STA stands for the static timing analysis and is one of the important milestones during the logic design. The major goal is to find out the timing violations from the design, and these violations are mainly the setup and hold time. During the pre-layout STA, the setup time violations are fixed as the design doesn't have the routing information. STA popular EDA tool is Synopsys PT (PT shell) and used to report the timing information by analyzing all timing paths from the design. The STA tool uses the input as the gate-level netlist, timing library, and technology library with the timing constraints to analyze the timing for the design.

More about the timing is discussed in the subsequent chapters.

2.1.2 Physical Design

The physical design flow with the Synopsys IC Compiler is discussed in Chap. 16. This section discusses what exactly the design team works during the physical design phase.

As the gate-level netlist is available from the logic design flow, the netlist with chip constraints and required libraries is used as input during the physical design flow.

The physical design starts with the floor planning that is planning of the design mapping, and the goal is that there should not be congestion while routing of the design and the logic blocks or functional blocks should meet the aspect ratio. The better floor planning is required to have the better area, speed, and power and will be useful to avoid the routing congestion. The power planning stage is used to plan for the power rings (VDD and VSS) and power straps depending on the power requirements (Fig. 2.3).

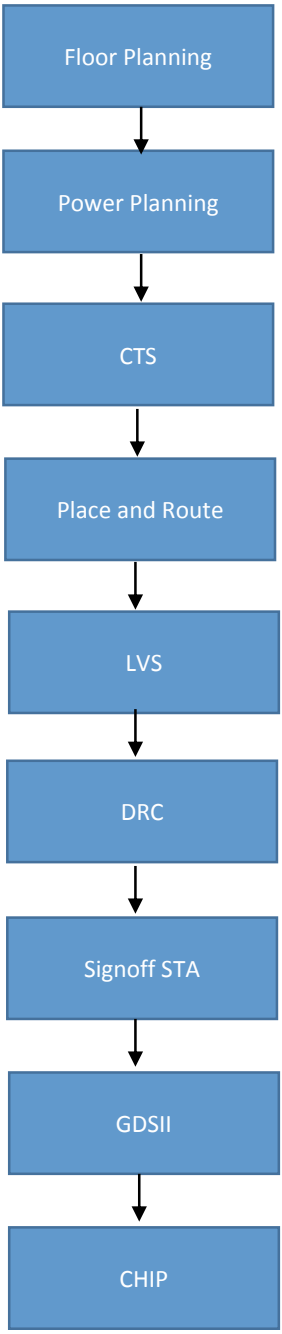


Fig. 2.3 ASIC physical design flow

After the power planning is done, the clock tree synthesis needs to be performed to balance the clock skew and to distribute the clock to the various functional blocks of the design. The clock tree can be H tree, X tree, balanced tree and discussed in Chap. 16.

The placement and routing are done to have the layout of the chip. The layout will have the routing delays, and many times the STA needs to be performed to find and fix the timing violations.

The layout of chip needs to be checked to verify the

(a) Foundry rules, that is, DRC
(b) LVS that is checking of the layout versus the schematic and the intent is to verify the layout with the gate level netlist.

If all the design rules are met and there are no any issues in the LVS, the team needs to perform the signoff STA. The reason being after the layout, most of the time the design will not meet the required timing and frequency and may requires the modification or tweaking at the various stages. The flow is iterative, and objective is to achieve the chip-level constraints.

After the signoff STA the GDSII is generated. The GDSII stands for the Generic or Geometric Data Structure Information Interchange and describes the layout of the design with the connectivity.

The foundry uses the GDSII to manufacture the chip. It is also treated as tapeout delivered to foundry!

2.2 FPGA Design Flow

FPGA design flow can be also treated as programmable ASIC flow and described in Fig. 2.4.

The important steps are

1. Design planning
2. RTL design and verification
3. Synthesis
4. Design implementation: It consists of the following steps
   (a) Logic functionality mapping
   (b) Place and route
   (c) SDF-based verification
   (d) Signoff STA
5. Device programming.

Few of the important FPGA blocks are shown in Fig. 2.5. The modern FPGA architecture is complex and consists of few of the important blocks:

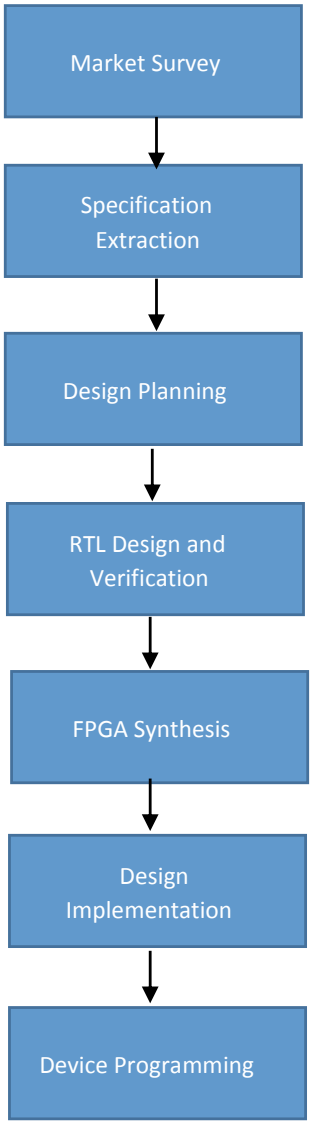


Fig. 2.4 FPGA design flow

1. Configurable logic block (CLB)
2. IO blocks
3. Switch boxes
4. DSP blocks
5. Multipliers
6. Processor block.

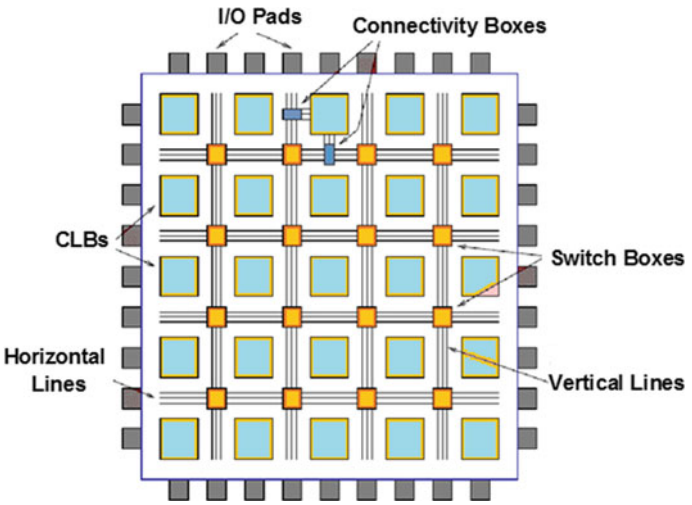


Fig. 2.5 FPGA architecture

Table 2.1 Comparison between ASIC and FPGA

Important parameters	ASIC	FPGA
Cost per piece	Low	High
Time to market	Slow	Fast
NRE cost	High	Low
Size	Small	Medium
Design complexity	Very complex	Moderately complex
Power consumption	Low	High
Performance	High	Moderate

The comparison between the ASIC and FPGA is shown in Table 2.1 by considering few important parameters.

2.3 Examples and Thought Process

Before the ASIC or FPGA design what we need to work on or what we need to revise as foundation is discussed in this section. Consider the complex ASIC what we should understand.

1. The role of ASIC and purpose of design
2. What is technology node
3. Functional and timing details

4. Important design elements such as standard cells, micros, IPs for the ASIC-based designs. Functional blocks and architecture for FPGA-based designs
5. What are the constraints ?
6. For semi-custom ASIC, the reusable elements and components
7. We should think about the
   (a) Area, speed, power requirements
   (b) Data bandwidth
   (c) Latency and clock requirements
   (d) Multiple clocks in design and synchronizer requirements
   (e) Low-power architecture and sequencing
8. Complexity of RTL design, verification, and available IPs
9. EDA tools and optimization efforts required
10. Testing requirements and test planning.

2.4 Design Challenges

If we consider the complex ASIC with million gate count, then important challenges are

1. Design partitioning
2. Longer time to market
3. Complexity in the design and verification
4. Large number of resources
5. Availability of the functional and timing proven IPs
6. Meeting the optimization and design constraints
7. Interconnect delays and requirement of high-speed IOs
8. Testing time and the requirements
9. Effect of noise and the field testing
10. OCV analysis and chip tests.

The subsequent chapters discuss the practical issues in the ASIC design and how to fix them at various stages of the ASIC design cycle.

2.5 Chapter Summary

The following are few of the important points to conclude the chapter.

1. Important logic design flow steps are: RTL design, RTL verification, synthesis, DFT and scan insertion, equivalence checking, and pre-layout STA.
2. Important physical design flow steps are: floor planning, power planning, CTS, place and route, DRC, LVS, signoff STA, GDSII.

3. During the logic design phase, the information about the clock network is not available.
4. The layout of ASIC is floor plan, placement, and routing of the design.
5. During the pre-layout STA, the objective is to fix the setup violations.
6. After the P and R, the overall timing and frequency for the design will never meet and hence post-layout STA is required.
7. Layout of the ASIC chip is the deliverable as the timing is clean and desired constraints are met!.
8. The DRC is performed to confirm whether all foundry rules are met or not!
9. GDSII is Generic Data Structure Information Interchange and delivered to foundry to manufacture ASIC.
10. The RTL to GDSII flow is basically ASIC design flow which consists of the logic design and physical design.

Chapter 3
Let Us Build Design Foundation

During the RTL design phase, we always use the synthesizable constructs, and during the synthesis phase, we will try with objective to achieve the required constraints. Synthesis is a process to get the lower level of abstraction from the higher level of design. During the ASIC design, we will get the various design abstraction levels such as

1. Functional design
2. Architecture and logic design
3. Gate-level design
4. Switch-level design
5. Physical design.

To understand these various abstraction level, the chapter discusses about the important design elements used frequently during the logic design.

3.1 Combinational Design Elements

In the combinational design, an output is function of the present input only. That is if input changes, then an output will change. In the practical scenario, the output will not change instant immediately, but it will change after some delay which we can treat as the gate delay or the propagation delay.

Understanding of the basic design elements plays important role during the logic design and synthesis.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

Table 3.1 Combinational design elements

Combinational logic elements	Description
Universal logic gates	NAND, NOR universal logic gates to implement the combinational design or Boolean function
XOR, XNOR	Used to perform the complement of input and used in parity detectors
Arithmetic resources	The adders, subtractor, multipliers used as the arithmetic resources
Multiplexers	Multiplexers have many inputs and single output and used as data selector or as universal logic to implement the Boolean function. Useful during the pin multiplexing and clock multiplexing
Demultiplexers	Demultiplexers have single input and many outputs and used as de-multiplexing logic. Useful during the pin de-multiplexing and clock de-multiplexing
Decoders	The decoders are extensively used to decode the data and to enable one of the functional block at a time
Encoders	Encoder is used to encode the binary data and performs the reverse operation as compare to decoder
Priority detectors	Priority detectors or priority encoders we can use to assign the output depending on the priority of the level sensitive inputs

The propagation delay of the logic gate is the amount of time required for the logic gate to generate the valid output when input changes either from 1 to 0 or vice versa. The propagation delay is defined for the standard cell or logic gate in the timing library.

The important elements are designed using the logic gates, and we mainly consider the NAND and NOR logic gates as universal gates. The important combinational elements are multiplexers, demultiplexers, decoders, encoders, code converters, and the arithmetic logic elements adders, subtractors, and multipliers.

Table 3.1 discusses about the important combinational elements and their role during the design phase and useful to design the glue logic, functionality for the specific requirement. For few of these elements, the design strategy is explained in this chapter and useful during the RTL design phase.

The following section discusses about these logic elements in the context of the ASIC and FPGA design.

3.2 Logic Understanding and Use of Construct

Understanding the functionality for the individual functional block is especially important during the ASIC design cycle. Whether it is moderate gate count design or the complex design, the understanding will play important role to choose the

suitable construct during the RTL design phase. For combinational modeling, we will choose the 'assign' construct. The 'assign' is continuous assignment, and it is neither blocking nor non-blocking. The multiple assign constructs will infer the combinational logic, and the multiple continuous assignments will execute concurrently.

The Syntax is

```
assign expression_1;
assign expression_2;
```

where ***assign*** is keyword. The keywords used in the Verilog (.v) file are marked using bold characters.

For example, consider the following continuous assignments

```
assign sum = a_in ∧ b_in;
assign carry = a_in & b_in;
```

Both above continuous assignments will execute and update in the active event queue, and they will execute concurrently to infer combinational logic as XOR and AND gate.

3.3 Arithmetic Resources and Area

Let us try to discuss the arithmetic resource which is the adder and subtractor. To optimize the area, the subtraction operation is implemented using the adders that is by using the 2's complement addition. For example, if we have 8-bit binary input A then A − 1, we can consider as A + 11111110 + 1=A + 1111_1111.

Now, let us discuss the reason for performing the subtraction using the 2's complement addition! Consider a practical scenario where we need to perform the 4-bit addition and subtraction where A is 4-bit binary input, B is 4-bit binary input and Control_Input controls the operation. Table 3.2 indicates these two operations.

Table 3.2 Description of the logic

Control_Input	Operation	Description
0	Addition (A, B)	A + B
1	Subtraction (A, B)	A − B

Area Requirement: If we don't use the 2's complement for the subtraction, then we need to use the resource as adder and subtractor and some data selection logic to generate the output. So more the resources, you can imagine the logic inferred as shown below. As shown, the design needs two 2:1 MUX, 4-bit adder and 4-bit subtractor.

Issues in the Design: The main issue in the design strategy is poor data path management and more resources in the data path. Both the adder and subtractor are performing the operation at a time, and depending on the Control_Input the output Result, carry/borrow is updated. So, the poor data path optimization due to lack of sharing the common resources. Another issue is the use of multiplexing logic at the outputs (Fig. 3.1).

Let us optimize the above logic using the concept of sharing of common resources. Now, to eliminate the data selection logic that is 2:1, multiplexer and subtractor let us use the full adder as arithmetic resource. The subtraction can be performed using the 2's complement method, and to perform A − B, use the A + (~B) + 1 that is perform 1's complement of B using the XOR logic. If one of the inputs of XOR gate is logic '1', then output of XOR is complement of the present input (Table 3.3).

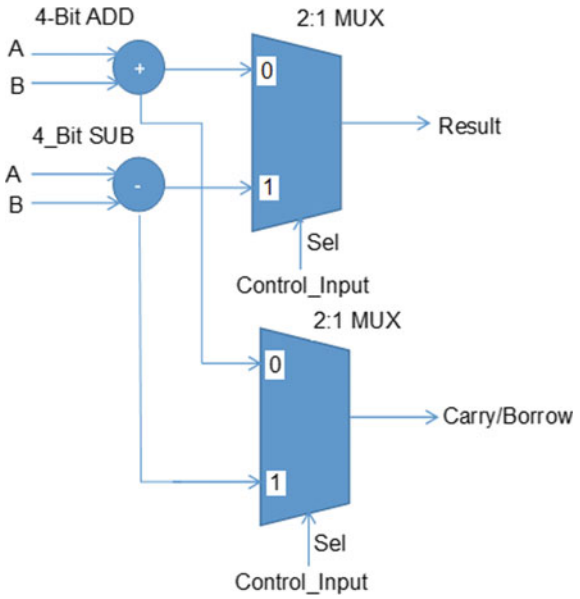


Fig. 3.1 Logic without resource sharing

Table 3.3 Description with goal of resource sharing

Control_Input	Operation	Description
0	Addition (A, B)	A + B + Control_input = A + B + 0
1	Subtraction (A, B)	A − B = A + B + Control_Input = A + B + 1

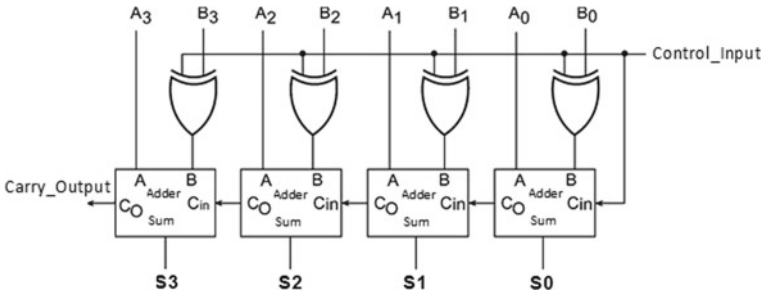


Fig. 3.2 Logic with resource sharing

Logic with Optimized Area: As the full adder is used to perform the addition and subtraction, it eliminates the need of multiplexers and subtractor. The Figure shown is the optimized logic for the 4-bit addition and subtraction. The main advantage of the logic is better data path optimization, and it performs the desired operation depending on the status of the Control_Input (Fig. 3.2).

3.4 Code Converter

Most of the time during the design we may need to use the code converters to convert the data in the suitable format. If we consider the data transfer or control signal transfer in the multiple clock domain designs, then we need to have the gray codes. The section discusses about the binary to gray code converter and gray to binary code converters.

In the two successive binary numbers, one or more than one bit changes, but in the two successive gray codes, only one bit changes. Gray codes are popular and used as pointers in the FIFO design or to pass the control signals in the multiple clock domain designs.

3.4.1 Binary to Gray Code Converter

The 4-bit binary to gray code converter inputs and outputs are listed in Table 3.4.

If we use the input as binary code, then to get the gray code, we need to have the logic as

$$G3 = B3$$
$$G2 = B3 \wedge B2$$
$$G1 = B2 \wedge B1$$

Table 3.4 Binary to gray code

Binary code (B3 B2 B1 B0)	Gray code (G3 G2 G1 G0)
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101
1010	1111
1011	1110
1100	1010
1101	1011
1110	1001
1111	1000

$$G0 = B1 \wedge B0$$

where the character $\wedge$ indicates the XOR operator in the Verilog. The logic diagram is shown below, and the RTL design using Verilog code is described in the example.

Example 1 RTL description of Binary to Gray code converter

```
////////////////////////////////////////////////////////////////////////////////
module code_converter

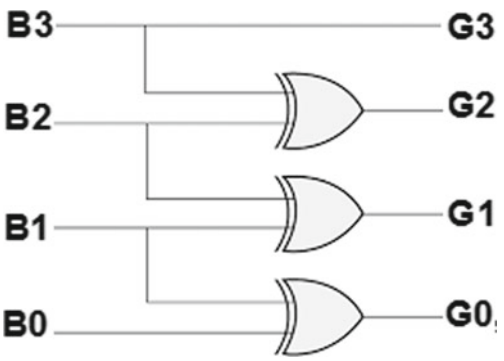
            ( input B3, B2, B1, B0,

                output G3, G2, G1,G0

              );

assign G3=B3;
assign G2 = B3 ^ B2;
assign G1 = B2 ^ B1;
assign G0 = B1 ^ B0;

endmodule
////////////////////////////////////////////////////////////////////////////////
```



3.4.2 Gray to Binary Code Converter

The 4-bit gray to binary code converter inputs and outputs are listed in Table 3.5.

If we use the input as gray code, then to get the binary code, we need to have the logic as

$$B3 = G3$$
$$B2 = G3 \wedge G2$$

Table 3.5 Gray to binary code

Gray code (G3 G2 G1 G0)	Binary code (B3 B2 B1 B0)
0000	0000
0001	0001
0011	0010
0010	0011
0110	0100
0111	0101
0101	0110
0100	0111
1100	1000
1101	1001
1111	1010
1110	1011
1010	1100
1011	1101
1001	1110
1000	1111

$$B1 = (G3 \wedge G2 \wedge G1) = B2 \wedge G1$$
$$B0 = (G3 \wedge G2 \wedge G1 \wedge G0) = B1 \wedge G0$$

where the character $\wedge$ indicates the XOR operator in the Verilog. The logic diagram is shown below, and the Verilog RTL is described in the Example.

Example 2 RTL description of Gray to Binary code converter

```
////////////////////////////////////////////////////////////////////////////////
module code_converter

                    ( output B3, B2, B1, B0,

                      input G3, G2, G1,G0

                    );

assign B3=G3;
assign B2 = G3 ^ G2;
assign B1 = G3 ^ G2 ^  G1;
assign B0 = G3 ^ G2 ^  G1 ^ G0;

endmodule

////////////////////////////////////////////////////////////////////////////////
```

For the FPGA based designs the technology schematic uses the LUTs and shown in Figure (Figs. 3.3 and 3.4).

3.5 Multiplexers

The multiplexers are popular in the ASIC and FPGA design as data selectors, and the multiplexers are treated as universal logic. Using the multiplexers, the desired combinational function of\r logic is implemented. The effective way to infer the multiplexer is using the continuous assign construct with the use of the conditional operator. Consider 2:1 multiplexer (Table 3.6).

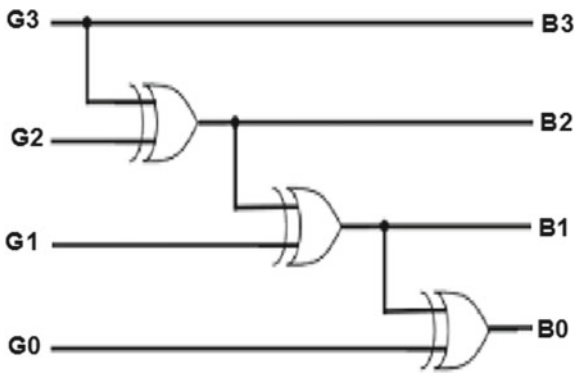


Fig. 3.3 4-bit gray to binary code converter

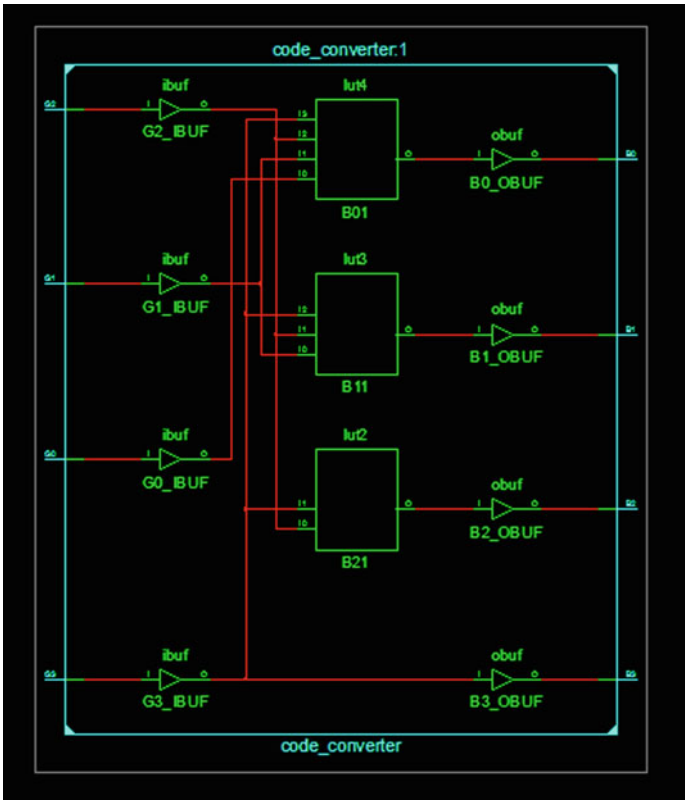


Fig. 3.4 Technology schematic for 4-bit gray to binary code converter

Table 3.6 The 2:1 multiplexer

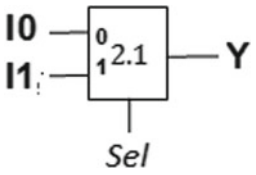
Control_Input (Sel)	Output (Y)	Description
0	I0	For Sel = 0 an output Y = I0
1	I1	For Sel = 1 an output Y = I1

```
////////////////////////////////////////////////////////////////////////////////
module mux_2to1( input I1, I0, Sel, output Y );

assign Y = (Sel) ? I1 : I0; // conditional operator is used to infer 2:1 MUX


endmodule

////////////////////////////////////////////////////////////////////////////////
```



As shown for Sel = 1 Y = I1 and for Sel = 0 Y = I0.

Using the multiple ***assign*** constructs and by using the conditional operator, the 4:1 multiplexer is implemented.

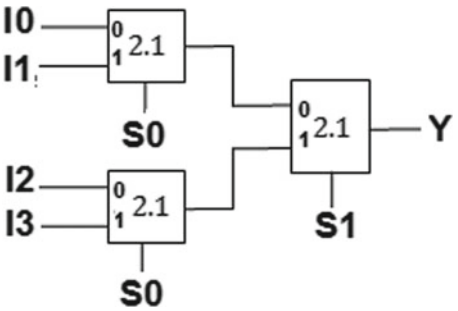
```
//////////////////////////////////////////////////////////////////////////////////
module mux_4to1( input I3, I2, I1, I0,  S1, S0, output Y );

wire Y1, Y0;

assign Y0 = (S0) ? I1 : I0;
assign Y1 = (S0) ? I3 : I2;
assign  Y = (S1) ? Y1 : Y0;


endmodule
//////////////////////////////////////////////////////////////////////////////////
```

The logic inferred is shown in the Figure, and it consists of the three 2:1 multiplexer. The issue with the design is the cascaded stages that is if each multiplexer has 1 ns propagation delay, the logic inferred will have the 2 ns propagation delay.



3.6 Cascading Stages of MUX Using Instantiation

```
////////////////////////////////////////////////////////////////////////////////
module Mux_16to1( input[15:0] I , input [3:0] S, output Y );

wire Y3, Y2, Y1, Y0;

mux_4to1   U0  ( .I3(I[3]),
                 .I2(I[2]),
                .I1(I[1]),
                .I0(I[0]),
                .S1(S[1]),
                .S0(S[0]),
                .Y(Y0)
                 );

mux_4to1  U1  ( .I3(I[7]),
                .I2(I[6]),
                 .I1(I[5]),
                .I0(I[4]),
                .S1(S[1]),
                .S0(S[0]),
                 .Y(Y1)

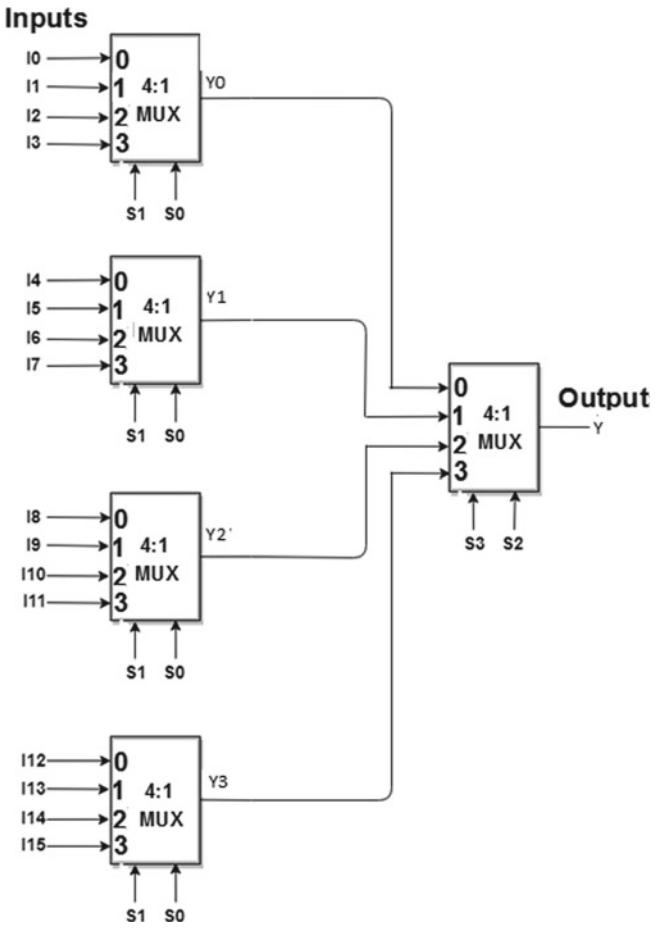
                );
 mux_4to1  U2  ( .I3(I[11]),
                 .I2(I[10]),
                .I1(I[9]),
                .I0(I[8]),
               .S1(S[1]),
                .S0(S[0]),
                .Y(Y2)
                );
```

```
mux_4to1  U3  ( .I3(I[15]),
                .I2(I[14]),
                .I1(I[13]),
                .I0(I[12]),
                .S1(S[1]),
                 .S0(S[0]),
                 .Y(Y3)
                 );

 mux_4to1 U4 ( .I3(Y3),
                .I2(Y2),
                 .I1(Y1),
                 .I0(Y0),
                 .S1(S[3]),
                 .S0(S[2]),
                 .Y(Y)
                 );


endmodule
//////////////////////////////////////////////////////////////////////////////////
```

The logic inferred consists of five four to one multiplexers and is inferred due to the component instantiation.



If we consider the ASIC-based designs, then the multiplexers are useful in following tasks.

1. The multiplexers are used to select from many clock frequencies and used as clock selector.
2. Used for the pin multiplexing to minimize the pin count of the design
3. Used as combinational logic to select from one of the data inputs.

3.7 Decoders

The decoders are extensively used in the ASIC- and FPGA-based designs to select one of the devices to establish communication. In the decoders, one of the outputs is active at a time depending on the status of the select inputs.

Consider the 2:4 decoder which has two select inputs and four outputs. The decoder is enabled when Enable = 1. Table 3.7 describes the relationship between the select lines and output lines.

Strategy During RTL Design: To infer the decoders which has 16 or 32 inputs, we can use the logic duplication concept where the 'Enable' input of the decoder can be controlled by the input side decoder. The designer can use the module instantiation or the RTL design using the case construct to infer the required logic (Table 3.8).

Table 3.7 Truth table 2:4 decoder

Enable	Select inputs (I1 I0)	Outputs (Y3 Y2 Y1 Y0)
1	00	0001
1	01	0010
1	10	0100
1	11	1000
0	xx	0000

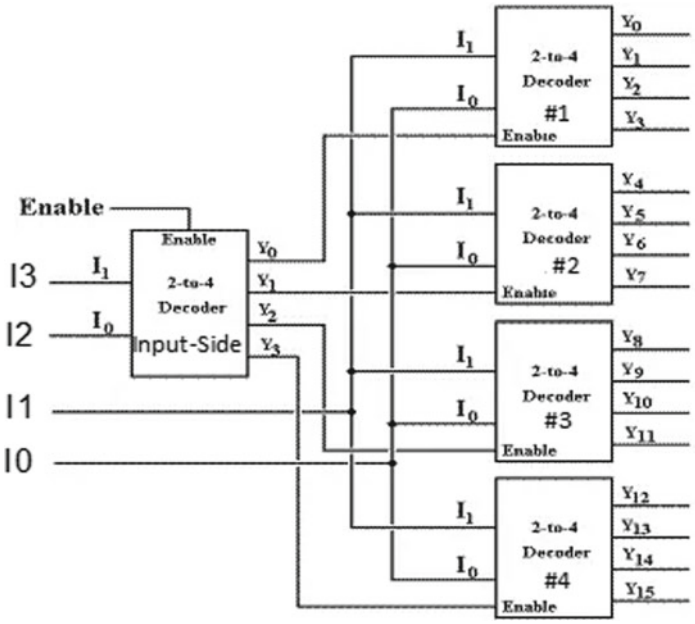
Table 3.8 Truth table 4:16 decoder

Enable	Select inputs (I3 I2 I1 I0)	Output in binary
1	0000	0000_0000_0000_0001
1	0001	0000_0000_0000_0010
1	0010	0000_0000_0000_0100
1	0011	0000_0000_0000_1000
1	0100	0000_0000_0001_0000
1	0101	0000_0000_0010_0000
1	0110	0000_0000_0100_0000
1	0111	0000_0000_1000_0000
1	1000	0000_0001_0000_0000
1	1001	0000_0010_0000_0000
1	1010	0000_0100_0000_0000
1	1011	0000_1000_0000_0000
1	1100	0001_0000_0000_0000
1	1101	0010_0000_0000_0000
1	1110	0100_0000_0000_0000
1	1111	1000_0000_0000_0000
0	xxxx	0000_0000_0000_0000

Table 3.9 The 2:4 decoder as selector

Enable	Select inputs (I3 I2)	Outputs (Y3 Y2 Y1 Y0)
1	00	Enable Decoder 1
1	01	Enable Decoder 2
1	10	Enable Decoder 3
1	11	Enable Decoder 4
0	xx	Disable all output decoders

If we observe the I3, I2 inputs, then the binary value remains same for the group of consecutive four entries starting from 0 and can be used to design the inputs decoder. The input decoder is used to select one of the output decoders at a time (Table 3.9).



From the figure, it is clear that the input-side decoder is used to enable one of the output decoders at a time. The logic duplication is popular technique and useful in the ASIC and FPGA designs to replicate the same logic. This technique is especially useful in the FPGA-based designs to reduce the number of LUTs. For example, if we need to infer the 8:256 decoder, then using the case, construct the logic inferred in having a greater number of LUTs. So, larger case constructs can be splitted into multiple using the logic duplication techniques. For ASIC, we can think of using the logic duplication depending on the specific scenarios as it impacts on the area of functional block.

Table 3.10 The 4:2 encoder truth table

Select inputs (I3 I2 I1 I0)	Outputs (Y1 Y0)	Status
1000	11	0
0100	10	0
0010	01	0
0001	00	0
0000	00	1

3.8 Encoders

The encoder performs the reverse operation as compared to decoder. Consider the four level-sensitive inputs and the design need to generate an output depending on the logic high value at one of the inputs.

If I3 = 1 and I2, I1 and I0 are logic 0, then the encoder generates valid output as Y1 = 1 and Y0 = 1. As outputs are valid, the status output is 0.

If all inputs are logic zero, then Status = 1 and indicates the invalid output and should be ignored by other design which uses encoder outputs as inputs.

As shown in Table 3.10, it is assumed that one of the inputs is logic '1' at a time, but in practical systems, the assumption does not hold good as many inputs can be active high at a time. So, to cater the need of the priority scheduling, it is essential to design the priority encoders.

3.9 Priority Encoders

The priority encoders are used in the design to sample the high priority input and to generate an valid output.

Consider the design having four inputs I3, I2, I1, I0, and I3 has highest priority as compared to other inputs. Table 3.11 describes the functionality where I3 has highest priority and I0 has lowest priority.

Table 3.11 The 4:2 priority encoder truth table

Select inputs (I3 I2 I1 I0)	Outputs (Y1 Y0)	Status
1xxx	11	0
01xx	10	0
001x	01	0
0001	00	0
0000	00	1

The RTL should be coded using the nested if-else construct and should infer the priority logic.

```
//////////////////////////////////////////////////////////////////////////////////


module  priority_encoder (
                    input    I3, I2, I1, I0,

                   output  reg  Y1, Y0, flag
                          );

always @ *
begin

 if ( I3)
          begin
           Y1 = 1;
           Y0=1;
           flag = 0;
           end
           else if ( I2)
          begin
           Y1 = 1;
           Y0=0;
           flag=0;
           end
            else if ( I1)
          begin
           Y1 = 0;
```

```
            Y0=1;
            flag=0;
            end
             else if (I0)
           begin
            Y1 = 0;
            Y0=0;
            flag=0;
           end

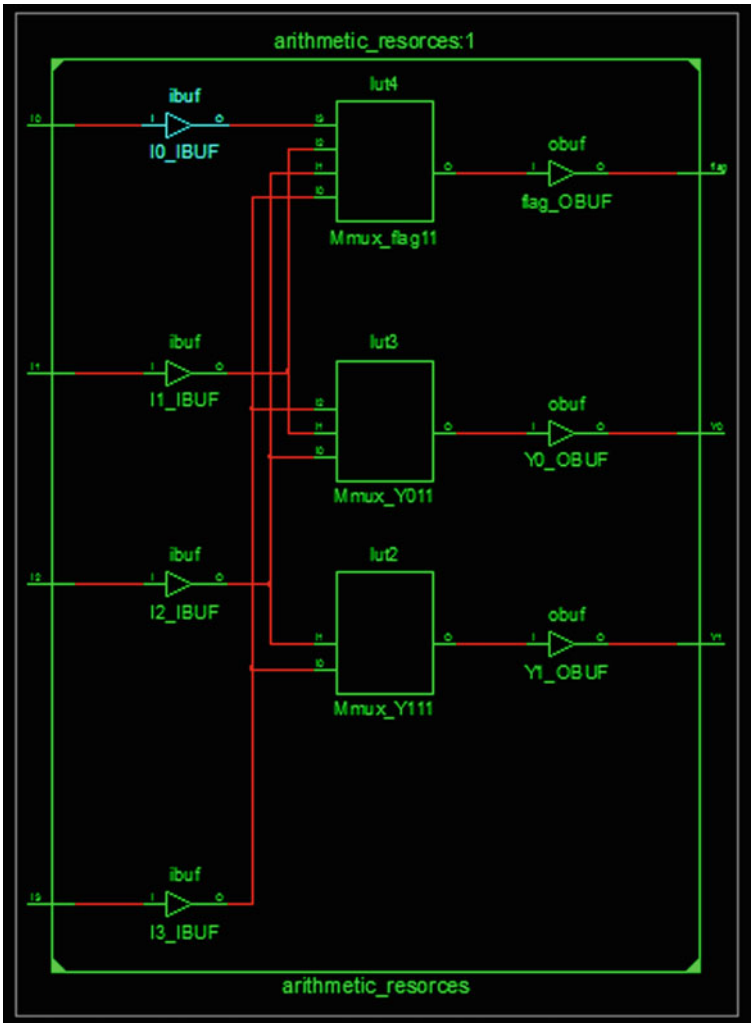
           else
            begin
           Y1 = 0;
            Y0=0;
           flag=1;
           end

end

endmodule

//////////////////////////////////////////////////////////////////////////////////
```

The FPGA synthesis is shown in the figure, and the technology schematic consists of three lookup tables. FPGA important resource is CLB, and it consists of the LUTs and slice registers.



3.10 Strategies During ASIC Design

During the ASIC design flow at the RTL design phase, try to use the following strategies.

1. Use the synthesizable constructs and do not use the #delays in the RTL design.
2. Try to avoid the use of multiple assign constructs as due to parallel execution the area may be higher.

3. Try to use ***always*** **@ *** to model the combinational logic. It includes the required inputs in the sensitivity list.
4. Use the blocking assignments to code the RTL using the always procedural block.
5. Try to use the naming conventions such as net_name_in, net_name_out for better readability.
6. Try to avoid the combinational looping as they have oscillatory behavior.
7. Try to use the parameter to have parameterized design.
8. Use the ***case*** construct to infer the parallel logic and nested ***if-else*** to infer the priority logic.
9. While using ***casez*** and ***casex*** take care of the dangling inputs and synthesis and simulation mismatches.
10. Try to divide the large number of assignments using multiple ***case*** constructs.

3.11 Exercises

1. Implement the digital logic to detect the even parity from the string of 8-bit binary data? What should be the RTL design strategy for ASIC design?
2. Work on the logic development of 4-bit multiplier. Find out the resource requirement for the 4-bit multiplier for the shift and add method.
3. If propagation delay of the full adder is 1 ns to generate the sum and 2 ns to generate the carry output, and if the 16-bit ripple carry adder is to be designed, then find the overall propagation delay to get the add (A, B)
4. Can you think of the high-level design strategy to pass data between the pipelined processor and the memory and IOs. If the four memory blocks of 16 Kbyte each and 128 IO devices need to establish communication with the processor, then what should be the decoding logic?
5. Sketch the logic and optimize the area required for following

   (a) A OR B
   (b) A AND B
   (c) A XOR B
   (d) NOT A

Consider A, B are 8-bit binary inputs, and to perform only one operation at a time, the design has 2-bit control inputs (control_operation).

3.12 Chapter Summary

Following are important points to conclude the chapter.

1. Use the resource sharing techniques to optimize the logic for the better data path optimization.

2. The multiplexer is universal logic, and the chain of multiplexers can be used to select from one of the inputs.
3. Multiplexers are used to have pin multiplexing.
4. Decoders are used as selection logic in the system design.
5. Priority encoders are used to generate the output by analysing the priority of inputs.

Chapter 4
Sequential Design Concepts

If we consider the ASIC design cycle then at various stages, we need to play around with the sequential circuits may be using the counters, shift registers, memories and other clock-based circuits and clock dividers. In such scenario, we may encounter few issues where we need to tweak the RTL or architecture to improve the performance of ASIC. By considering all the above points, the chapter discusses about important sequential elements with their use and the design of synchronous and asynchronous sequential circuits.

4.1 Sequential Design Elements

In the sequential circuits, an output is function of the present input and past output. Table 4.1 discuss about the important sequential elements and their use during the design phase. For the synchronous or asynchronous designs, we will try to use the edge-sensitive elements that are D flip-flops and our objective is to design the sequential circuit with speed as the timing goal.

Understanding of the synchronous and asynchronous design techniques plays important role during the design cycle.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

Table 4.1 Important sequential elements

Sequential elements	Description
Latch	Level sensitive and used in the latch-based designs. Latch-based clock gating and few scenarios in DFT for time borrowing
Flip-flop	Edge sensitive and mainly used in the design of asynchronous and synchronous sequential circuits. Application can be counters, shifters, design with the registered inputs and registered outputs
Counters	The synchronous or asynchronous counters where counting happens on the active edge of the clock. Used to count the predefined sequence and also used as clock dividers
Shift-registers	Sensitive on the active edge of clocks and designed using D flip-flops and used to perform the right, left shift and rotate operations
Asynchronous circuits	Can be used to generate clocks but incurs more delay so avoid the use of the asynchronous circuits
Clock dividers	The PLL clock can be divided to get the internally generated clock using the D flip-flops (Toggle flip-flop designed using D flip-flop)
Finite state machine (FSM) controller	For the larger ASIC designs different FSM controllers can be used to detect the sequence with the glitch free and clean output
Random number generators	Used to generate the random numbers for specific requirements using edge-sensitive elements and suitable combinational elements

4.2 Let Us Understand Blocking Versus Non-blocking Assignments

Important Verilog assignments used within the procedural block 'always' and 'initial' are

- Blocking Assignments (BA)
- Non_Blocking Assignments (NBA).

The BA assignments are evaluated and updated in the active event queue, and the right-hand side of the NBA is evaluated in the active event queue and updated in the NBA queue.

The name blocking indicates that it blocks all the future assignments unless and until present assignment is evaluated and updated, and hence, BA are not recommended to use in the sequential design. It is recommended to use the BA for the combinational design.

4.2.1 Blocking Assignments

The RTL description using the blocking assignments is shown in Example 1 and the intent is to infer the 3-bit shift register. But it infers the single flip-flop as the future assignment evaluation is blocked during the current assignment execution.

Example 1 RTL description using the blocking assignments

```
////////////////////////////////////////////////////////////////////////////////

module blocking_assignments( input data_in, clk, reset_n, output reg
data_out);

reg [1:0] tmp;

always @ ( posedge clk or negedge reset_n)
begin

        if ( ~ reset_n)
        begin
                { data_out, tmp } = 3'b000;

        end

        else
   begin
                tmp[0] = data_in;
                tmp[1] = tmp[0];
                data_out = tmp[1];

         end
end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

The RTL schematic is shown in Fig. 4.1 and the inferred logic has single flip-flop with asynchronous reset.

4.2.2 Reordering of the Blocking Assignments

If we reorder the blocking assignments as shown in Example 2, the RTL infers the 3-bit shift register, and it indicates that the order of the blocking assignments will play important role during the RTL description.

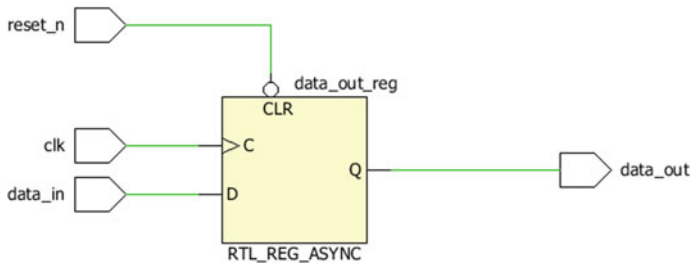


Fig. 4.1 RTL schematic for Example 1

Example 2 RTL description using the blocking assignments reorders

```
////////////////////////////////////////////////////////////////////////////////

module blocking_assignments( input data_in, clk, reset_n, output reg
data_out);

reg [1:0] tmp;

always @ ( posedge clk or negedge reset_n)
begin

        if ( ~ reset_n)
        begin
                { data_out, tmp } = 3'b000;

        end

        else
   begin
                data_out = tmp[1];
                tmp[1] = tmp[0];
                tmp[0] = data_in;

                end
end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

The RTL schematic is shown in Fig. 4.2, and the RTL design description (Example) infers the 3-bit shift register using the asynchronous active low reset.

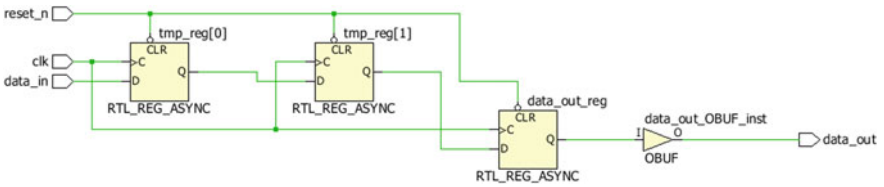


Fig. 4.2 RTL schematic for Example 2

4.2.3 Non-blocking Assignments

The RTL description using the non-blocking assignments is shown in Example 3 and the intent is to infer the 2-bit shift register, and it infers the 2-bit shifter using D flip-flops as the assignments executes concurrently within the ***begin-end***.

Example 3 RTL description using the non-blocking assignments

```
////////////////////////////////////////////////////////////////////////////////
module non_blocking_assignments( input data_in, clk, reset_n, output
reg data_out);

reg  tmp;

always @ ( posedge clk or negedge reset_n)
begin

        if ( ~ reset_n)
        begin
                { data_out, tmp } <= 2'b00;

        end

        else
                begin

                        data_out <= tmp;

                        tmp <= data_in;
                end
end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

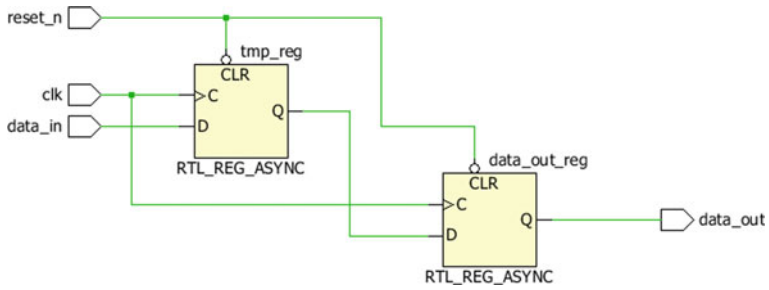


Fig. 4.3 RTL schematic for Example 3

The RTL schematic is shown in Fig. 4.3, and the RTL design description (Example 3) infers the 2-bit shift register using the asynchronous active low reset.

4.2.4 Reordering of the Non-blocking Assignments

Reordering of the non-blocking assignments as shown in Example 4, the RTL synthesis infers the 2-bit shift register, and it indicates that the order of the non-blocking assignments will not effect on the synthesis result of the design. The RTL schematic is shown in Fig. 4.3.

Example 4 RTL description using the non-blocking assignments reorders

```
////////////////////////////////////////////////////////////////////////////////
module non_blocking_assignments( input data_in, clk, reset_n, output
reg data_out);

reg  tmp;

always @ ( posedge clk or negedge reset_n)
begin

        if ( ~ reset_n)
        begin
                { data_out, tmp } <= 2'b00;

        end

        else
                begin

                        data_out <= tmp;

                         tmp <= data_in;
                end
end
endmodule
////////////////////////////////////////////////////////////////////////////////
```

Use the following recommendations during the ASIC RTL designs
1. Use the non-blocking assignments to model the sequential logic
2. Use the blocking assignments to model the combinational logic
3. Don't mix blocking and non-blocking assignments.

4.3 Latch-Based Designs

Latches are level sensitive and used in few applications such as latch-based clock gating in low-power ASIC designs and few other applications such as DFT scan chains with the latch borrowing.

Consider the functionality of the positive-level sensitive latch where the clk acts as latch enable input. When the latch enable is active high, the latch acts as transparent and Q = D where Q is an output of latch and D is data input. When latch is not enabled

that is clk = 0, the latch holds previous value as previous output Q is circulated through the even number of NOT gates.

Consider the CMOS implementation of positive-level sensitive latch where the CMOS switch-1 passes data for CLK = 1 to generate valid output Q = D. Latch is transparent during the positive level of the CLK. The even number of NOT gates is used in the forward path and during the negative level of CLK that is CLK = 0 the CMOS switch-2 is ON which holds the previous output Q.

So for the better understanding, the functionality is described in Table 4.2 (Fig. 4.4).

The timing sequence for the positive-level sensitive latch is shown in Fig. 4.5 and the latch output Q = D during the positive level of the CLK. During negative level of clk, it holds the previous output value.

RTL Design Strategy

The Intentional latches can be inferred if during the RTL design else clause is eliminated. Consider the 4-bit latch coded using the Verilog using the if-else construct within the always @ * procedural block. As discussed in Chap. 3 'if-else' generates the 2:1 multiplexer. Now to infer the intentional latches eliminate the 'else' condition.

Table 4.2 The D latch truth table

CLK = Enable	D	Q	CMOS switch status
1	0	0	CMOS Switch-1 is ON and CMOS Switch-2 is OFF. Q = D
1	1	1	CMOS Switch-1 is ON and CMOS Switch-2 is OFF. Q = D
0	X	Hold past output	CMOS Switch-1 is OFF and CMOS Switch-2 is ON. Q = Previous Output

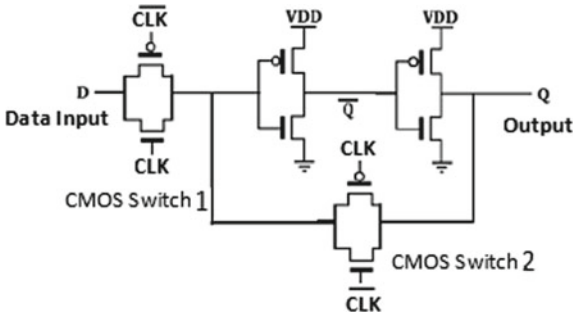


Fig. 4.4 D latch

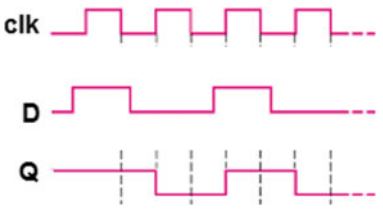


Fig. 4.5 Timing diagram of the D latch

During the ASIC or FPGA synthesis you will get the warning that: Latches may be generated from incomplete case or if statements. We do not recommend the use of latches in designs, as they may lead to timing problems.

Example 5 is RTL description of 4-bit latch and coded using the Verilog synthesizable constructs.

Example 5 RTL description of 4-bit latch

```
////////////////////////////////////////////////////////////////////////////////
module  latch_4bit (
                    input [3:0]  d_in,
                    input   clk,
                   output  reg  [3:0]  q_out
                     );

always @ *

begin

  if (clk)

          q_out <= d_in;

end

endmodule

////////////////////////////////////////////////////////////////////////////////
```

The four-bit latch schematic is shown in Fig. 4.6, and latch is level sensitive as D is sampled during the active high level of CLK.

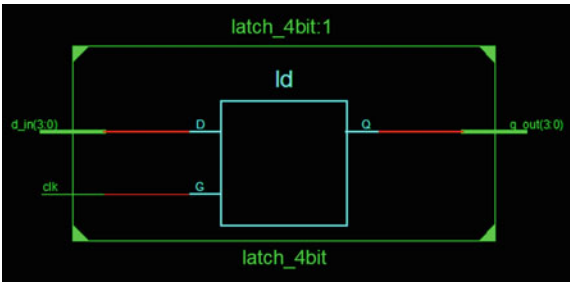


Fig. 4.6 RTL schematic for Example 5

4.4 Flip-Flop-Based Designs

The flip-flops are edge sensitive; that is, they either operate on the rising edge (low to high transition) or falling edge (high to low transition) and used as sequential design element in the synchronous or asynchronous designs.

The falling edge D flip-flop is shown in Fig. 4.7, and on the high to low transition of the clk, the flip-flop samples the data input D to generate valid output.

The functional Table 4.3 describes the relationship between input and output on the rising edge of clk.

The latch based implementation of the negative edge-sensitive flip-flop is shown in Fig. 4.7. As shown, the flip-flop has two latches connected in cascade. The master latch is positive-level sensitive, and the slave latch is negative-level sensitive.

The timing sequence for the negative edge-sensitive flip-flop is shown in the Fig. 4.8. As shown, the master latch output Q1 changes on the positive level of clk and slave latch output Q changes on the negative level of clock. The output from flip-flop Q is sensitive to the falling edge of clk.

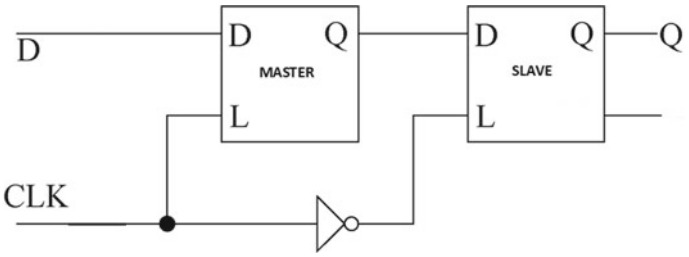


Fig. 4.7 D flip-flop

Table 4.3 Truth-table of negative edge-triggered flip-flop

CLK	D	Q
Falling edge	0	0
Falling edge	1	1
Rising edge or level	X	Hold past output

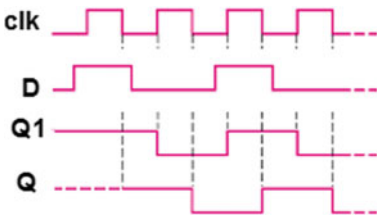


Fig. 4.8 Timing diagram of D flip-flop

The flip-flop or parallel-in-parallel-out registers are used in the ASIC design to hold the data or to change the data on the active edge of the clock.

The edge sensitive elements are inferred if we use the procedural block ***always*** with event control as ***posedge*** or ***negedge***.

To infer the edge sensitive element use either one from following
always @ (posedge clk),
always @ (negedge clk)

Example 6 is the RTL description of the 4-bit parallel-in-parallel-out register. The schematic is the representation of the 4-bit register which is rising edge-triggered and uses four flip-flops (Fig. 4.9).

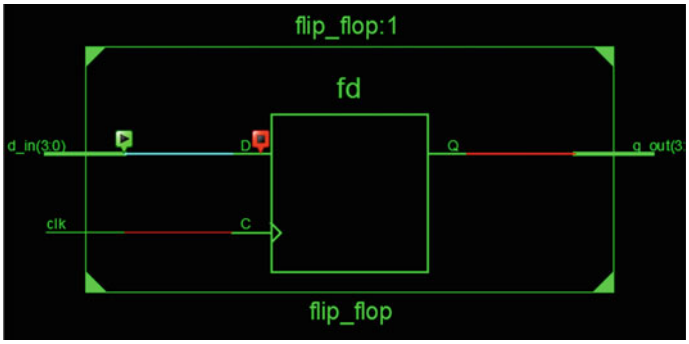


Fig. 4.9 RTL schematic for Example 6

Example 6 RTL description of 4-bit parallel-input-parallel-output register

```
////////////////////////////////////////////////////////////////////////////////
module   flip_flop   (

                          input [3:0] d_in,
                         input clk,
                        output reg [3:0] q_out
                    );

always @ (posedge clk)

begin

   q_out <= d_in;


end

endmodule

////////////////////////////////////////////////////////////////////////////////
```

4.5 Reset Strategies

For the FPGA and ASIC designs, the role of reset to initialize the design plays an important rule. During the ASIC implementation to synchronize the internal resets with the master reset input at various stages, we need to deploy the reset synchronizers.

The dedicated reset tree and reset network can be used to generate the reset signal required for the different functional blocks.

For the reset, we need to consider the

- Reset Recovery Time
- Reset Removal Time.

The resets are of two types.

1. **Asynchronous Reset**
2. **Synchronous Reset**.

4.5.1 Asynchronous Reset

If the reset need to be used to initialize the design irrespective of the active edge of the clock, then the reset is treated as asynchronous reset. The ASIC RTL design which uses the D flip-flop is shown in Example 7.

Example 7 RTL description of the asynchronous-reset D flip-flop

```
////////////////////////////////////////////////////////////////////////////////
module flip_flop(

                    input  d_in,
                     input clk, reset_n,
                     output reg q_out

                                                          );

always @ (posedge clk or negedge reset_n)

begin

          if ( ~ reset_n)
          q_out <= 1'b0;

          else

         q_out <= d_in;


end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

The reset is asynchronous input (reset_n) and the RTL schematic is shown in Fig. 4.10.

The testbench for Example 8 is described using the non-synthesizable constructs and is shown in example, and the sanity check result is shown in the waveform (Fig. 4.11).

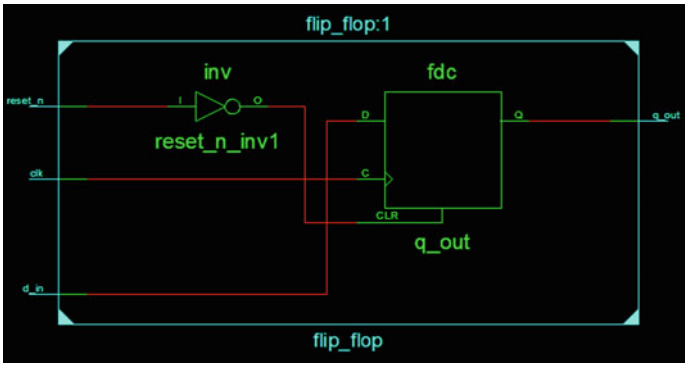


Fig. 4.10 Asynchronous-reset logic

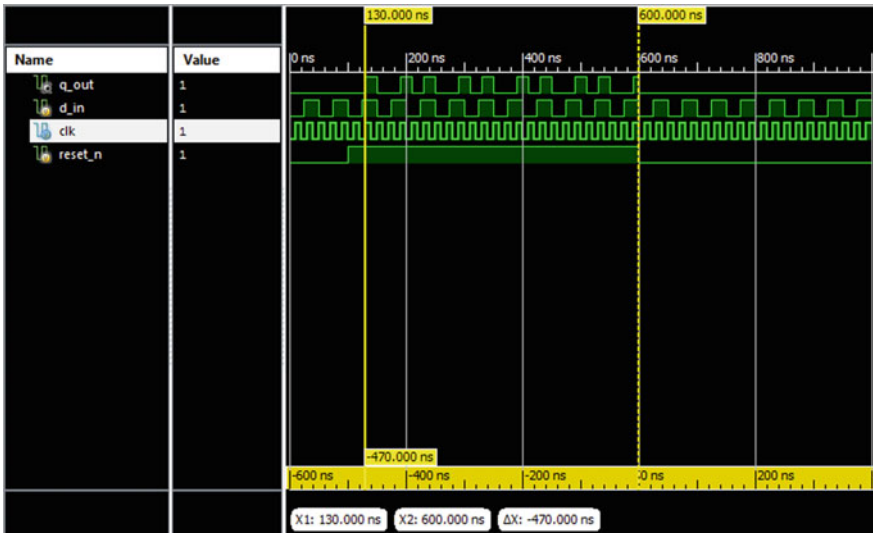


Fig. 4.11 Asynchronous-reset sampling

Example 8 Testbench for the D flip-flop

```
////////////////////////////////////////////////////////////////////////////////

module test_flip_flop;

        // Inputs
        reg d_in;
        reg clk;
        reg reset_n;
        // Outputs
        wire q_out;

        // Instantiate the Unit Under Test (UUT)
                flip_flop  uut  (
                .d_in(d_in),
                .clk(clk),
                .reset_n(reset_n),
                .q_out(q_out)
        );

always #10 clk=~clk;

// Add stimulus here

always # 25 d_in = ~d_in;

        initial begin
                // Initialize Inputs
                d_in = 0;
                clk = 0;
                reset_n = 0;

                // Wait 100 ns for global reset to finish
                #100;

                reset_n =1'b1;


                #500 reset_n = 1'b0;

                #400 $finish;


        end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

As shown in the simulation waveform the reset is sampled irrespective of the active edge of clock and used to initialize the state of flip-flop to ‘0’.

4.5.2 Synchronous Reset

If the reset need to be used to initialize the design on, the active edge of the clock then the reset is treated as synchronous reset. The ASIC RTL design which uses the D flip-flop is shown in Example 9.

Example 9 RTL description of the synchronous-reset D flip-flop

```
////////////////////////////////////////////////////////////////////////////////
module flip_flop (

                    input d_in,
                    input clk, reset_n,
                    output reg q_out

                   );

always @ ( posedge clk )

begin

          if ( ~ reset_n)

          q_out <= 1'b0;

          else

           q_out <= d_in;

end

endmodule
////////////////////////////////////////////////////////////////////////////////
```

As the always procedural block is sensitive to positive edge of the clock, the reset is checked within the always procedural block and it reset_n = 0; the output of the flip-flop is initialized to logic '0'.

The testbench for Example 9 is described using the non-synthesizable constructs and is shown in Example 8, and the sanity check result is shown in Fig. 4.12.

As shown in the simulation waveform, the reset is sampled on the active edge of clock and used to initialize the state of flip-flop to '0'.

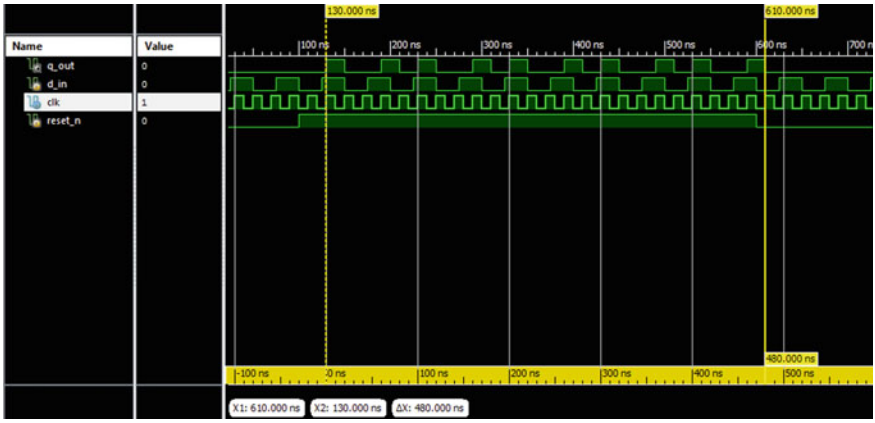


Fig. 4.12 Synchronous-reset sampling

4.6 Frequency Divider

Let us consider the use of the flip-flop in the ASIC or FPGA designs. We can use the D flip-flops with additional combinational logic if required to design the clock divider.

Consider that the PLL generates the 550 MHz square wave and used as a clock. For internal logic, we need to use the 225 MHz clock; then in such scenario, we can design the divide-by-two circuits. The circuit can be designed using the D flip-flop where the output complement of Q is feedback to the input D (Fig. 4.13).

Example 10 RTL description of the clock divider logic

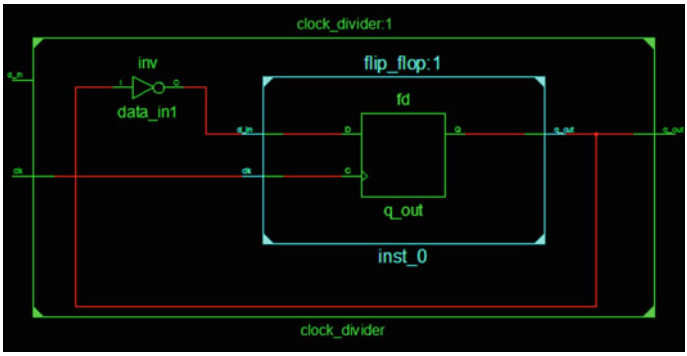


Fig. 4.13 Clock divider logic

```
////////////////////////////////////////////////////////////////////////////////
module clock_divider (
                        input d_in,
                        input clk,reset_n,
                       output q_out

                                                    );

wire data_in;
flip_flop inst_0 (

                .d_in(data_in),
                 .clk(clk),
                .reset_n(reset_n),
                .q_out(q_out)

                );

assign data_in = ~q_out;
endmodule


////////////////////////////////////////////////////////////////////////////////
```

As shown in Example 11 for the divide-by-two circuits is described using the non-synthesizable constructs. The simulation waveform for the design is shown in the Fig. 4.14. The synthesis outcome is shown in the Fig. 4.15.

Example 11 Testbench to check for the clock division

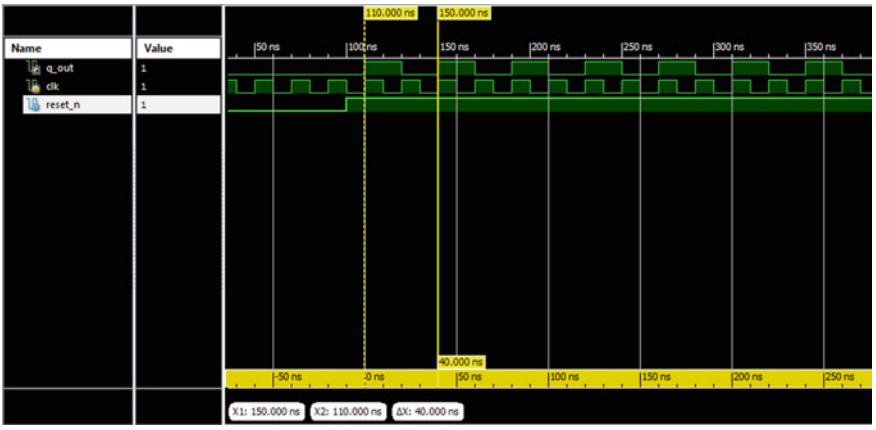


Fig. 4.14 Waveform for Example 11

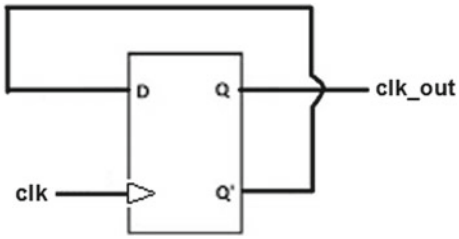


Fig. 4.15 ASIC divide-by-two clock logic

```
////////////////////////////////////////////////////////////////////////////////

module divide_by_2_test;

        // Inputs
        reg clk;
        reg reset_n;

        // Outputs
        wire q_out ;

        // Instantiate the Unit Under Test (UUT)
        clock_divider   uut  (
                                .clk(clk),
                                .reset_n(reset_n),
                                .q_out(q_out)
                        );

        initial begin
                // Initialize Inputs
                clk = 0;

                reset_n =1'b0;

                #100;
```

```
                // Wait 100 ns for global reset to finish
                reset_n = 1'b1;
                #500 reset_n = 1'b0;

                end
        always # 10 clk = ~ clk;
endmodule
```

Issue of using above RTL during ASIC Design: With above Verilog RTL we can get the divide by 2 sequential circuits which are sensitive to rising edge of the clock. But the design has issue due to use of the NOT gate as it increases the data required time. This limits the maximum frequency of the design.

How to fix the Issue: To improve the maximum frequency for the clock divider use fed the complement of Q from the flip-flop directly to data input D. For more details refer Chaps. 5 and 10.

4.7 Synchronous Design

In the synchronous design the all sequential element that is D flip-flops uses the clock generated from the common clock source. For the better performance of the ASIC and to have the clean timing, it is recommended to use the synchronous design.

The 2-bit binary up-counter which is used to count the sequence 00-01-10-11-00…. is described in the RTL (Example 12) using the synthesizable constructs. The RTL design uses the asynchronous active low reset.

Example 12 RTL description of the 2-bit synchronous counter

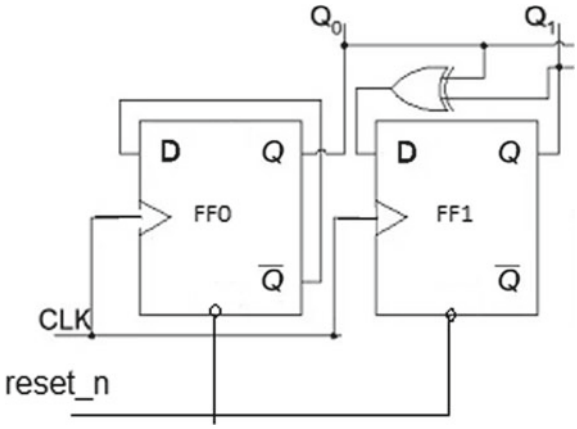


Fig. 4.16 2-bit synchronous binary up-counter schematic

```
////////////////////////////////////////////////////////////////////////////////
module Binary_up_counter (
                        input clk,reset_n,
                        output reg [1:0] Q
                        );

always @ (posedge  clk or negedge  reset_n)
begin

 if (~ reset_n)

          Q <= 2'b00;

else

  if ( Q== 2'b11)

             Q <= 2'b00;

  else

             Q <= Q+1;
end
endmodule
////////////////////////////////////////////////////////////////////////////////
```

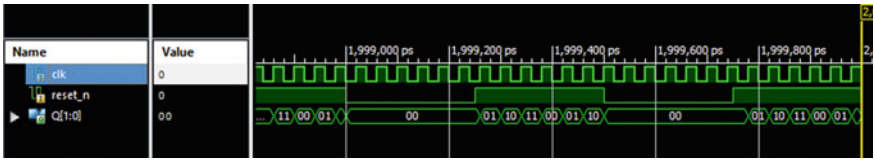


Fig. 4.17 Timing diagram of the 2-bit binary up-counter

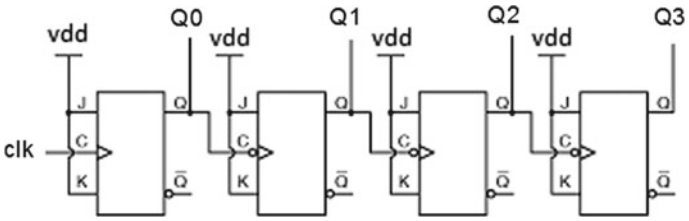


Fig. 4.18 Asynchronous counter as clock divider

The RTL description (Example 12) infers the logic and shown in Fig. 4.16. The simulation waveform for the 2-bit counter is shown in Fig. 4.17.

4.8 Asynchronous Design

In the asynchronous design the clock of all the sequential elements is not from the common clock source. As shown in Fig. 4.18, the design uses the rising edge-sensitive toggle flip-flops, and the design is used to generate the internal clocks.

If the clock frequency at clk input is 500 MHz, then the internally generated clocks are the following

- Clock output Q0 having frequency of the 250 MHz.
- Clock output Q1 having frequency of the 125 MHz.
- Clock output Q2 having frequency of the 62.5 MHz.
- Clock output Q3 having frequency of the 31.25 MHz.

The issue with the design is the asynchronous clocking, and hence, it is very dangerous to use such kind of designs in the ASIC.

4.9 RTL Design and Verification for Complex Designs

For the complex designs during the RTL design and verification phase, use the following strategies.

1. Try to understand the architecture and micro-architecture and partition the logic to have the efficient RTL description using the moderate gate count blocks.
2. Use the bottom-up approach and during the top-level design try to deploy the synchronizers if required.
3. Use the synthesizable constructs during the RTL design and non-synthesizable constructs during the RTL verification.
4. Use the blocking assignment to model the combinational logic (glue logic between the registers) and non-blocking assignments for the sequential designs
5. Do not mix the blocking and non-blocking assignments.
6. Use the optimization constraints at the RTL level to improve the performance. Refer the subsequent chapters to have better understanding of the design and optimization.
7. Have the better verification architecture and the verification planning for the design.
8. Understand the coverage requirements and work on the verification strategies to achieve the specified coverage goals.

The discussion on the RTL verification topic is not an objective of the book. The subsequent chapters of this book are useful to understand the ASIC design terminology and concepts.

4.10 Exercises

1. Implement the RTL using the synthesizable Verilog constructs to infer following logic (Fig. 4.19).

2. Implement the RTL using the synthesizable Verilog constructs to infer following logic. Use asynchronous active low reset and positive edge-sensitive elements (Fig. 4.20).

3. Implement the RTL using the synthesizable Verilog constructs to infer following logic (Fig. 4.21).

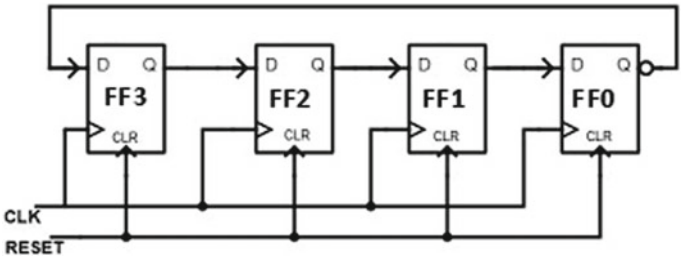


Fig. 4.19 Exercise 1

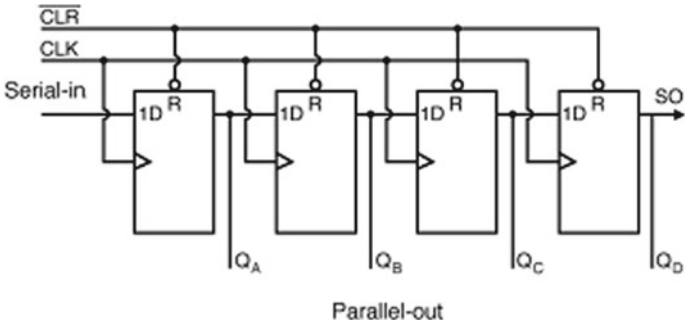


Fig. 4.20 Exercise 2

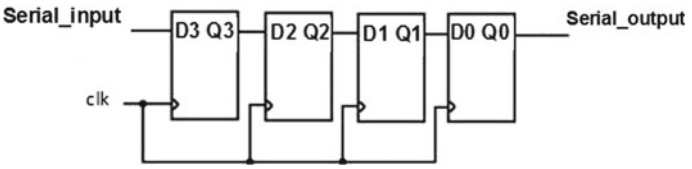


Fig. 4.21 Exercise 3

4.11 Chapter Summary

The following are the important points to conclude the chapter.

1. Use the non-blocking assignments to model the sequential logic.
2. Use the blocking assignments to model the combinational logic.
3. Do not mix blocking and non-blocking assignments.
4. The resets are of the two types asynchronous: reset and synchronous reset.
5. In the synchronous design, the all sequential element that is D flip-flops uses the clock generated from the common clock source.
6. In the asynchronous design, the clock of all the sequential elements is not from the common clock source.
7. Avoid the internally generated clocks or asynchronous clocking in the design.

Chapter 5
Important Design Considerations

If we use the synchronous sequential designs or any IP in the design or to finalize the architecture and micro-architecture, then we need to work out on various strategies. Few of them are listed below:

1. Functionality of the design and compatibility
2. Parallelism, concurrency, and pipelining strategies
3. External IO and high-speed interfaces
4. Area and initial gate count estimation for the design
5. Speed and maximum frequency requirements
6. Power requirements and use of the low-power design cells
7. Clock network and latency
8. Interface and IO delays and modeling strategies
9. Effect of the on-chip variation on the design
10. IPs required and timing requirements
11. Memory requirements and different micros
12. Top and block-level design constraints.

By considering all above, the team of experienced technical members finalizes the architecture and micro-architecture of the ASIC/SOC design.

The chapter discusses few of the design considerations for ease of understanding of the architecture and the case studies.

During the architecture and micro-architecture design, important considerations are speed, power, and area.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

5.1 Timing Parameters

Important sequential circuit timing parameters are shown in Fig. 5.1 for rising edge-sensitive flip-flop, and they are:

1. Setup time (t_{su})
2. Hold time (t_h)
3. Propagation delay of flip-flop (t_{pd}).

These timing parameters for the flip-flop are discussed in this section.

- **Setup Time (t_{su})**: The minimum amount of time for which the data input of the flip-flop should maintain the stable value prior to arrival of the active edge of the clock is called as setup time.
  Here, the active edge indicates the low to high transition for rising edge (positive edge)-sensitive flip-flop, and the high to low transition for falling edge (negative edge)-sensitive D flip-flop.
  During the setup time window if the data input changes either from 1 to 0 or vice versa, then the flip-flop output will be metastable which indicates the setup violation. For more details, refer Chaps. 12 and 15.
- **Hold Time (t_h)**: The minimum amount of time for which the data input of the flip-flop should maintain the stable value after the arrival of the active edge of the clock is called as hold time.
  Here, the active edge indicates the low to high transition for rising edge (positive edge)-sensitive flip-flop, and the high to low transition for falling edge (negative edge)-sensitive D flip-flop.
  During the hold time window if the data input changes either from 1 to 0 or vice versa, then the flip-flop output will be metastable which indicates the hold violation. For more details, refer Chaps. 12 and 15.

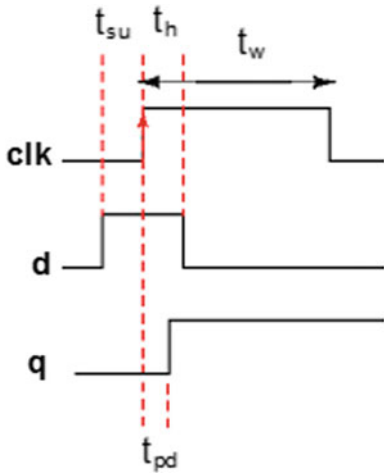


Fig. 5.1 Timing parameters for D flip-flop

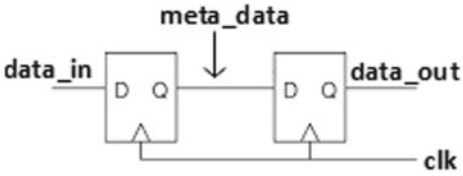


Fig. 5.2 Level synchronization concept

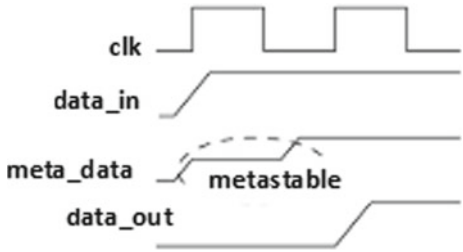


Fig. 5.3 Timing sequence for Fig. 5.2

- **Propagation Delay of Flip-Flop** ($t_{pd} = t_{cq}$): The amount of time required for which the flip-flop to generate the valid output after the arrival of the active edge of the clock is called as propagation delay of flip-flop.
  The propagation delay is also called as the **clock to q delay**, and it is also referred as t_{cq}.

5.2 Metastability

If the data input of the design shown in Fig. 5.2 is connected to the other module whose clk is generated from the different clock sources, then the first flip-flop output will go into metastable state. The meta_data indicates the flip-flop data is metastable and hence there is timing violation for the first flip-flop.

The metastability indicates the data output is not valid and to get the valid data output the design needs to use the multiflop-level synchronizers.

The timing sequence of the data sampled by the first flip-flop and the output of the second flip-flop is shown in Fig. 5.3. As shown, the output of the first flip-flop is in the metastable state and the data_out output from the output flip-flop is having the valid legal state.

5.3 Clock Skew

If multiple clocks are in the ASIC design, then the clock distribution and clock tree synthesis will play very important role to balance the clock skew between the different clock inputs of various blocks.

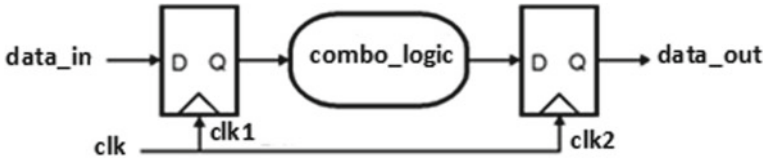


Fig. 5.4 Synchronous design

The clock tree synthesis which is named as CTS in this book is discussed in Chap. 16. To get basic details about the clock network, let us understand about the clock skew.

If two different clocks in design arrive at the different time instances, then the design has the clock skew.

The reason for clock skew is the routing delay that is wire delay for the single clock domain design. Consider the figure shown, and let us consider that the clk edge at the launch flip-flop is arrived at time instance 't0' and at capture flip-flop at time instance 't2'. As the clock arrival time is different for this synchronous design, there is phase shift between the clk1 and clk2 and we can consider this as clock skew.

Another reason we consider is the aging of the oscillator; then, there is cycle-to-cycle frequency variation of the clock generated from the oscillator and hence difference in the arrival time can be named as jitter.

In Fig. 5.4, the clock skew is due to the interconnect delays between the clk1 pin and clk2 pin.

Practically in the ASIC design, we experience the two different types of skew (Fig. 5.5).

1. **Positive clock skew**: It indicates the launch flip-flop clock (clk1) is triggered first and then the capture flip-flop clock (clk2) arrives. Figure describes the t_{skew}, and it is the difference between the arrival times of clk1 and clk2. In other words, we can imagine the positive clock skew is the data and clock running in the same direction and positive clock skew is better for the setup time but not good for hold time as there is positive margin to manage for!
2. **Negative clock skew**: It indicates the launch flip-flop clock (clk1) is triggered last and the capture flip-flop clock (clk2) is triggered first. Figure describes the t_{skew}, and it is the difference between the arrival times of clk1 and clk2. In other words, we can imagine the negative clock skew is the data and clock running in the opposite direction and negative clock skew is better for the hold time but not good for setup time!

In the ASIC design we always experience the clock skew due to jitter or due to interconnect that is wire delays and the following are important points which we should know.

1. Positive clock skew is good for the setup time but bad for the hold time.

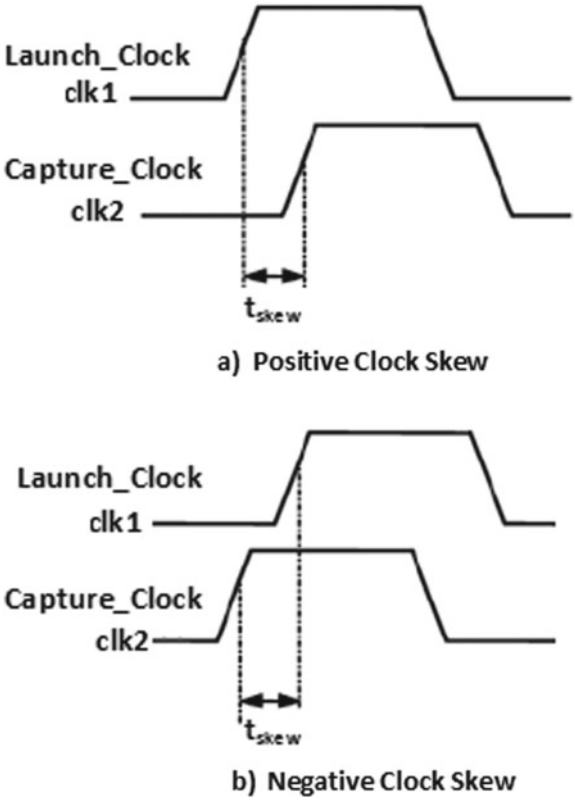


Fig. 5.5 Skew in the design

2. Negative clock skew is good for the hold time and bad for setup time.

5.3.1 Positive Clock Skew

As discussed before the positive clock skew where the launch flip-flop is triggered first and then capture flip-flop. There is margin of the buffer delay between the launch clock and capture clock and can be used to improve upon the required frequency for the design. The frequency calculation is discussed in Chap. 6.

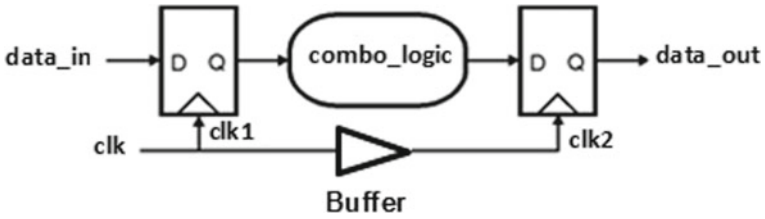


Fig. 5.6 Positive clock skew

Figure 5.6 shows the synchronous design with the positive clock skew and skew between the clk1 and clk2 is t_{buffer}.

Let us find the data required time and data arrival time.

$$\textbf{Data Arrival Time (AT)} = t_{\text{pff1}} + t_{\text{combo}}$$
$$\textbf{Data Required Time (RT)} = T_{\text{clk}} + t_{\text{buffer}} - t_{\text{su}}$$

where the T_{clk} is clock time period or clock to q delay, t_{buffer} is buffer delay, t_{su} is setup time of flip-flop, t_{pff1} is flip-flop propagation delay, and t_{combo} is combinational delay.

Setup slack is the difference between the data required time and data arrival time and should be positive. The positive setup slack indicates there is no any setup violation in the design.

To avoid the setup violations in the design, the design should have the fast data, fast launch clock (clk1), and slow capture clock (clk2). That is, the actual arrival of data should be fast as compared to the data required time (Fig. 5.7).

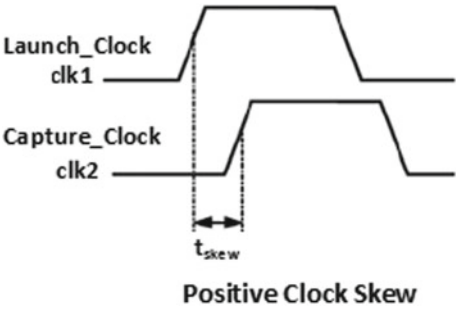


Fig. 5.7 Relationship between the launch and capture clock for the positive clock skew

5.3.2 Negative Clock Skew

As discussed before, the negative clock skew where the launch flip-flop is triggered last and the capture flip-flop is triggered first. As there is margin of the buffer delay between the launch clock and capture clock, this reduces the maximum frequency for the design. The frequency calculation is discussed in Chap. 6.

Figure 5.8 shows the synchronous design with the negative clock skew and skew between the clk1 and clk2 is t_{buffer}.

Let us find the data required time and data arrival time.

$$\textbf{Data Arrival Time (AT)} = t_{\text{buffer}} + t_{\text{pff1}} + t_{\text{combo}}$$
$$\textbf{Data Required Time (RT)} = T_{\text{clk}} - t_{\text{su}}$$

where the T_{clk} is clock time period or clock to q delay, t_{buffer} is buffer delay, t_{su} is setup time of flip-flop, t_{pff1} is flip-flop propagation delay, and t_{combo} is combinational delay (Fig. 5.9).



Fig. 5.8 Negative clock skew

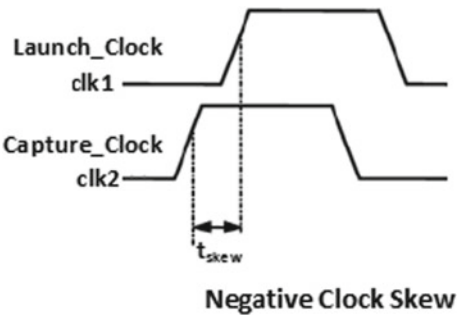


Fig. 5.9 Relationship between the launch and capture clock for the negative clock skew

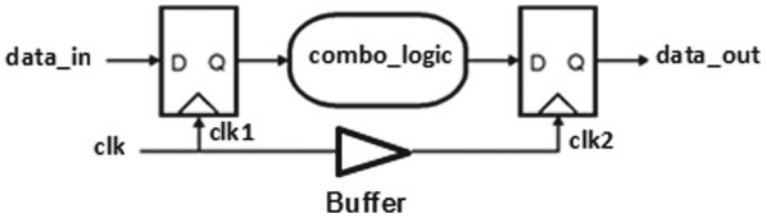


Fig. 5.10 Register-to-register path in synchronous design

5.4 Slack

During the ASIC design cycle, two terms are used for the slack that is setup slack and hold slack (Fig. 5.10).

5.4.1 Setup Slack

Setup slack is the difference between the data required time and data arrival time and should be positive. The positive setup slack indicates there is no any setup violation in the design.

$$\textbf{Data Arrival Time (AT)} = t_{\text{buffer}} + t_{\text{pff1}} + t_{\text{combo}}$$
$$\textbf{Data Required Time (RT)} = T_{\text{clk}} - t_{\text{su}}$$
$$\textbf{Setup Slack} = \textbf{RT} - \textbf{AT}$$

5.4.2 Hold Slack

Hold slack is the difference between the data arrival time and data required time and should be positive. The positive hold slack indicates there are no any hold violations in the design.

5.5 Clock Latency

The clock is generated form the PLL for the single clock domain designs and for the multiple clock domains we may need to have the multiple PLLs. The discussion on the PLL is ruled out during this book as it is analog component and needs to be designed by analog design team.

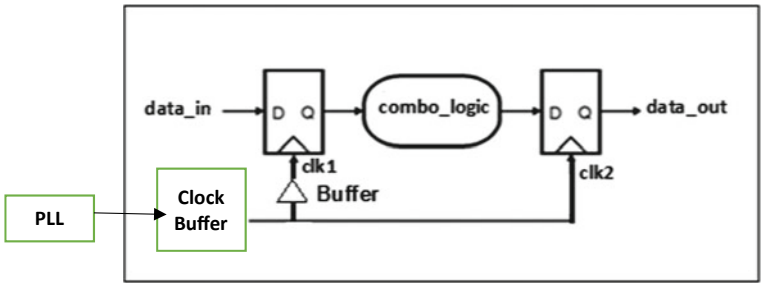


Fig. 5.11 Clock network latency

The clock network introduces the latency and it is effectively the time required for the clock to reach to the chip and the clock latency is due to the clock network delay during the clock distribution (Fig. 5.11).

5.6 Area for the Design

The overall area for the ASIC is due to the standard cell, macros, and IP cores. During the ASIC design of million or billion gates, the area constraints and better floor planning to get the desired performance play an important role. The area optimization we can think at various design phases such as:

1. During the architecture design by describing the better strategies for different functional block interactions
2. During the RTL design by using tool-based directives and commands and using the resource sharing techniques
3. In the physical design during the floor plan stage by having strategies to place the functional blocks to minimize routing delays and area due to use of the routing resources.

5.7 Speed Requirements

The speed is another important consideration during the design of ASIC. The ASIC performance can be improved by using different speed improvement techniques. For example, consider that the processor design works at the operating frequency of 500 MHz and we have the challenge to improve the design frequency. In such circumstances, various strategies can be used during the ASIC design cycle and few of them may be

1. Having better partitioning at the sequential boundaries during the architecture and micro-architecture design.
2. Having the initial floor plan where the interdependent blocks can be placed close to each other to minimize the area and hence the routing delay and to improve the speed.
3. During the RTL design stage, use the balance register and register duplication, optimization commands to improve on the design performance. But they may affect the logic area.
4. During the RTL design, use the registered inputs and outputs to have the better performance for the design.
5. Wherever it is feasible, use the pipelining concepts and architectures.
6. If FSM designs and controllers need to be used in the design, then try to work on the control and data path synthesis for the clean timing and better performance.
7. Try to use the synchronous designs as they are faster as compared to the asynchronous designs.
8. Try to avoid the internal clock generators; instead of that, think about the clock tree and optimize the clock tree during the CTS.
9. During the routing stage, try to work on the tool-based improvement techniques as enabling tool directives can play especially important role to balance the skews.

5.8 Power Requirements

For any kind of ASIC or SOC design, the important consideration is power and the objective of the design team is to reduce the leakage and dynamic power. The power planning by considering the power constraints is done during the physical design. The power optimization techniques should be used at the different stages during the ASIC design flow.

1. Having the low-power-aware architecture for the ASIC.
2. Use the UPF at various design levels.
3. During the RTL to minimize the dynamic power, use the dedicated clock gating cells.
4. The power can be also optimized during the RTL stage by avoiding unnecessary assignments and toggling of the data values.
5. Have the better power planning and power sequencing for the multiple power domains during the physical design.
6. Have the better strategies for the power shutdown during the physical design.

For more details, refer to Chap. 7.

5.9 What Are Design Constraints?

The design constraints are basically the design rule constraints and optimization constraints. The constraints we can consider as block-level constraints, top-level constraints, and chip-level constraints.

Design Rule Constraints (DRC): These constraints we can consider as the foundry-laid rules and should be met. During the physical design, we will carry out DRC to check for whether all the foundry-laid rules meeting or not. Layout is clean indicates no DRC violations. These constraints are mainly

- Transition
- Fanout
- Capacitance.

Optimization Constraints: These constraints are used during the design and optimization phases. These constraints are mainly

- Area
- Speed
- Power.

Mainly using the Synopsys DC, we will use the area and speed constraints and will try to optimize design during the various optimization phases.

The physical design tool such as Synopsys IC Compiler uses the area, speed, power constraints to meet the constraints to have the clean chip layout. These constraints are discussed in Chap. 10.

5.10 Exercises

Few of the exercises are useful during the architecture and RTL design of the processor.

1. Design the ALU to perform the arithmetic and logic operations on the two 8-bit numbers. Consider arithmetic operations as the addition, subtraction, increment, decrement, and logic operations such as OR, AND, XOR, and complement. Sketch the logic and try to work on the area optimization strategies using the digital design concepts.
2. Find the data required time and data arrival time for the following design (Fig. 5.12)?

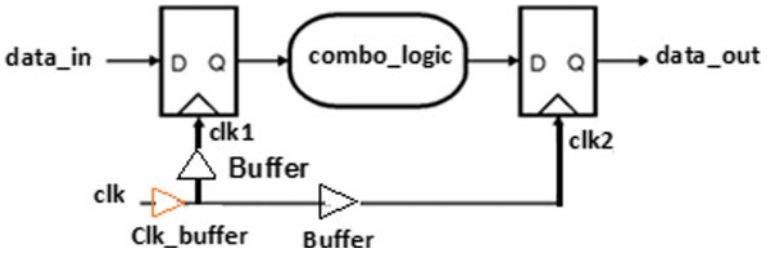


Fig. 5.12 Exercise 2

5.11 Chapter Summary

The following are few of the important points to conclude the chapter.

1. The minimum amount of time for which the data input of the flip-flop should maintain the stable value prior to arrival of the active edge of the clock is called as setup time.
2. The minimum amount of time for which the data input of the flip-flop should maintain the stable value after the arrival of the active edge of the clock is called as hold time.
3. The propagation delay of flip-flop is also called as clock (clk) to output (q), that is, t_{clktoq} delay.
4. Skew is the difference between the arrival times of clock.
5. The positive clock skew is the data and clock running in the same direction, and positive clock skew is better for the setup time but not good for hold time!
6. The negative clock skew is the data and clock running in the opposite direction, and negative clock skew is better for the hold time but not good for setup time!
7. Setup slack is the difference between the data required time and data arrival time and should be positive.
8. Hold slack is the difference between the data arrival time and data required time and should be positive.

Chapter 6
Important Considerations for ASIC Designs

The ASIC design phase of the architecture and micro-architecture design plays the important role to yield better performance for the ASIC chip. The design should have the less area, more speed, and low power that are demand of the chip. In such scenarios, we need to understand the following considerations to finalize the architecture.

1. Clocking sources
2. Clock latency and network latency
3. Single and multiple clock domain designs
4. Low power-aware architecture
5. Impact of skew on the speed
6. Clocking and reset policies
7. Synchronization and data integrity checks.

Few of these concepts are discussed in the previous few chapters, and few will be discussed in the chapter so that readers can have better understanding of the architecture and micro-architecture designs!

6.1 Synchronous Design and Considerations

As discussed in Chaps. 4 and 5 we always used the synchronous design as asynchronous designs are prone to the glitches and slower. The asynchronous clocking introduces the delays in the triggering of the unit and not recommended in the ASIC designs. Practically to understand the synchronous design, let us try to understand the logic shown in the figure. As shown, the register to register (reg to reg) path frequency is one of the deciding factors for the synchronous design.

ASIC is Application-Specific Integrated Circuits. The understanding of the important design considerations plays an important role during design phase!

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

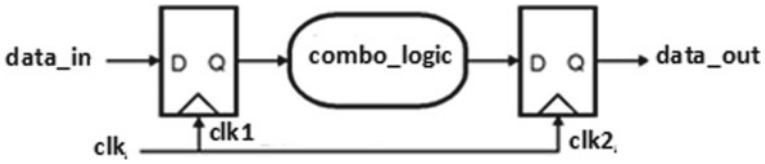


Fig. 6.1 Synchronous sequential circuit

We have the source flip-flop which launches the data, and the capture flip-flop captures the data. The launch clock is clk1, and capture clock is clk2. The maximum operating frequency for the design indicates that the frequency for which all the timing paths in the design doesn't have any kind of timing violations. The timing paths are discussed in the next session and during the constraint definition in Chap. 10.

What is requirement for correct functionality?

The destination flip-flop data should be present prior to arrival of the active edge of the clock the design will not have any kind of the setup violation and that is the requirement for the design. So, the data required time (RT) is $T - t_{su}$, and the actual data arrival time (AT) depends on the propagation delay of D flip-flop ($t_{ctoq} = t_{pff}$) and the combinational delay (t_{combo}). So, the actual data arrival time is ($t_{ctoq} + t_{combo}$), and this limits the overall frequency for the design.

What is maximum clock frequency for synchronous design?

The setup slack is difference between the RT and AT and should be positive to avoid the timing violations in the design.

$$\begin{aligned}
&\text{Setup Slack} >= 0 \\
&\text{RT} - \text{AT} >= 0 \\
&(T - t_{su}) - (t_{ctoq} + t_{combo}) >= 0 \\
&T = t_{ctoq} + t_{combo} + t_{su}
\end{aligned}$$

The flip-flop timing parameters are t_{su} as setup time, t_{ctoq} as flip-flop delay and t_{combo} as combinational delay.

where T is clock time duration and the f_{max} is maximum frequency for the design which is equal to $f_{max} = 1/T$ (Fig. 6.1).

6.2 Positive Clock Skew and Impact on Speed

As discussed in the previous chapters, the positive clock skew is better for the setup as the capture flip-flop is triggered after the buffer delay but not good for the hold.

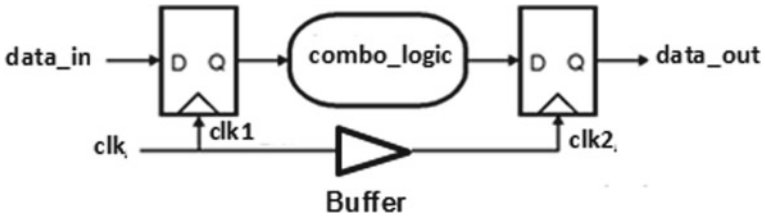


Fig. 6.2 Positive clock skew

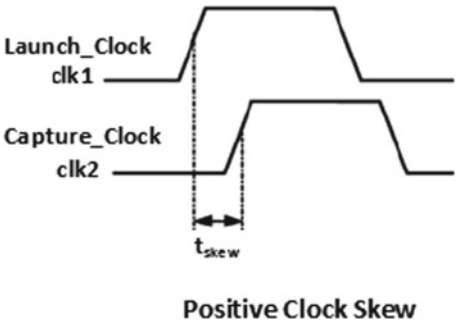


Fig. 6.3 Timing diagram for positive clock skew

The register to register path with the positive clock skew between the clk1 and clk2 is shown in Fig. 6.2. As shown, the clk2 is triggered after the t_{buffer} time delay and hence the chance of improving the clock frequency for the design (Fig. 6.3).

Consider the flip-flop timing parameters for the required technology node; the design works at the maximum clock frequency and the maximum frequency calculation is shown below.

$$\begin{aligned}
&\text{Setup Slack} >= 0\\
&\text{RT} - \text{AT} >= 0\\
&(T + t_{\text{buffer}} - t_{\text{su}}) - (t_{\text{ctoq}} + t_{\text{combo}}) >= 0\\
&T = t_{\text{ctoq}} + t_{\text{combo}} + t_{\text{su}} - t_{\text{buffer}}
\end{aligned}$$

The flip-flop timing parameters are t_{su} as setup time, t_{ctoq} as flip-flop delay t_{combo} as combinational delay, t_{buffer} as buffer delay.

6.3 Negative Clock Skew and Impact on the Speed

In the negative clock skew the source flip-flop is triggered last and the capture flip-flop is triggered first. Here, we assume that the source flip-flop is the launch flip-flop and capture flip-flop as destination flip-flop.

The register to register path with the negative clock skew between the clk1 and clk2 is shown in Fig. 6.4. As shown, the clk1 is triggered after the t_{buffer} time delay, and hence, it reduces the overall operating frequency for the design (Fig. 6.5).

Consider the flip-flop timing parameters for the required technology node. The design works at the maximum clock frequency, and the maximum frequency calculation is shown below.

$$\begin{aligned}
&\text{Setup Slack} >= 0\\
&\text{RT} - \text{AT} >= 0\\
&(T - t_{\text{buffer}} - t_{\text{su}}) - (t_{\text{ctoq}} + t_{\text{combo}}) >= 0\\
&T = t_{\text{ctoq}} + t_{\text{combo}} + t_{\text{su}} + t_{\text{buffer}}
\end{aligned}$$

The flip-flop timing parameters are t_{su} as setup time, t_{ctoq} as flip-flop delay and t_{combo} as combinational delay, t_{buffer} as buffer delay.

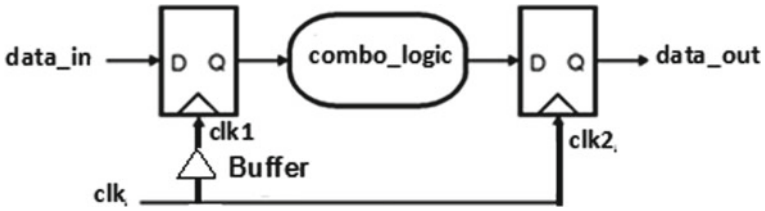


Fig. 6.4 Negative clock skew

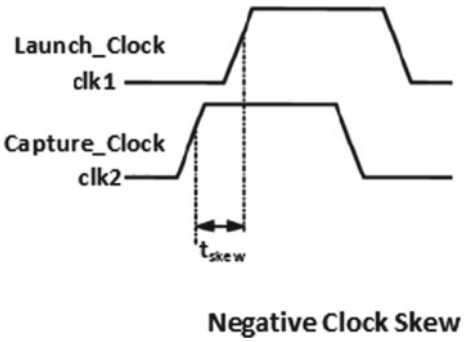


Fig. 6.5 Timing diagram for negative clock skew

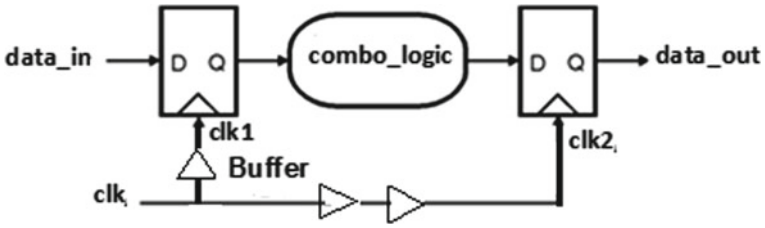


Fig. 6.6 Buffers in the clock network

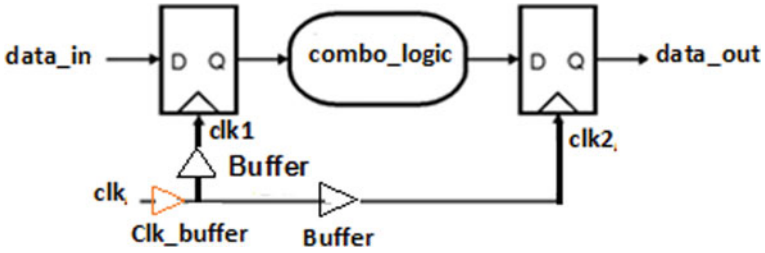


Fig. 6.7 Clock network latency and buffers

6.4 Clock and Network Latency

The clock network introduces the latencies in the design, and during the physical design, it is essential to optimize for the clock tree so that the distribution of clock can be with uniform clock skew.

If the clock tree strategies are not efficient and tool is not able to optimize for the clock network, then due to issues in the clock propagation, the design may have the issues due to timing violations.

It is real nightmare for the physical design team to understand the fix issues during the signoff static timing analysis (Figs. 6.6 and 6.7).

6.5 Timing Paths in the Design

Let us try to understand the timing paths in the synchronous design. Consider the design shown in Fig. 6.8 which has mainly four timing paths, and they are named as

1. Input to reg path
2. Reg to output path
3. Reg to reg path
4. Input to output path.

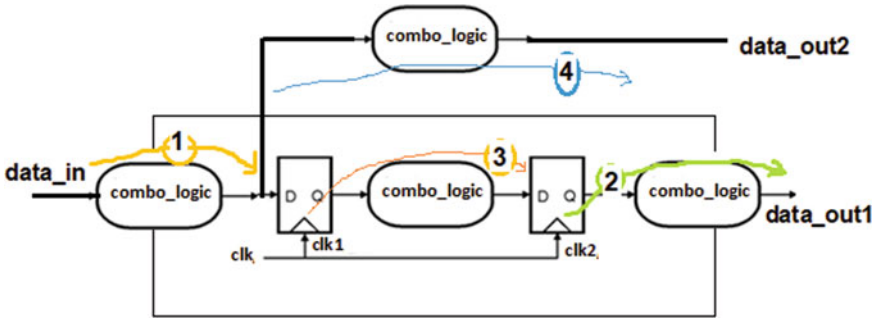


Fig. 6.8 Synchronous design

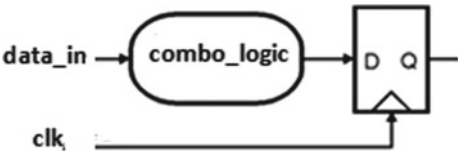


Fig. 6.9 Input to reg path

To identify the timing paths in the design, the designer should know the start point and end point.

Start point: The clock input of the sequential element (clk), data inputs of the sequential design is treated as the start point, and the tool algorithm identifies initially the start points for the design and then end points.

End Point: The end point of the design is the output of the sequential element or data input of the sequential element D flip-flop (D).

6.5.1 Input to Reg Path

It marked as path 1 in the figure, and it is from the input port data_in of the design to the D input of the sequential element (Fig. 6.9).

6.5.2 Reg to Output Path

It marked as path 2 in the figure, and it is from the clock pin clk2 of the flip-flop to the data_out1 of the sequential element (Fig. 6.10).

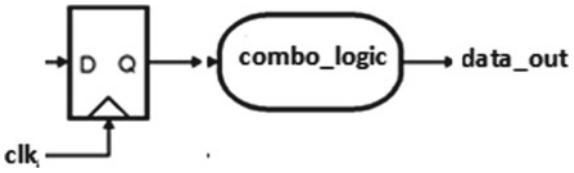


Fig. 6.10 Reg to output path

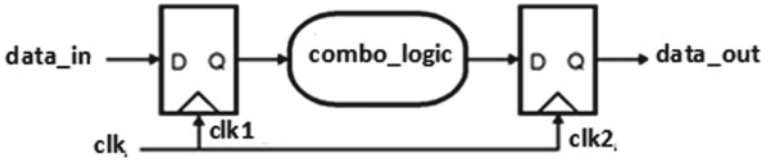


Fig. 6.11 Reg to reg path



Fig. 6.12 Combinational path

6.5.3 Reg to Reg Path

It marked as path 3 in the figure, and it is from the clock pin clk1 of the flip-flop to the data input of the D flip-flop that is sequential element 2 (Fig. 6.11).

6.5.4 Input to Output Path

It is unconstrained path and also called as combinational path. It is marked as path 4, and it is from data_in of the design to the data_out2 of the design (Fig. 6.12).

6.6 Frequency Calculations

To find the maximum operating frequency for the design Fig. 6.13, consider the timing parameters of flip-flop as

$$t_{\text{ctoq}} = 2\,\text{ns}$$
$$t_{\text{combo}} = 2\,\text{ns}$$

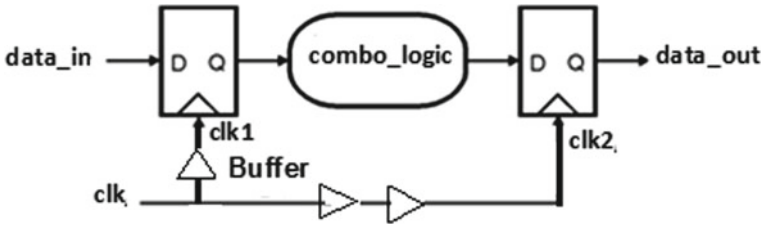


Fig. 6.13 Design example-1

$$t_{su} = 1\,\text{ns}$$
$$t_{buffer} = 1\,\text{ns}$$
$$t_{h} = 0.5\,\text{ns}$$

For the design shown below, the data arrival time is equal to

$$\begin{aligned} \text{AT} &= t_{ctoq} + t_{combo} + t_{buffer} \\ &= 2\,\text{ns} + 2\,\text{ns} + 1\,\text{ns} = 5\,\text{ns} \end{aligned} \tag{6.1}$$

The data required time is (RT)

$$\begin{aligned} \text{RT} &= T - t_{su} + 2 * t_{buffer} \\ &= T - 1\,\text{ns} + 2 * 1\,\text{ns} \\ &= T + 1\,\text{ns} \end{aligned} \tag{6.2}$$

Setup slack is RT-AT and should be greater than or equal to zero.

Using Eqs. 6.1 and 6.2, we get the T as 4 ns; therefore, the operating frequency for the design is 250 MHz.

For the design shown in Fig. 6.14, the clock buffer delay is common and hence can be considered as clock latency from the source and can be specified in the constraints so data arrival time is equal to

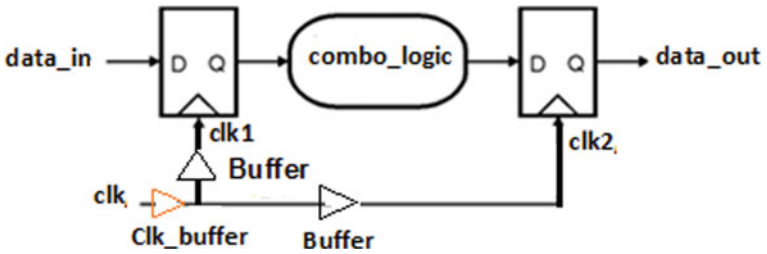


Fig. 6.14 Design example-2

$$\begin{aligned} \text{AT} &= t_{\text{ctoq}} + t_{\text{combo}} + t_{\text{buffer}} \\ &= 2\,\text{ns} + 2\,\text{ns} + 1\,\text{ns} \\ &= 5\,\text{ns} \end{aligned} \tag{6.3}$$

The data required time is (RT)

$$\begin{aligned} \text{RT} &= T - t_{\text{su}} + t_{\text{buffer}} \\ &= T - 1\,\text{ns} + 1\,\text{ns} \\ &= T \end{aligned} \tag{6.4}$$

Setup slack is RT-AT and should be greater than or equal to zero.

Using Eqs. 6.3 and 6.4, we get the T as 5 ns; therefore, the operating frequency for the design is 200 MHz.

6.7 What Is On-Chip Variations

Practical ASIC designs will have the on-chip variation that is also called as PVT variation where P is process, V is voltage and T is temperature.

- **Process**: This specifies the process, and in simple words, it indicates the oxide thickness, device length and other parameters as the doping concentrations.
- **Voltage:** Every chip operates at specific voltage, and this parameter indicates the voltage level at which chip operates.
- **Temperature:** This parameter indicates the operating temperature for the chip.

The variation of the operating parameters PVT is responsible to have the different operating conditions such as

1. Min
2. Max.

That is, in other words, we need to work on the best case, worst case and typical case scenarios during design synthesis and timing analysis.

The operating conditions are defined in the library, and they indicate the PVT and delays. The library is characterized for the specific operating conditions. For the on-chip variation that is for another operating condition, the derate factor can be used during the synthesis and timing analysis.

Library design and cell characterization is out of scope as per objective of this book is concern, but the library developers can specify the number of operating conditions in the library.

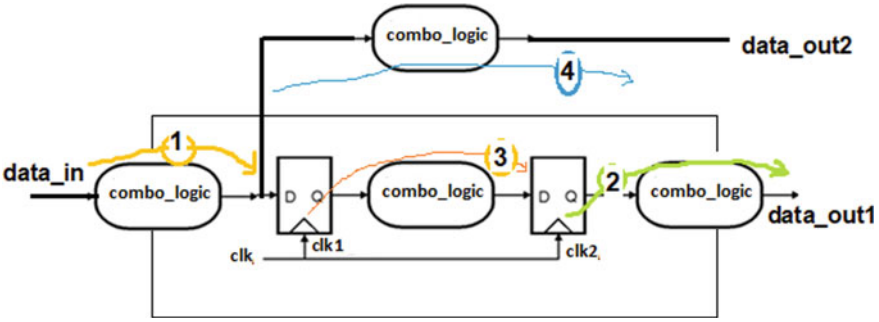


Fig. 6.15 Exercise-1

6.8 Exercises

1. Consider the design shown in the figure and find out maximum frequency for the design. Consider timing parameters as? (Fig. 6.15)

$$t_{\text{ctoq}} = 2\,\text{ns}$$
$$t_{\text{combo}} = 2\,\text{ns}$$
$$t_{\text{su}} = 1\,\text{ns}$$
$$t_{\text{buffer}} = 1\,\text{ns}$$
$$t_{\text{h}} = 0.5\,\text{ns}$$

6.9 Chapter Summary

The following are important points to conclude the chapter.

1. Use the synchronous design as asynchronous designs are prone to the glitches and slower.
2. In the negative clock skew, the source flip-flop is triggered last and the capture flip-flop is triggered first.
3. In the positive clock skew, the source flip-flop is triggered first and the capture flip-flop is triggered last.
4. Four timing paths are named as

- Input to reg path
- Reg to output path
- Reg to reg path
- Input to output path

5. ASIC designs will have the on-chip variation that is also called as PVT variation where P is process, V is voltage and T is temperature.

Chapter 7
Multiple Clock Domain Designs

Consider the ASIC design scenario in which the requirement is to have the different blocks for the complex designs, and few of these blocks are

1. Processor
2. Memories
3. Floating-point engine
4. Memory controllers
5. Bus interfaces
6. High-speed interfaces.

Consider that the processor and memories working on the operating frequency of 500 MHz and the floating-point engine with the memory controller works at the operating frequency of the 666.66 MHz. The bus interfaces and high-speed interfaces working on the operating frequency of 250 MHz that is the design have the multiple clocks and are treated as multiple clock domain design.

7.1 General Strategies for Multiple Clock Domain Designs

As discussed earlier, the issue in the multiple clock domains is while transferring of the data and control signals, and it has impact on the data integrity. The following strategies can be helpful during the ASIC design phase.

1. Try to have strategies for the data and control path optimization.
2. Try to have the strategy to define are create the multiple clock domain groups.
3. Try to have strategies to deploy the synchronizers while passing the control signals between the multiple clock domains.

ASIC chips have many clocks, and clock domain managament is important during the ASIC design cycle.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

4. Try to use the data path synchronizers using FIFO and buffers to improve the data integrity.

The subsequent sessions discuss important issues and the strategies and their use during the multiple clock domain designs.

7.2 What Are Issues in the Multiple Clock Domain Design

If we consider the moderate gate count design or the processor core which works using the single clock and the design, it may have the timing violations during the layout stage due to additional interconnect delays. But such kind of designs may meet the timing and performance through the architecture, RTL, synthesis, and the tool-based optimization tweaks.

Now consider the design which demands the need of the multiple clocks as shown in the Figure and let us try to understand the issues in the design!

1. Due to multiple clock domains, the data integrity is major issues and design needs to go through the data integrity checks.
2. The flip-flops at the boundary of the clock domains without the use of synchronizers will have the metastability issues due to setup and hold violations.
3. The design will have timing violations, and it becomes difficult to force the sequential circuit output into valid legal states.

Let us understand the above using the sequential circuit which has multiple clock domains. Due to phase difference between the arrival of the clk1 and clk2, the flip-flop in the second clock domain will have the setup and hold violations that is the flip-flop output data_out will be metastable. The reason being the q output from the clock domain 1 can change during the setup and hold window of the clk2 active edge, and hence the data_out will be forced into the illegal state that is a metastable state (Fig. 7.1).

The timing sequence is shown in Fig. 7.2.

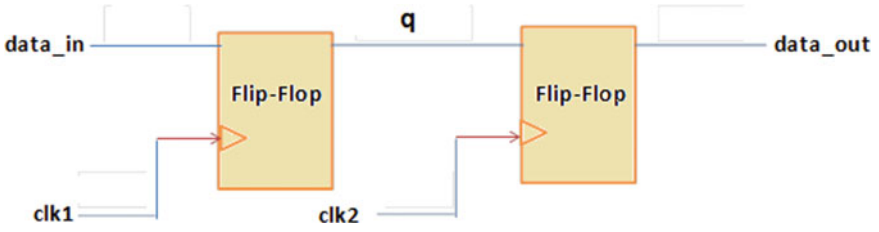


Fig. 7.1 Multiple clock domain concept

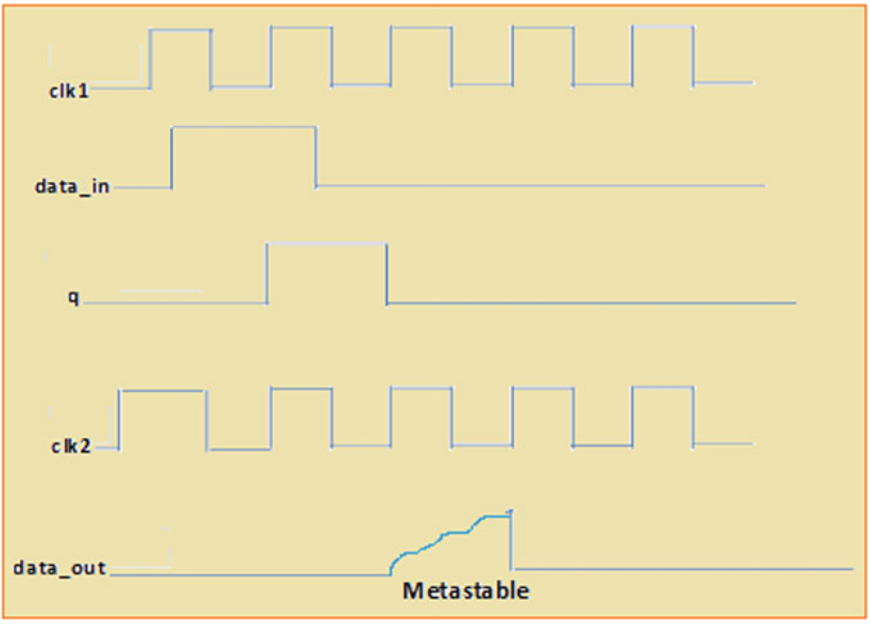


Fig. 7.2 Metastable output

7.3 Architecture Design Strategies

Consider the design which has three clock domains as shown in Fig. 7.3, and Table 7.1 describes information about the clock domains operating at various clock frequencies. Refer Chap. 9 for more information on the architecture and micro-architecture design.

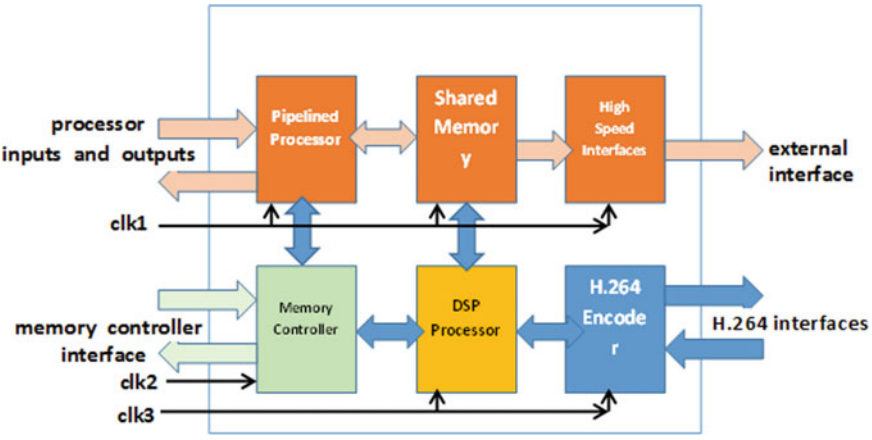


Fig. 7.3 Multiple clock domain architecture

Table 7.1 Multiple clock domain clock groups

Clock domain control	Frequency in MHz	Description
clk1	500	The clock domain one operating at frequency of 500 MHz
clk2	666.66	The clock domain two operating at frequency of 666.66 MHz
clk3	250	The clock domain three operating at frequency of 250 MHz

As a designer or architect, we need to think about the overall data integrity checks for the multiple clock domain design and need to have the clean timing for the data path and control path.

By considering this, we need to deploy the synchronizers to carry the data between the multiple clock domain designs. The synchronizers such as level, mux, and pulse are useful while passing the control signal between the multiple clock domain designs. The asynchronous FIFO can be used as synchronizer to carry the data between the clock domain and used in the data path.

Following are few of the guidelines which we should use during the multiple clock domain designs to eliminate the CDC errors, where CDC is clock domain crossing

1. **Avoid Metastability**: While passing the control signal information, use the registered output as this is useful to avoid the glitches and hazards. The multiple transitions during single clock cycle can be avoided by using the registered output logic while passing the control signal. Metastability blocking logic is shown in Fig. 7.4.
2. **Use of MCP**: Multicycle path formulation is highly recommended to avoid the metastability issue while passing the data and control signal information between the multiple clock domains. In the MCP, the strategy is to create the control and data pairs to pass the multibit data with the single bit control signal from sending clock domain to receiving clock domain. The control information can

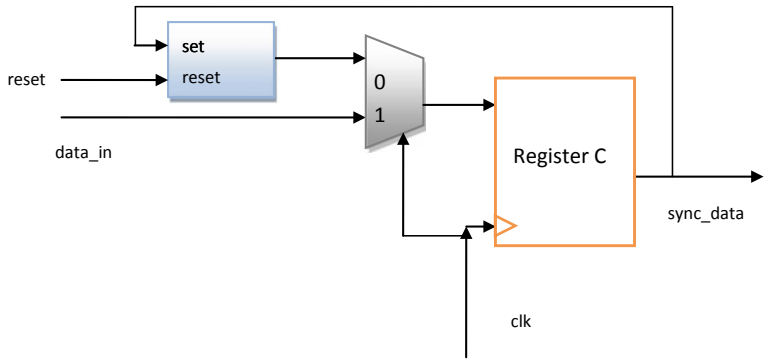


Fig. 7.4 Metastability blocking logic

be sampled in the receiving clock domain by using the pulse synchronizer, and data can be passed to the receiving clock domain with or without synchronizers. This technique is highly effective as the data can maintain the stable value for multiple cycles and can be sampled in the receiving clock domain by using the synchronized signal generated by using pulse synchronizer. Across the clock domain crossing boundaries, following are key points need to be considered.

(a) Control signals must be synchronized using the multistage synchronizers.
(b) Control signals should be free of hazards and glitches.
(c) There should be single transition across clock boundaries.
(d) Control signal should be stable for at least single clock cycle.
The MCP formulation is shown in Fig. 7.5.

3. **Use FIFO**: The effective technique to pass the multibit control signals or data information is using asynchronous FIFO. In this technique, the sending clock domain writes the data into FIFO memory buffer when the FIFO is not full and receiving clock domain reads the data from the FIFO buffer when the FIFO is not empty.

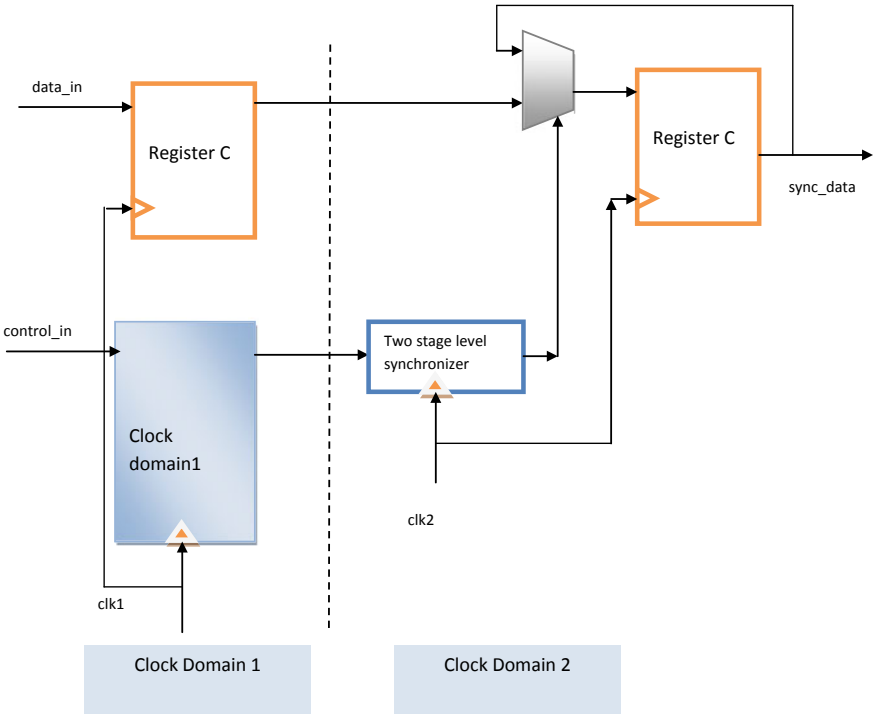


Fig. 7.5 MCP formulation

4. **Use Gray Code Counters**: In most of the ASIC designs with clock domain crossing (CDC), it is essential to pass the counter value across the clock domains. If binary counters are used to exchange the data at the clock domain boundaries, then due to toggling of one or more than one bit the data exchange can be error prone. In such scenarios, it is recommended to use the gray code counters to pass the data across the clock boundaries. In the receiver clock domain, use the gray to binary code converter to get the original data back!
5. **Design Partitioning**: While designing the logic for the multiple clock domain design, partition the design by using the clock groups.
6. **Clock Naming Conventions**: It is recommended to use the clock naming conventions to identify the clock source for better readability. The naming conventions for the clock should be supported by the meaningful prefix. For example, for sending clock domain, use clk_s, and for receiving clock domain, use clk_r.
7. **Reset Synchronization**: For the ASIC designs, it is highly recommended to use the reset synchronizers.
8. **Avoid Hold Time Violations**: To avoid the hold time violations, it is recommended to have close look at the architecture and have a strategy while passing the stable data between the multiple clock cycles.
9. **Avoid Loss of Correlation**: Across the clock domain boundary, there are several ways due to which loss of correlation can occur. Few of them are

   (a) Multiple bits on the bus
   (b) Multiple handshake signals
   (c) Unrelated signals

   To avoid this, use the clock intent verification technique as these techniques will ensure the passing of multibit signal across the clock boundaries.

7.4 Control Path and Synchronization

The section discusses various synchronizers and strategies to use them during the ASIC design.

7.4.1 Level or Multiflop Synchronizer

The control signals passing between the multiple clock domains mainly from the faster to slower clock domains will experience the timing failure, and the design will have timing violations. So, the better strategy during the architecture design is to identify the interface boundaries for the multiple clock domain design and then have the strategy in the RTL design to use the synchronizers.

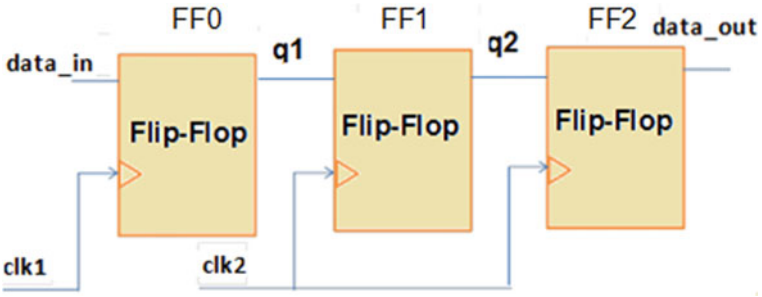


Fig. 7.6 Two-stage level synchronizer in the control path

The issue of metastability can be addressed by deploying the level synchronizers (may be y\using two or three flip-flops) while passing the control signals between the multiple clock domains. Figure 7.6 uses the two-stage level synchronizer logic.

As shown in Fig. 7.6, the level synchronizer is used to pass the control single q1 from clock domain 1 to clock domain 2. The main design strategy is to pass the valid output q1 to second clock domain. The level synchronizer is deployed in the second clock domain to sample the output q1. The input flip-flop in the second clock domain will be metastable due to violation of the setup or hold time, and this should be ignored by setting up the EDA tool attributes. The output data_out is valid data, and the design has the latency of two clocks due to use of the synchronizer.

The timing sequence for design shown in Fig. 7.6 is described below (Fig. 7.7).

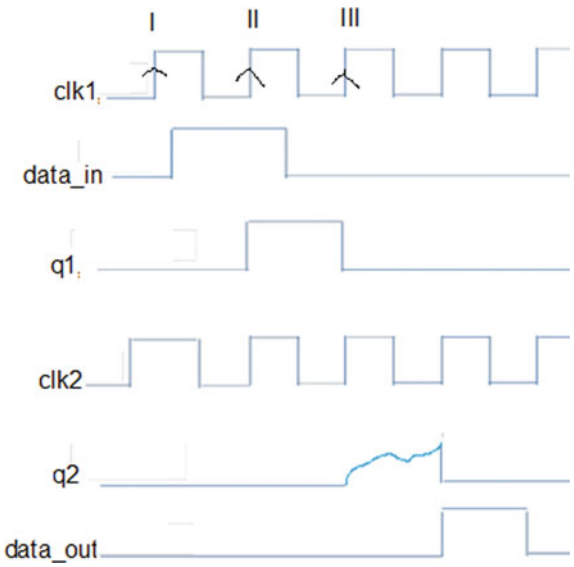


Fig. 7.7 Timing sequence with use of the two-stage synchronizer

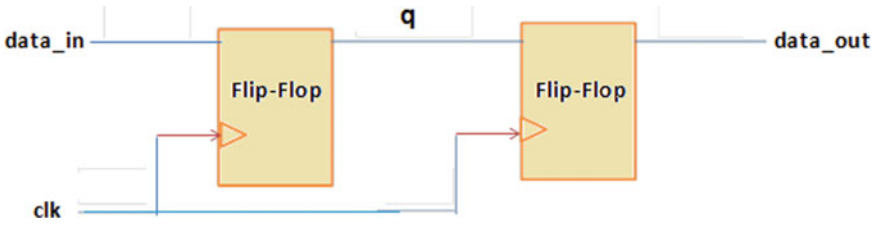


Fig. 7.8 Level synchronizer

As shown in Fig. 7.7, q1 is output from the first clock domain. On the rising edge of 'clk2,' the output of flip-flop FF1 that is q2 will go to the metastable state due to violation of either setup or hold time. But the flip-flop FF2 output that is data_out during the next subsequent clock edge is valid output. Set the false path using the command

```
set_false_path –from FF0/q –to FF1/q
```

The level synchronizers using the two flip-flops are shown in Fig. 7.8 and can be deployed in the design. The better strategy is to have the RTL description of the level synchronizers as separate module during the RTL design. The latency introduced depends upon the number of flip-flops required to drive the output into the valid legal state.

The piece of the RTL description is described below

```
always @ (posedge clk)
begin
        q<=data_in;
        data_out<=q
end
```

In the ASIC design, the issue of data integrity occurs when the control information needs to be passed from faster clock domain to the slower clock domain. The issue is due to non-converging of the legal states of the flip-flop outputs while passing the control signals from clock domain 1 to clock domain 2.

The issue of sampling the data from faster clock domain to the slower clock domain can be resolved by using pulse stretcher. The level to pulse generator which works on the positive clock edge is shown in Fig. 7.9.

Another mechanism which is handshake of signals can be used to have the data convergence.

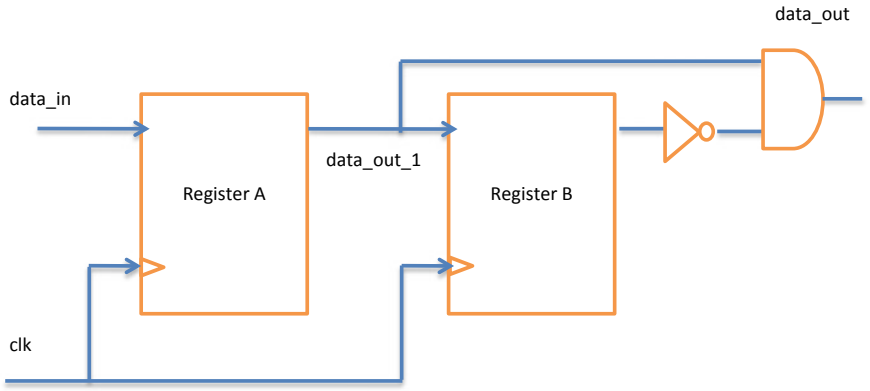


Fig. 7.9 Level to pulse conversion

As shown in Fig. 7.10, the sampled signal in the clock domain 2 is feedback as a handshaking signal to clock domain 1. This handshake mechanism is like acknowledgement or notification to the faster clock domain 1 that the control signal passed by the faster clock domain is successfully sampled by the slower clock domain. In most of the practical scenarios, this kind of mechanism is used, and even the faster clock domain can send another control signal after receiving the valid notification or acknowledgement signal from the slower clock domain.

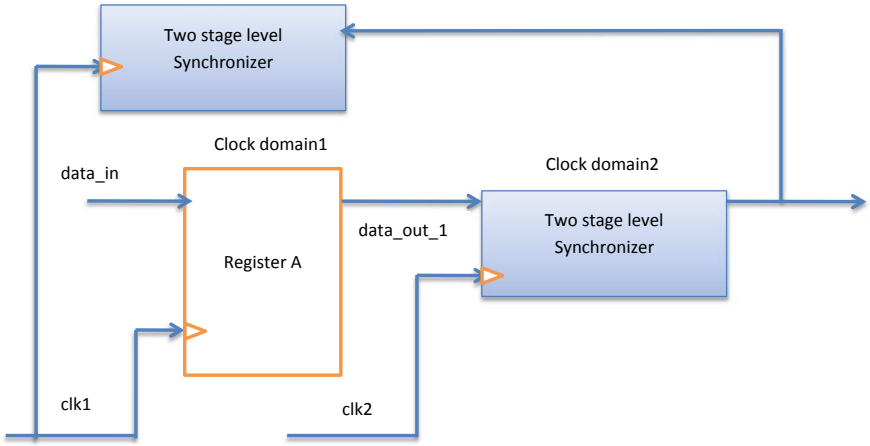


Fig. 7.10 Handshake mechanism for control signals

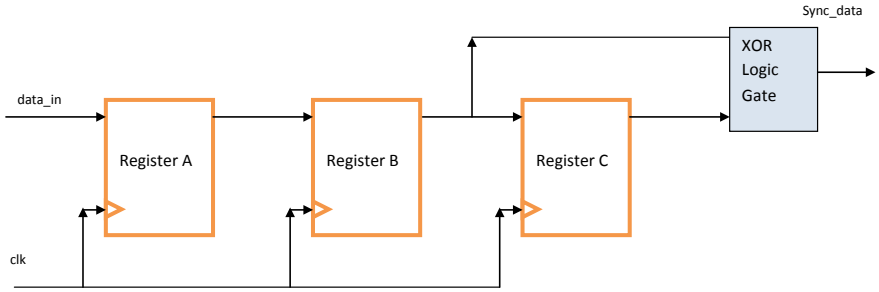


Fig. 7.11 Pulse synchronizer

7.4.2 Pulse Synchronizers

This type of synchronizer uses the multistage level synchronizer where the output of two-stage level synchronizer is sampled by the output flip-flop. This kind of synchronizer is also named as toggle synchronizer and used to synchronize the pulse generated in the sending clock domain into the destination clock domain. While passing the data from faster clock domain to the slower clock domain, the pulse can be skipped if two-stage level synchronizers are used. In such scenarios, the pulse synchronizers are efficient and useful. The pulse synchronizer diagram is shown in Fig. 7.11.

7.4.3 MUX Synchronizer

Use the pair of the data and control signals while sending the information from clock domain 1 to clock domain 2. Use the multiple bit data and use the single bit control signal. At the receiving end depending on the ratio of the sending clock and receiving clock, use the level or pulse synchronizer to generate the control signal for the multiplexer. This technique is like the MCP and effective if the data is stable for multiple clock cycles across the clock boundaries. The diagram is shown in Fig. 7.12.

7.5 Challenges in the Multiple Bit Data Transfer

Passing multiple control signals between the multiple clock domains is one of the important challenges. The issue is the different time of the arrival of these control signals If the arrival of these control signal is not managed properly, then the real issue is due to the skew. Consider the scenario shown in Fig. 7.13, where ‘enable’ ‘load_en’ and ‘ready’ need to be passed from one of the clock domains to another clock domain. In such scenario, if independent level synchronizers are used, then

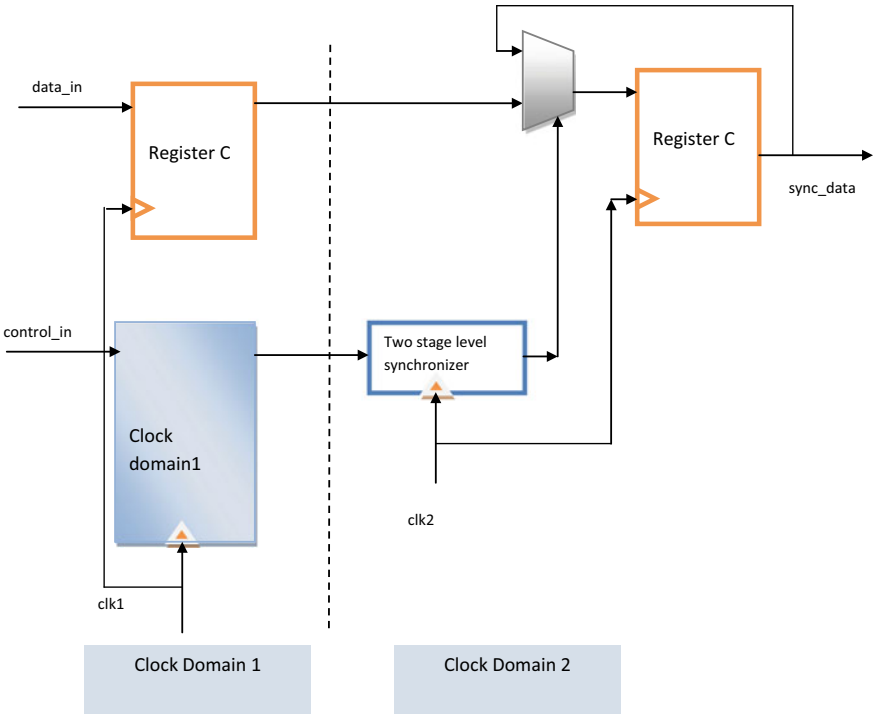


Fig. 7.12 Mux synchronizer

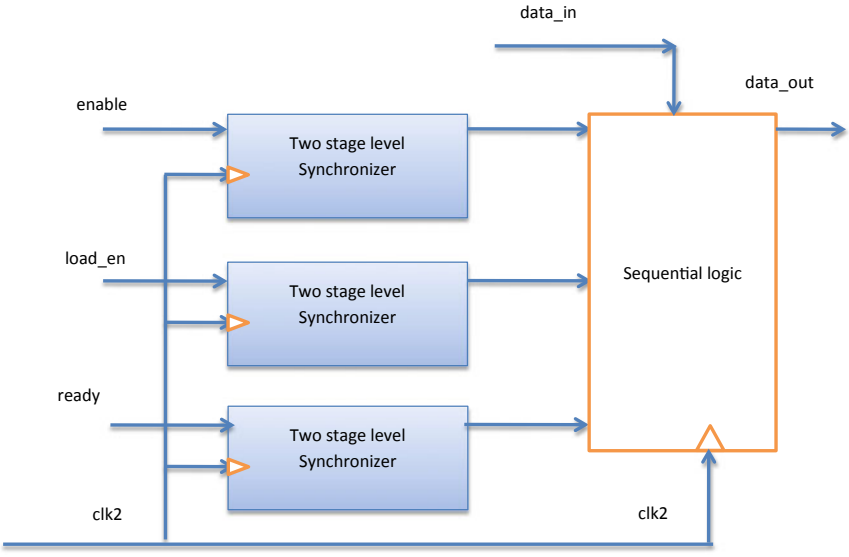


Fig. 7.13 Sampling of the multiple signals in the receiver clock domain

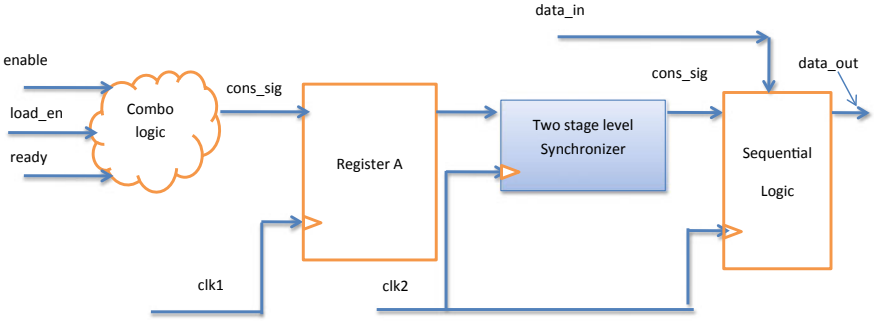


Fig. 7.14 Consolidated control signal passing in the multiple clock domain

there might be synchronization failure at the receiving end due to skew (different arrival time of these signals).

Consider that one of the control signal, for example, enable arrives late, then there may be synchronization failure in the control path, and to avoid this group these three control signals and try to pass the common signals between the clock domain. The strategy is shown in Fig. 7.14.

7.6 Data Path Synchronizers

The techniques used to pass the multiple bits of the data between the clock domains are

1. Handshaking mechanism
2. FIFO memory buffers.

7.6.1 Handshaking Mechanism

Use of the handshake mechanism is one of the techniques useful while passing of the multibit signals between the clock domain. Consider Fig. 7.15 as shown, the transmitter operates at clk1, and receiver operates at clk2. The data can be passed from transmitter to receiver.

Receiver clock domain can generate the handshake signals such as data valid and device ready. So, the purpose is to notify the transmitter that valid data is available on the bus and the device is not ready to receive the new data.

Handshake Signal Datavalid It is active high handshake signal from clock domain 2 and indicates that the data transmitted is valid data and receiver needs few clocks to sample this data. The clock latency while transferring the data is dependent upon

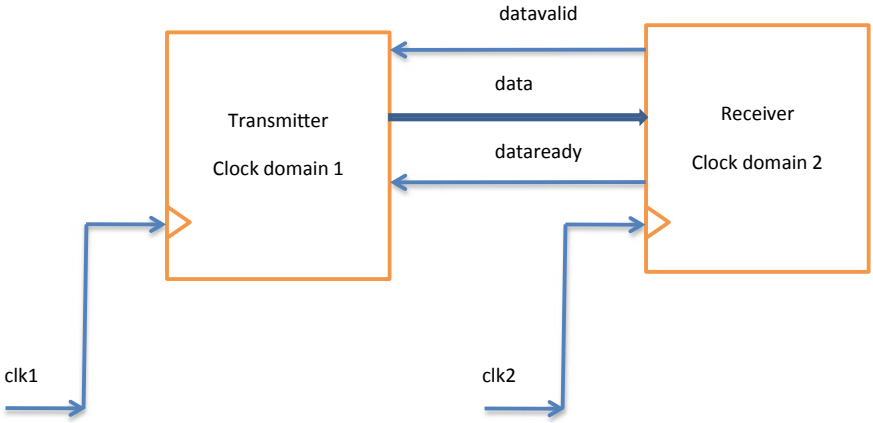


Fig. 7.15 Block diagram for handshake mechanism

the number of flip-flops used in the synchronizer, and the poor latency is one of the biggest disadvantages of the handshake mechanism.

Handshake Signal Deviceready It indicates that the receiver is ready to accept the new data when the data valid is de-asserted and the device ready can go high to notify transmitter that place new data on the data bus.

If we have the FSM controllers in the multiple clock domains, then design the architecture to establish synchronization by using the request and acknowledge (ack) signals. For the FSM control, handshake mechanism is shown in Fig. 7.16.

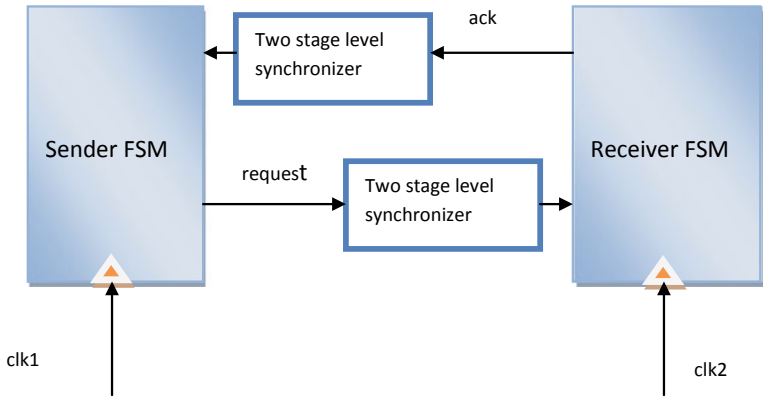


Fig. 7.16 FSM handshaking mechanism

7.6.2 FIFO Synchronizer

FIFOs are useful as data path synchronizer is used to exchange the data between multiple clock domains. The sender clock domain or transmitter clock domain can write the data into the FIFO memory buffer using write_clk if FIFO memory buffer is not full and receiver clock domain can read the data by using the read_clk if FIFO memory buffer is not empty (Fig. 7.17).

The FIFO consists of the following blocks

1. **Memory**: memory buffer
2. **Write Clock Domain**: write domain logic which is working on write_clk
3. **Read clock Domain**: read domain logic which is working on read_clk
4. **Flag Logic**: empty and full flag generation logic.

The FIFO with the associated logic blocks is shown in Fig. 7.17.

How to get the depth of the FIFO?

Consider the write clock domain works at operating frequency of 250 MHz and read clock domain at 100 MHz and no latency then to transfer the burst of the 50 bytes use following calculations

1. *Write clock time: T1 = 1/250 MHz = 4 ns.*
2. *The Time Required to Write Burst of 50 Bytes of data = 4 ns*50 = 200 ns*
3. *Read Clock Time: T2 = 1/100 MHz = 10 ns*
4. *Number of Reads with 10 ns = 200 ns/10 ns = 20*
5. *Depth of FIFO = 50–20 = 30 Bytes.*

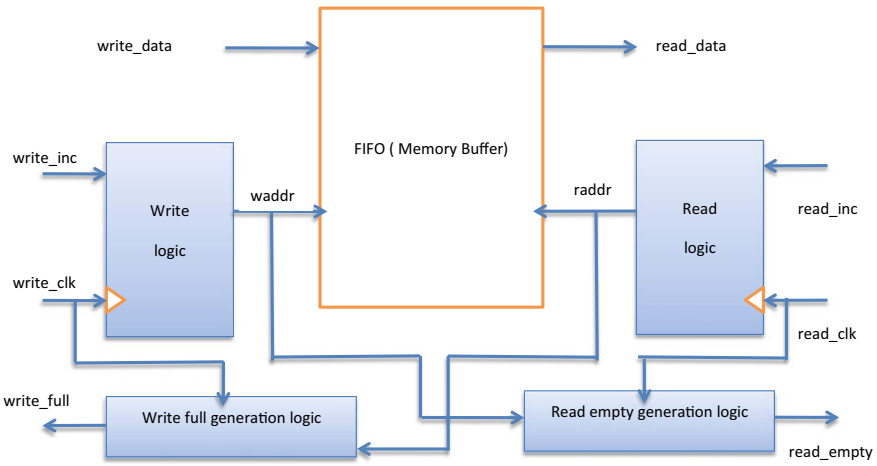


Fig. 7.17 Block diagram of FIFO

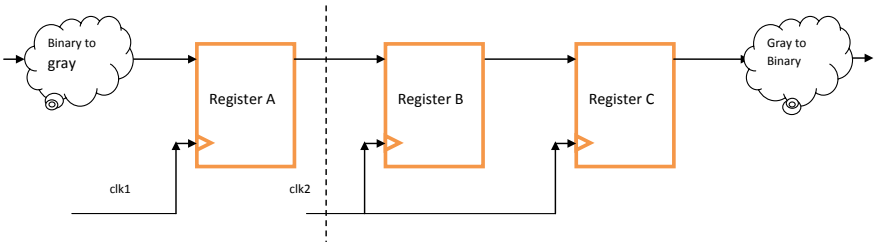


Fig. 7.18 Gray encoding technique

If read and write latency are specified, then try to modify the above steps to get the depth of FIFO.

7.6.3 Gray Encoding

While passing the multiple bit of the data or control signals, it is essential to use the gray encoding technique as this technique guarantees about the only one-bit change in two successive numbers.

For example, if 4-bit binary data needs to be passed between the multiple clock domains, then one or more than one bit toggles and hence more power is required and the chance of error. So to avoid this and to improve the performance, use the binary to gray code converter logic in the transmitter or sender clock domain. This guarantees only one-bit change across the clocking boundary. To get the original binary data in the receiver clock domain, use the gray to binary code converter. The technique is shown in Fig. 7.18.

7.7 Summary and Future Discussions

The following are few important points to conclude the chapters.

1. Deploy the data path synchronizers while passing the data between the multiple clock domains.
2. Deploy the control path synchronizers while passing the control signals between the multiple clock domains.
3. Multicycle path formulation is highly recommended to avoid the metastability issue while passing the data and control signal information across the clock domains.

4. The common and effective technique to pass the multibit control or data information is the use of asynchronous FIFOs.
5. For multibit control signal passing between the multiple clock domains, use the techniques by grouping these signals to avoid the skew due to different arrival time.

Chapter 8
Low Power Design Considerations

The ASIC design constraints are area, speed, and power. The issues related to the area and speed we have already discussed in the previous few chapters, and in the subsequent section we will discuss about the low-power designs. Following are few of the important goals of the ASIC designer to optimize for the power

1. Have the design strategies using the low-power cells.
2. Plan for the overall power requirements and have the better power planning during the physical design.
3. Try to optimize for the leakage and dynamic power using various techniques during power optimization.
4. Use the UPF at various stages with the power compilers.
5. Have the strategies in place to have the power sequencing and power shutdowns.

8.1 Introduction to Low Power Design

For the ASIC designs, the power optimization is especially important and overall power analysis and understanding play important role. To have the power optimization, the team works on the strategies to have the low-power design architecture. Power is basically dependent on the voltage, and during this decade the technology node has shrunk enough and hence the requirement of the core and IO voltage has reduced.

The power dissipation for the cell is $p = (1/2) * C_s * V^2 * f$. The power dissipation for any standard cell is directly proportional to the stray capacitance (C_s), applied voltage (V), and the frequency. To optimize for the power, we should have the lower voltage, lower capacitance, and low frequency. Practically, ASIC chips work at the higher frequencies, and reducing frequency is not the objective! There is always trade-off between the speed and power, and the architecture design should consider

Low-power aware designs and architecture are the real requirement during ASIC design.

© Springer Nature Singapore Pte Ltd. 2021

V. Taraate, *ASIC Design and Synthesis*,

the speed and frequency requirements to have the better architecture in place which can have the desired speed and the low power.

So, the power we consider is the leakage and dynamic power mainly. The primary source of power dissipation in CMOS is leakage current. The leakage current is summation of the cell leakage current and is state dependent.

$$\mathrm{P_{leakage}} = \sum \text{Cell Leakage}$$

where cell leakage can be computed by using the library cell leakage, and it is state dependent.

The dynamic power is defined as addition of the summation of the cell dynamic power and summation of power dissipated due to wires. The following are the few equations which describes the leakage and dynamic power.

$$\mathrm{P_{dynamic}} = \sum \text{Cell dynamic power} + \sum \frac{1}{2} * \mathrm{Cl} * V * V * \mathrm{Tr}$$

where the Cl is the capacitive load at pin or net, V is voltage level, and Tr is toggle rate.

Having the low-power design architecture is useful to have the low-power management and to have the long battery which is the real requirement. It is expected that the ASICs and devices should be of lightweight, small, cool, and even they should have the long battery life.

In simple words, let me describe this in the block diagram. We can imagine following to have the low-power aware architecture

1. Multiple power domains
2. Power sequencer and scheduling algorithms
3. Different cells such as
   (a) Level shifter
   (b) Isolation cells
   (c) Retention cells.

The basic architecture is described in Fig. 8.1. As shown, the architecture has the overall power management using the power sequencer/schedular and used to control the power domain I and power domain II.

8.2 Sources of Power

As discussed in the previous section, the power dissipation for the cell is $p = (1/2) * C_s * V^2 * f$. The power dissipation for any standard cell is directly proportional to the stray capacitance (C_s), applied voltage (V), and the frequency.

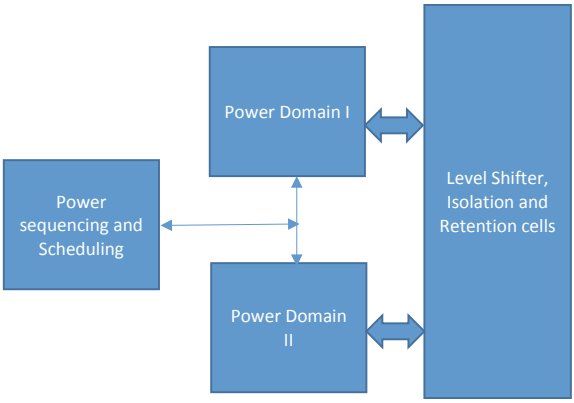


Fig. 8.1 Low-power design architecture

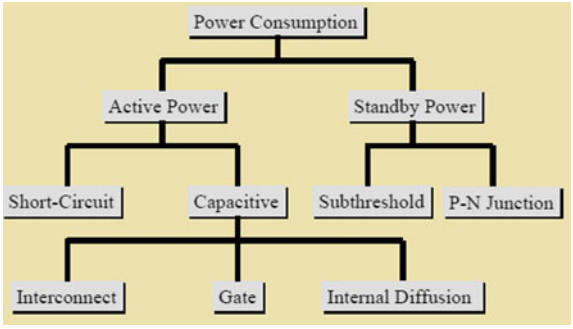


Fig. 8.2 Sources of power consumption

The sources of power consumption in CMOS are described in Fig. 8.2.

Following are few of the important points to understand the power dissipation in the CMOS

1. The power dissipation for any CMOS cell is function of the switching activity, capacitance, voltage, and the structure of transistor. So power is described as

$$\text{Power} = P_{\text{switching}} + P_{\text{short-circuit}} + P_{\text{leakage}}$$

2. The total power for any CMOS cell is summation of the dynamic and leakage power.
3. Dynamic power is summation of the switching power and short-circuit power.
4. The short-circuit power dissipation is due to the gate switching state, and it is due to the short circuit between the supply voltage and ground. The following equation describes the switching and short-circuit power

Table 8.1 Percentage power saving

Design abstraction stage	% Power saving
System design and architecture	70–80
Behavioral design	40–70
RTL design	25–40
Logic design	15–25
Physical design	10–15

$$P_{switching} = \alpha * f * C_{eff} * V_{dd} * V_{dd}$$

where α is switching activity, f is switching frequency, C_{eff} is effective capacitance, and V_{dd} is supply voltage.

$$P_{short\text{-}circuit} = I_{sc} * V_{dd} * f$$

where I_{sc} is short-circuit current during switching, f is switching frequency, and V_{dd} is supply voltage.

5. Dynamic power can be reduced by reducing the switching activity, clock frequency (it reduces the design performance), also by using the capacitance, and the supply voltage. If faster slew cells are used, then it consumes the less dynamic power and hence cell selection is important in reduction of the dynamic power.
6. Leakage power is given by the following equation, and it is function of the supply voltage V_{dd}, the switching threshold voltage V_{th}, and size of transistor.

$$P_{leakage} = f\left(V_{dd}, V_{th}, \frac{W}{L}\right)$$

In the above equation, the W is width of transistor and L is length of transistor.

Powers-saving opportunities at the different design phases are listed in Table 8.1.

8.3 Power Optimization During the RTL Design

As discussed earlier during the RTL design, the power can be optimized by 25–40% using various techniques and strategies. The section discusses the low-power design technique.

1. **Modeling and power estimation**: For the low-power design and the management of power for any SOC, it is essential to prepare the library models with the required power data. It is required to develop the transistor-level models for the custom blocks. The common practice in the SOC design at the RTL level is use of power